A.1. Definition Modules.

This section of the appendix contains the definition modules for the sample implementation.

ADM.DEF 2
Exceptions.DEF 4
Expression.DEF 5
Interface.DEF 6
Lex.DEF 10
Listing.DEF 12
Location.DEF 15
ObjectGenerator.DEF 16
PseudoOps.DEF 18
Strings.DEF 19
Table.DEF 22
TableExt.DEF 24
TableTrees.DEF 26

```
DEFINITION  MODULE  ADM;

(*
          Module: ADM                Version 1.0      (c)1990 Mark Wickens

           Created: 22/01/90

             This is the module in the meta-assembler that performs the assembler
             specific operations.  This module defines assembler operation for the
             Motorola MC680x0 series of processors (specifically including
             instructions for the MC68000).  The instructions are as defined in
             the proposed IEEE Assembly Language Standard Draft of 1979.
*)

FROM Strings      IMPORT   String;

CONST
  WORDLENGTH                        = 16;
  MAXINDICES                        = 2;
  MAXOPERANDS                       = 2;
   MAXWORDSPERINSTRUCTION           = 5;
  MEMLOW                            = 0;
  MEMHIGH                            = 1000 * 1024;
  NOOFREGISTERS                     = 17;

TYPE
  Word             = SET OF [0..WORDLENGTH - 1];
   Instruction      =  ARRAY  [1..MAXWORDSPERINSTRUCTION]  OF  Word;
   ValidAddressingModes    =  (
                                     Dir,
                                     Abs,
                                     Rel,
                                      BPage,
                                     Ind,
                                     Imm,
                                      Index,
                                     Reg,
                                      AutoPreInc,
                                      AutoPostInc,
                                      AutoPreDec,
                                      AutoPostDec,
                                      IndPreIndex,
                                      IndPostIndex
                            );
  OperandElement            =  RECORD
                                 Prefix       : CHAR;
                                  Argument     : String;
                                    Indices       : ARRAY [1..MAXINDICES] OF String;
                                 Postfix      : CHAR;
                                  NoOfIndices : CARDINAL
                              END;

  DecodedLine               =  RECORD
                                 Label        : String;
                                 Command      : String;
                                 Modifier     : CHAR;
                                   Operand      : ARRAY [1..MAXOPERANDS] OF
                                                                          OperandElement;
                                 Comment      : String;
                                  NoOfOperands: CARDINAL
                              END;

VAR
   ValidRegisters             :  ARRAY  [1..NOOFREGISTERS]  OF  String;
```

```
PROCEDURE  InsertOpcodesInTable;
(*
          PRE:     The table has been initialised.
           POST:    The opdcodes specific to the assembler have been inserted in
                   the opcode/symbol table.
*)

PROCEDURE  Assemble(DecInstr:  DecodedLine;  PassNo:  CARDINAL;
                       VAR AI: Instruction; VAR Length: CARDINAL);
(*
         PRE:     None.
           POST:     The decoded instruction DecInstr is assembled if possible,
                     otherwise error messages are produced.   The assembled
                     instruction is returned in AI and its length in Length.   If
                     PassNo = 1 then the instruction is not assembled.
*)

PROCEDURE  ADMShutdown;
(*
         PRE:     None.
          POST:    The following actions are taken:
                     1. Any memory allocated by the module is returned to the
                     system.
                     2. Any files opened by the module are closed, flushing any
                        buffers which still contain data.
                     3. Any other influence which the module might have on the
                        system environment is reduced to a minimum.
*)

END  ADM.
```

```
DEFINITION MODULE Exceptions;

(*
          MODULE: Exceptions        Version 1.0       (c) 1989 Mark Wickens

          Created: 28-11-89
          Updated: 04-01-90         Uses interface module.

            This module handles all exception processing for the meta-assembler.
*)

FROM Strings      IMPORT   String;

TYPE
   ExceptionLevel            = (Warning, Error, Fatal);
   ExceptionType              = (Code, User, Environment, Internal);
   StatusType                 = (Normal, NoCode, NoList, NoOutput);

PROCEDURE Raise(Type: ExceptionType; Level: ExceptionLevel; Message: String);

(*
          PRE:     Message is ready for processing.
           POST:    The exception is brought to the attention of the user, either
                     via the screen or listing file (depending on the Type).
                    Action taken depends on the Level:
                     Warning - Code and Listing generation is continued.
                    Error    - Code generation is terminated.
                     Fatal    - All files are close and the assembler exited.
*)

PROCEDURE ChangeStatus(NewStatus: StatusType);

(*
         PRE:     None.
           POST:    The output status of the program is changed to NewStatus.
*)

PROCEDURE ProgramStatus(): StatusType;

(*
         PRE:     None.
           POST:    The current output status of the assembler is returned.
*)

PROCEDURE Number(): CARDINAL;

(*
         PRE:     None.
           POST:    Returns the number of exceptions raised since asssembly started.
*)

END Exceptions.
```

```
DEFINITION  MODULE  Expression;

(*
          MODULE: Expression        Version 1.0       (c) 1989 Mark Wickens

          Created: 03-01-89

           Evaluates an expression according to the symbols in the symbol table
          and constants defined.
*)

FROM  Strings      IMPORT   String;
FROM  TableExt     IMPORT   SymbolStatus;

TYPE
   BaseType        = (Bin, Oct, Dec, Hex);

PROCEDURE  Evaluate(Input:  String;  VAR  Result:  LONGINT;
                                                    VAR ResultType: SymbolStatus);

(*
         PRE:     None.
           POST:    If Input is a valid expression conforming to the syntax
                     described in the IEEE Assembly Language Draft Proposal it is
                     evaluated.  Symbols which appear in the expression are
                     retrieved from the assembler table.  Constants not proceeded
                    by a base are assumed to be in the default base.
*)

PROCEDURE  CheckRegister(Operand:  String;  VAR  Register:  String):  BOOLEAN;
(*
         PRE:     None.
           POST:    If the Operand is a register it's capitilized form is returned
                     in register and the procedure returns TRUE.  If the operand
                     is not a register, the parameter Register will be empty and
                    the procedure returns false.
*)

PROCEDURE  CurrentBase():  BaseType;

(*
         PRE:     None.
           POST:    The current default base is returned.
*)

PROCEDURE  ChangeBase(NewBase:  BaseType);

(*
         PRE:     None.
           POST:    The current default base is changed to NewBase.
*)

PROCEDURE  ExpressionShutdown;
(*
         PRE:     None.
           POST:    The following actions are taken:
                     1. Any memory allocated by the module is returned to the
                     system.
                     2. Any files opened by the module are closed, flushing any
                       buffers which still contain data.
                     3. Any other influence which the module might have on the
                       system environment is reduced to a minimum.
*)

END  Expression.
```

```
DEFINITION  MODULE  Interface;

(*
          MODULE: Interface         Version 1.0       (c) 1990 Mark Wickens

          Created: 01-01-90

            This module provides the interface for the meta-assembler to the
          outside world.
*)

FROM  Strings      IMPORT   String;
FROM  FIO              IMPORT   StandardInput,  StandardOutput,  ErrorOutput,  AuxDevice,
                              PrinterDevice, File;

TYPE
   FileHandle      = File;

CONST
   StdIn                      = StandardInput;
   StdOut                     = StandardOutput;
   ErrOut                     = ErrorOutput;
   AuxDev                     = AuxDevice;
   PrnDev                     = PrinterDevice;
   LongestFileName          = 12;
   NULLFile                 = 16;


(*  File  Operations.  *)

PROCEDURE  FileExists(FileName:  String):  BOOLEAN;

(*
         PRE:     None.
           POST:    If a file with FileName exists, RETURNs TRUE otherwise FALSE
*)

PROCEDURE  CreateAFile(FileName:  String):  FileHandle;

(*
            PRE:      LengthOfString(FileName)  <=  LongestFileName.
                     If the file to be opened is a standard file (as above), no
                    opening is required.
           POST:    The file FileName is created and F becomes a valid FileHandle
                     If a file with filename FileName already exists a warning
                      exception is raised and the filehandle NULLFile is returned.
*)

PROCEDURE  OpenAFile(FileName:  String):  FileHandle;

(*
            PRE:      LengthOfString(FileName)  <=  LongestFileName.
                     If the file to be opened is a standard file (as above), no
                    opening is required.
           POST:    The existing file FileName is opened and its valid FileHandle
                     is returned.  If the file does not exist a warning exception is
                    raised and NULLFile is returned.
*)

PROCEDURE  CloseAFile(F:  FileHandle);

(*
          PRE:     F is a valid FileHandle.
          POST:    The file F is closed.  If an error occurs while
```

```
                    closing the file, an exception is raised.
*)

PROCEDURE  DeleteAFile(Filename:  String);

(*
         PRE:     None.
           POST:    IF FileExists(Filename) THEN it is deleted.
*)

PROCEDURE  EndOfFile():  BOOLEAN;

(*
         PRE:     None.
           POST:    Returns TRUE if the last read operation encountered the end
                  of file.
*)

PROCEDURE  ReadAString(F:  FileHandle;  VAR  S:  String);

(*
          PRE:     S is a string that is ready for processing.
                   F is a valid FileHandle.
           POST:    S is a string of characters which have been read
                    via the current file F.  The string contains a
                     precise copy of characters entered upto either when
                    the end of line was reached or when the number of
                     characters exceed the value of LongestString.
*)

PROCEDURE  WriteAString(F:  FileHandle;  S:  String);

(*
          PRE:     S is a string that is ready for processing.
                   F is a valid FileHandle.
           POST:    The string S is written to the file F, terminated
                    with an end of line character.
*)

PROCEDURE  ReadAChar(F:  FileHandle):  CHAR;

(*
         PRE:     F is a valid FileHandle.
           POST:    A single character is read from the input and returned.
*)

PROCEDURE  WriteAChar(F:  FileHandle;  C:  CHAR);

(*
         PRE:     F is a valid FileHandle.
           POST:    A single character is written to the file F.
*)

PROCEDURE  NextPrinting(F:  FileHandle):  CHAR;

(*
         PRE:     F is a valid FileHandle.
           POST:    The next character which isn't a CR or NewLineCh is read from
                  the file F.
*)

PROCEDURE  WriteALine(F:  FileHandle);

(*
```

```
          PRE:     F is a valid FileHandle.
           POST:    A newline and carraige return is sent to the file F.
*)

PROCEDURE  ReadACard(F:  FileHandle):  CARDINAL;

(*
          PRE:     F is a valid FileHandle.
           POST:    A valid cardinal number is read from file F.
*)
PROCEDURE  WriteACard(F:  FileHandle;  C:  CARDINAL;  Length:  CARDINAL);

(*
          PRE:     F is a valid FileHandle.
           POST:    The cardinal number C is written to file F, padded out to
                      Length characters long with spaces if Length exceeds the length
                   of the cardinal.
*)

PROCEDURE  ReadALongInt(F:  FileHandle):  LONGINT;

(*
          PRE:     F is a valid FileHandle.
           POST:    A long integer is read from the file F and returned.
*)

PROCEDURE  WriteALongInt(F:  FileHandle;  L:  LONGINT;  Length:  CARDINAL);

(*
          PRE:     F is a valid FileHandle.
           POST:    The long integer L is written to file F, padded out to
                      Length characters long with spaces if Length exceeds the length
                   of the long integer.
*)

PROCEDURE  ReadALongHex(F:  FileHandle):  LONGCARD;

(*
          PRE:     F is a valid FileHandle.
           POST:    A long cardinal is read from file F in hexadecimal format and
                   returned.
*)

PROCEDURE  WriteALongHex(F:  FileHandle;  L:  LONGCARD;  Length:  CARDINAL);

(*
          PRE:     F is a valid FileHandle.
           POST:    A long cardinal is written to file F in hexadecimal format and
                     padded out to Length characters long with spaces if the long
                     cardinal is less than length characters long.
*)


(*  Command  Line  Operations.  *)

PROCEDURE  ReadArgument(Number:  CARDINAL;  VAR  Parameter:  String);

(*
           PRE:      Number >= 1 AND Number <= NumberOfArguments()
                     Parameter is ready for processing.
            POST:     Parameter contains the string of printable characters in the
                     Number'th command line argument.
*)
```

```
PROCEDURE  NumberOfArguments():  CARDINAL;

(*
         PRE:     None.
           POST:    The number of command line arguments is returned.
*)

PROCEDURE  GetTime(VAR  Hours,  Minutes,  Seconds:  CARDINAL);

(*
         PRE:     None.
          POST:    The current time is returned.
*)

PROCEDURE  GetDate(VAR  Year,  Month,  Day:  CARDINAL);

(*
         PRE:     None.
          POST:    The current date is returned.
*)

PROCEDURE  InterfaceShutdown;
(*
         PRE:     None.
          POST:    The following actions are taken:
                     1. Any memory allocated by the module is returned to the
                     system.
                     2. Any files opened by the module are closed, flushing any
                        buffers which still contain data.
                     3. Any other influence which the module might have on the
                        system environment is reduced to a minimum.
*)

END  Interface.
```

```
DEFINITION  MODULE  Lex;

(*
          MODULE: Lex                Version 1.0      (c) 1989 Mark Wickens

           Created: 03-12-89
            Updated: 22-01-90          The procedure ScanOperand has been included
                                        as it is an assembler-inspecific operation.

            This module converts the free format source code into a fixed format
            assembler instruction, the layout of which conforms to the IEEE
            proposed assembler language format, which is:

             [label:]  comm  [[pre]  arg  [(index[,  index])]  [post]  [,...]]  [;comment]

          where,

            label    a user defined symbol which can contain any alphanumeric
                      characters apart from ':' and spaces.  It must appear in the
                     first column of the source line.

            comm      command, an assembler opcode or directive containing
                      alphanumeric characters, but no spaces.

           pre       prefix, alphanumeric.

            arg       argument, which can be an expression, constant, or label.

            index    argument index, similar to argument in structure.

           post      postfix, alphanumeric.

            comment  may appear on its own in a source line but must start with a
                    semicolon.
*)

FROM Strings                IMPORT   String;
FROM ADM                        IMPORT   MAXINDICES, MAXOPERANDS, ValidAddressingModes,
                                       DecodedLine, OperandElement;

PROCEDURE  ScanLine(Line:  String):  DecodedLine;
(*
          PRE:     None.
            POST:     Input:   A free-format assembly language instruction.
                       Output: A fixed-format assembly language instruction.

            This procedure performs the lexical scan of the source line.
*)

PROCEDURE  ScanOperand(    OperandToScan:  OperandElement;
                               VAR AddressingMode: ValidAddressingModes);
(*
            PRE:      OperandToScan has is part of a DecodedLine that has been
                    decoded.
            POST:    The addressing mode of the operand is returned.
                      If the addressing mode is invalid an error is produced.
*)

(*  Utility.  *)
PROCEDURE  LexShutdown;
(*
          PRE:     None.
            POST:    The following actions are taken:
                      1. Any memory allocated by the module is returned to the
```

```
                    system.
                    2. Any files opened by the module are closed, flushing any
                       buffers which still contain data.
                    3. Any other influence which the module might have on the
                       system environment is reduced to a minimum.
*)

END  Lex.
```

```
DEFINITION  MODULE  Listing;

(*
          MODULE: Listing           Version 1.0      (c) 1989 Mark Wickens

          Created: 28-11-89

          Creates the listing file.
*)

FROM ADM           IMPORT   Instruction;
FROM Strings      IMPORT   String;

CONST
  MINPAGELENGTH           = 3;
  MAXPAGELENGTH           = 255;
  MINLINELENGTH           = 132;
  MAXLINELENGTH           = 255;
   DEFAULTPAGELENGTH       = 60;
   DEFAULTLINELENGTH       = 132;
   MAXTITLELENGTH           = MINLINELENGTH;

(* Constructors. *)
PROCEDURE  AddLine(Address:  LONGCARD;  MCode:  Instruction;  MCLength:  CARDINAL;
                                                    SourceLine: String);
(*
          PRE:     The Listing File has been opened.
           POST:     Post-Listing-File := Pre-Listing-File + Address + Hex
                     Representation of the Machine Code generated + Source Line
                    which generated Machine Code.
*)

PROCEDURE  AddError(ErrorMessage:  String);

(*
          PRE:     The Listing File has been opened.
            POST:     Post-Listing-File := Pre-Listing-File + ErrorMessage + NewLine
*)

PROCEDURE  StartListing(Filename:  String);
(*
         PRE:     None.
           POST:    The Listing-File is opened.  If there was a listing file
                     previously created with the same name, it is overwritten after
                    a warning has been produced.
*)

PROCEDURE  StopListing;
(*
          PRE:     Pre-Listing-File exists.
            POST:     Post-Listing-File := Pre-Listing-File + Symbol-Table IF
                     Included + Assembly-Statistics
                      Post-Listing-File is closed and therefore Listing-File no
                   longer exists.
*)

PROCEDURE  SetPageLength(NewLength:  CARDINAL);
(*
         PRE:     None.
           POST:     Post-Listing-File will have pages NewLength lines long.
*)

PROCEDURE  SetLineLength(NewLength:  CARDINAL);
(*
```

```
         PRE:     None.
            POST:     Post-Listing-File will have lines NewLength characters long.
*)

PROCEDURE  INCPage;
(*
           PRE:     Pre-Listing-File exists.
            POST:     Post-Listing-File := Pre-Listing-File with the last page
                     finished and a new page started.
*)

PROCEDURE  INCLine;
(*
           PRE:     Pre-Listing-File exists.
            POST:     Post-Listing-File := Pre-Listing-File with a blank line
                     inserted in the current position.
*)

PROCEDURE  SetTitle(TitleString:  String);
(*
         PRE:     None.
            POST:     Post-Listing-File := Pre-Listing-File with the Title on the
                     second line of the page set to TitleString.
*)

PROCEDURE  PurgeError;
(*
         PRE:     None.
            POST:     Post-Listing-File := Pre-Listing-File.   If an error is pending
                     it is removed from the listing file.
*)

PROCEDURE  Address(Include:  BOOLEAN);
(*
         PRE:     None.
            POST:     IF Include THEN Addresses will be included in Listing-File.
                      ELSE Addresses will be omitted from the listing file.
*)

PROCEDURE  MachineCode(Include:  BOOLEAN);
(*
         PRE:     None.
            POST:     IF Include THEN Hex Representation of Machine Code will be
                     included in the Listing-File.
                     ELSE M/C will not be included.
*)

PROCEDURE  SymbolTable(Include:  BOOLEAN);
(*
         PRE:     None.
            POST:     IF Include THEN A Table of all the Symbols defined during
                      Assembly will be dumped at the end of the listing file
                     ELSE No symbols will be dumped.
*)

(*  Predicates.  *)
PROCEDURE  CurrentlyListing():  BOOLEAN;
(*
         PRE:     None.
            POST:     IF Pre-Listing-File exists then RETURNs TRUE otherwise FALSE
*)

PROCEDURE  CurrentSourceLine():  CARDINAL;
(*
```

```
          PRE:     A previous call to StartListing has been made.
           POST:    Returns the current source line being assembled.
*)

PROCEDURE  CurrentLine():  CARDINAL;
(*
          PRE:     Pre-Listing-File exists.
           POST:    The Current Line Number in the Listing-File relative to the
                   top of the page is returned.
*)

PROCEDURE  CurrentPage():  CARDINAL;
(*
          PRE:     Pre-Listing-File exists.
           POST:    The Current Page Number in the Listing-File relative to the
                   first page is returned.
*)

PROCEDURE  CurrentPageLength():  CARDINAL;
(*
         PRE:     None.
           POST:    The Current-Page-Length is returned.
*)

PROCEDURE  CurrentLineLength():  CARDINAL;
(*
         PRE:     None.
           POST:    The Current-Line-Length is returned.
*)

PROCEDURE  MCAssemblyRate():  LONGINT;
(*
           PRE:     Previous calls to StartListing and StopListing have been made.
           POST:    The LONGINT returned contains the assembly rate in lines/min
                   of the program assembled.
*)

PROCEDURE  ListingShutdown;
(*
         PRE:     None.
          POST:    The following actions are taken:
                    1. Any memory allocated by the module is returned to the
                    system.
                    2. Any files opened by the module are closed, flushing any
                       buffers which still contain data.
                    3. Any other influence which the module might have on the
                       system environment is reduced to a minimum.
*)

END  Listing.
```

```
DEFINITION  MODULE  Location;

(*
          MODULE: Location          Version 1.0      (c) 1989 Mark Wickens

          Created: 28-11-89

           Handles the location of the location counter relative to the start
          of the source file.
*)

PROCEDURE  ResetLocation;
(*
         PRE:     None.
           POST:    The location counter is reset to MEMLOW.
*)

PROCEDURE  SetLocation(NewLocation:  LONGCARD);
(*
         PRE:     None.
            POST:     IF NewLocation >= MEMLOW AND NewLocation <= MEMHIGH THEN
                        The location counter is set to NewLocation
                  ELSE
                      An error is produced.
                  END
*)

PROCEDURE  CurrentLocation():  LONGCARD;
(*
            PRE:      ResetLocation has previously been called.
           POST:    Returns the current location.
*)

PROCEDURE  INCLocation(Increment:  CARDINAL);
(*
            PRE:      ResetLocation has previously been called.
            POST:     IF CurrentLocation() + Increment <= MEMHIGH THEN
                         The current location is incremented by the amount Increment.
                  ELSE
                      An error is produced.
                  END
*)

PROCEDURE  LocationShutdown;
(*
         PRE:     None.
           POST:    The following actions are taken:
                     1. Any memory allocated by the module is returned to the
                     system.
                     2. Any files opened by the module are closed, flushing any
                        buffers which still contain data.
                     3. Any other influence which the module might have on the
                        system environment is reduced to a minimum.
*)

END  Location.
```

```
DEFINITION  MODULE  ObjectGenerator;

(*
           MODULE: ObjectGenerator Version 1.0       (c) 1989 Mark Wickens

          Created: 28-11-89

            Produces the object file in a format determined by calling module.
*)

FROM Strings                IMPORT   String;
FROM ADM                     IMPORT   Instruction;

TYPE
    ObjectFormat  =  (Generic,  Motorola);

(*  Constructors.  *)
PROCEDURE  StartObject(Filename:  String);
(*
          PRE:     SetFormat is called.
           POST:    The file Filename is created and subsequently used to output
                   the object code into.
*)

PROCEDURE  StopObject;
(*
           PRE:      An object file has previously been opened with StartObject.
           POST:    The file being used for object code is closed.
*)

PROCEDURE  SetFormat(NewType:  ObjectFormat);
(*
           PRE:      Any previous calls to StartObject have been followed with a
                      call to StopObject (i.e., there isn't an object code file
                   currently open.
           POST:    The object code produced when StartObject is called will
                   be of NewType format.
*)

PROCEDURE  AddCode(Address:  LONGCARD;  MCCode:  Instruction;  MCLength:  CARDINAL);
(*
           PRE:      StartObject has been called and not followed by StopObject
                    (i.e., the object code file is open).
           POST:    The object code is added to the object file in the format
                   specified by SetFormat.
*)

PROCEDURE  ObjectOutput(Switch:  BOOLEAN);
(*
          PRE:     StartObject has not been called.
           POST:    When StartObject is called, it will have no effect if Switch
                   was set to FALSE (Default is TRUE)
*)

PROCEDURE  ObjectGeneratorShutdown;
(*
         PRE:     None.
           POST:    The following actions are taken:
                     1. Any memory allocated by the module is returned to the
                     system.
                     2. Any files opened by the module are closed, flushing any
                        buffers which still contain data.
                     3. Any other influence which the module might have on the
                        system environment is reduced to a minimum.
```

```
*)

END ObjectGenerator.
```

```
DEFINITION  MODULE  PseudoOps;

(*
          MODULE: PseudoOps         Version 1.0       (c)1990 Mark Wickens

          Created: 03/03/90

            This module handles all valid operations not assembler specific,
             i.e., all the assembly language-independent pseudo operations.
*)

FROM ADM           IMPORT   DecodedLine;

PROCEDURE  AssemblePseudoOp(DecInstr:  DecodedLine;  PassNo,  Type:  CARDINAL;
                              VAR Length: CARDINAL);

PROCEDURE  PseudoOpsShutdown;
(*
         PRE:     None.
          POST:    The following actions are taken:
                     1. Any memory allocated by the module is returned to the
                     system.
                     2. Any files opened by the module are closed, flushing any
                       buffers which still contain data.
                     3. Any other influence which the module might have on the
                       system environment is reduced to a minimum.
*)

END  PseudoOps.
```

```
DEFINITION  MODULE  Strings;

(*
            This module defines the data type 'String' and operations on
           variables of type 'String'.

           The implementation of the data type is defined to be as
          an array.

            The procedures in this module are defined in terms of Pre-conditions
            and Post-conditions which must hold for the instant before and after
           the procedures are called.
*)

CONST
          LongestString    = 133;
          EndOfLine        = CHR(0);

TYPE
           StringLengths     =  [1..LongestString];
          String             = ARRAY StringLengths OF CHAR;

PROCEDURE  InitialiseAString(VAR  S:  String);

(*
         PRE:     None.
           POST:    The variable S is returned as a string that is
                    ready for processing.
*)

PROCEDURE  ArrayToString(Array:  ARRAY  OF  CHAR;  VAR  S:  String);

(*
           PRE:      S is a string that is ready for processing.
            POST:    S is a String which contains the ordered sequence of
                       characters contained in Array, upto HIGH(Array) or
                      LongestString, whichever is the shortest.
*)

PROCEDURE  StringToArray(S:  String;  VAR  Array:  ARRAY  OF  CHAR);

(*
           PRE:      S is a string that is ready for processing.
            POST:     Array is a character array containing the ordered sequence of
                        characters contained in S, upto LengthOfString(S) characters
                      long.  The last character in Array is an EndOfLine.
*)

PROCEDURE  CapitaliseString(VAR  S:  String);

(*
           PRE:      S is a string that is ready for processing.
           POST:    FOR Every Character in the string S DO
                       Capitalise the Character.
*)

PROCEDURE  ConcatStrings(VAR  S1:  String;  S2:  String);
(*
           PRE:      Strings S1 and S2 are both ready for processing.
             POST:     IF LengthOfString(S1)  +  LengthOfString(S2)  <=  LongestString
                     THEN S1 := S1 + S2 (concatenated with)
                      ELSE S1 := S1 + Substring of S2 upto LongestString
*)
```

```
PROCEDURE  CharInString(S:  String;  P:  CARDINAL):  CHAR;

(*
          PRE:     S is a string that is ready for processing.
           POST:    IF Position <= LengthOfString(S)  THEN
                       Returns the character in string S whose position
                       corresponds to the P'th character from the start
                     of the string.
                 ELSE
                      Returns the character EndOfLine.
                 END
*)

PROCEDURE  LengthOfString(S:  String):  CARDINAL;

(*
          PRE:     S is a string that is ready for processing.
           POST:    Returns a cardinal value which corresponds to the
                    number of characters in the string S.
*)

PROCEDURE  MakeSubstring(S:  String;  StartPos,  Length:  CARDINAL;
                            VAR Substring: String);

(*
          PRE:     S and Substring are strings that are ready for
                 processing.
                   1 <= StartPos <= LengthOfString(S).
                    Length + StartPos <= LengthOfString(S)  + 1.
           POST:    Substring is a sequence of characters taken from S
                     whose starting position in S is StartPos and whose
                  length is Length.
*)

PROCEDURE  EmptyString(S:  String):  BOOLEAN;

(*
          PRE:     S is a string that is ready for processing.
          POST:    IF LengthOfString(S)  = 0
                      THEN
                           RETURN TRUE;
                      ELSE
                           RETURN FALSE
                    Returns a value of TRUE is string S is empty, otherwise
                  returns FALSE.
*)

PROCEDURE  ReverseString(VAR  S:  String);

(*
          PRE:     S is a string that is ready for processing.
          POST:    The string S is reversed.  That is, if the string
                    contains n chars, then char 1 is swapped with char n,
                   char 2 with char (n-1) and so on.
*)

PROCEDURE  AlphabeticallyLess(S1,  S2:  String):  BOOLEAN;

(*
          PRE:     S1 & S2 are strings that are ready for processing.
                    S1 & S2 are strings that contain only alphabetic
                  characters.
          POST:    Returns a value of TRUE if S1 occurs before S2
                    alphabetically.  If S1 = S2 or S1 occurs after S2
```

```
                    then the function returns a value of FALSE.
*)

PROCEDURE  EqualStrings(S1,  S2:  String):  BOOLEAN;

(*
           PRE:      S1 & S2 are strings that are ready for processing.
                      S1 & S2 are strings that contain only alphabetic
                    characters.
           POST:     Returns TRUE if S1 has the same content, char for char,
                     as S2.  Otherwise, it returns FALSE.
*)

PROCEDURE  StringsShutdown;
(*
          PRE:     None.
           POST:    The following actions are taken:
                     1. Any memory allocated by the module is returned to the
                     system.
                     2. Any files opened by the module are closed, flushing any
                        buffers which still contain data.
                     3. Any other influence which the module might have on the
                        system environment is reduced to a minimum.
*)

END  Strings.
```

```
DEFINITION  MODULE  Table;

(*
          MODULE Table               Version 1.0      (c) 1989 Mark Wickens

          Created:          24-11-89

            This module implements a hash table. Each entry in the hash table
            is a pointer to the head of a binary search tree. Thus any entry in the
            hash table can contain any number of records.

            The speed of the hash table in big 'O' notation approximates to one
            when less than 90% of the entries in the hash table contain
            only one record (the rest containing none). However, the speed even in
            the worst scenario where every record hashes to one entry is only
            log N, thanks to using binary search trees.

            The one operation which hash tables are very slow at performing is an
            ordered traversal. The NextInOrder procedure has a big 'O' speed of
          N cubed.
*)

FROM  TableExt      IMPORT    EntryType,  SymbolStatus,  TableType;
FROM  Strings      IMPORT   String;

(*  Table  operations.  *)

PROCEDURE  InitialiseTable;
(*
          PRE:     None.
            POST:    The hash table is initialised to be empty and is subsequently
                     ready for processing.
*)

PROCEDURE  IsRoom():  BOOLEAN;
(*
           PRE:      The hash table is ready for processing.
           POST:     If there is room in the hash table then TRUE is returned.
                      The only case when this will ever be FALSE is if dynamic
                     allocation of memory fails.
*)

PROCEDURE  IsIn(Key:  String):  BOOLEAN;
(*
           PRE:      The hash table is ready for processing.
            POST:    Returns TRUE if an entry with key field Key already exists
                    in the table.
*)

PROCEDURE  Retrieve(Key:  String):  TableType;
(*
           PRE:      The hash table is ready for processing.
                      An entry with key field Key exists in the table.
           POST:    Returns the record with key field Key.
*)

PROCEDURE  Delete(Key:  String);
(*
           PRE:      The hash table is ready for processing.
                      An entry with key field Key exists in the table.
            POST:    Post-Table = Pre-Table - entry with key field Key.
*)

PROCEDURE  Insert(Entry:  TableType);
```

```
(*
          PRE:     The hash table is ready for processing.
                    An entry with key field Entry.Key does not exist in the table.
           POST:    Post-Table = Pre-Table + entry Entry.
*)

PROCEDURE  Amend(VAR  Entry:  TableType);
(*
          PRE:     The hash table is ready for processing.
                    An entry with key field Key exists in the table.
           POST:    Post-table = Pre-table - old entry Entry.Key + Entry.
*)

PROCEDURE  Greater(Key:  String):  BOOLEAN;
(*
          PRE:     The hash table is ready for processing.
           POST:    Returns TRUE if an entry with a key alphabetically greater
                   than Key exists.
*)

PROCEDURE  NextInOrder(Key:  String;  VAR  Entry:  TableType);
(*
          PRE:     The hash table is ready for processing.
                   Greater(Key).
           POST:    Returns the entry with next key in order from Key.
*)

PROCEDURE  SizeOfTable():  CARDINAL;
(*
          PRE:     The hash table is ready for processing.
           POST:    Returns the number of possible locations which can be hashed
                   to (the number of entries).
*)

PROCEDURE  ElementsAtLocation(Location:  CARDINAL):  CARDINAL;
(*
          PRE:     The hash table is ready for processing.
                   0 < Location <= SizeOfTable()
           POST:    Returns the number of records in the BST at entry number
                   Location in the hash table.
*)


PROCEDURE  TableShutdown;
(*
         PRE:     None.
           POST:    The following actions are taken:
                     1. Any memory allocated by the module is returned to the
                     system.
                     2. Any files opened by the module are closed, flushing any
                       buffers which still contain data.
                     3. Any other influence which the module might have on the
                       system environment is reduced to a minimum.
*)

END  Table.
```

```
DEFINITION  MODULE  TableExt;

(*
          Module: TableExt          Version 1.0      (c) 1989 Mark Wickens

          Created: 24-12-89

           This module serves two purposes.

           (1).  It contains all the data specifics so that any changes to the
            record type stored within the table are contained in one module only.

           (2).  It extends the basic table operations to allow specific fields
            of a table record to be specified as parameters in an insert procedure,
            allowing the initial set-up of the table to be performed without
          reference to variables.
*)

FROM ADM           IMPORT   Word;
FROM  Strings      IMPORT   String;
FROM  Interface    IMPORT   FileHandle;

TYPE
   EntryType        =  (Command,  Symbol);
    SymbolStatus    =  (Absolute,  Relative,  Register);

   TableType  =  RECORD
                  Key: String;
                   CASE Kind: EntryType OF
                     Symbol:        Value: LONGINT;
                                     Status: SymbolStatus
                   |
                     Command:       Type: CARDINAL;
                                    Mask: Word;
                 END
             END;


(*  Extended  Operations.  *)

PROCEDURE  InsertCommand(Key:  String;  Mask:  Word;  Type:  CARDINAL);
(*
          PRE:     Key is a string that is ready for processing.
           POST:   Enables the insertion of a record into the hash table
                    which is of the type Command by specifying all the relevant
                   fields of the record TableType.
*)

PROCEDURE  InsertSymbol(Key:  String;  Value:  LONGINT;  Status:  SymbolStatus);
(*
          PRE:     Key is a string that is ready for processing.
           POST:   Enables the insertion of a record into the hash table
                    which is of type Symbol by specifying all the relevant
                   fields of the record TableType.
*)

(*  I/O  Operations.  *)

PROCEDURE  GetRec(Input:  FileHandle;  KeyToInput:  String;  VAR  NewRec:  TableType);
(*
          PRE:     Input is a valid FileHandle.
                    KeyToInput is a string that is ready for processing.
           POST:   If a valid record of type TableType has been put in the file
                    Input it is retrieved and placed in NewRec.
```

```
*)

PROCEDURE  PutRec(Output:  FileHandle;  Record:  TableType);
(*
          PRE:     Output is a valid FileHandle.
           POST:    The record Record is written to the file Output using an
                     internal format. This record can be read by the procedure
                  GetRec.
*)

(*  Conversion  Operations.  *)

PROCEDURE  ConvertTo(Mask:  String):  Word;
(*
          PRE:     Mask is a string that is ready for processing.
                    LengthOfString(Mask) = WORDLENGTH (from ADM).
           POST:    A type conversion is made from the input string to a valid
                     Word which is returned. The string mask must consist of a
                      sequence of '1' characters and '0' characters which represents
                     the corresponding bits in the word Word.
*)

PROCEDURE  ConvertFrom(Mask:  Word;  VAR  ConvString:  String);
(*
         PRE:     None.
           POST:    ConvString contains a string of '1' characters and '0'
                     characters corresponding to the bit pattern of the word Mask.
*)

PROCEDURE  TableExtShutdown;
(*
         PRE:     None.
           POST:    The following actions are taken:
                     1. Any memory allocated by the module is returned to the
                     system.
                     2. Any files opened by the module are closed, flushing any
                        buffers which still contain data.
                     3. Any other influence which the module might have on the
                        system environment is reduced to a minimum.
*)

END  TableExt.
```

```
DEFINITION  MODULE  TableTrees;

(*
          MODULE TableTrees         Version 1.0       (c) 1989 Mark Wickens

         Created:          02-12-89

           This library module implements a binary search tree.
           Each node of the tree data of type TableType, and the tree does not
           contain duplications.  It can handle any number of trees, the type of
           which is exported opaquely to user programs.
*)

FROM  TableExt     IMPORT   TableType;
FROM  Strings      IMPORT   String;

(*
           The type BST is used by user programs to define the head of a tree.
*)
TYPE
   BST;   (*        Opaque export.   *)

PROCEDURE  InitialiseATree(VAR  Tree:  BST);
(*
         Pre-conditions:           None.
           Post-conditions:           The BST Tree is empty and ready for processing.
*)

PROCEDURE  IsRoomInTree():  BOOLEAN;
(*
          PRE:     The BST is ready for processing.
           POST:    Returns TRUE if there is room for another entry.
*)

PROCEDURE  InsertInTree(VAR  Tree:  BST;  DataItem:  TableType);
(*
           Pre-conditions:            The BST Tree is ready for processing and
                                       Key is not already in the BST Tree.
           Post-conditions:           Key is inserted in the correct position
                                     in the tree.
*)

PROCEDURE  IsInTree(Tree:  BST;  Key:  String):  BOOLEAN;
(*
           Pre-conditions:            The BST Tree is ready for processing.
           Post-conditions:           Post-Tree = Pre-Tree.
                                       If Key is in Tree then returns TRUE else
                                     returns FALSE.
*)

PROCEDURE  DeleteFromTree(VAR  Tree:  BST;  Key:  String);
(*
           Pre-conditions:            The BST Tree is ready for processing and
                                     Key is in the tree.
           Post-conditions:           Post-Tree = Pre-Tree except the record
                                       Key no longer appears in the BST Tree.
*)

PROCEDURE  ElementsInTree(Tree:  BST):  CARDINAL;
(*
           Pre-conditions:            The BST Tree is ready for processing.
           Post-conditions:           Post-Tree = Pre-Tree.
                                       The value returned is the number of items in
                                     the BST Tree.
```

```
*)

PROCEDURE  RetrieveFromTree(Tree:  BST;  Key:  String;
                                                        VAR DataItem: TableType);
(*
           Pre-conditions:            The BST Tree is ready for processing, and Key
                                     exists.
           Post-conditions:           Post-Tree = Pre-Tree.
                                       Key contains the data in the BST Tree
                                       stored under the entry Key.
*)

PROCEDURE  NextInOrderInTree(Tree:  BST;  VAR  CurrentNext:  String;
                                                      Key: String);
(*
           Pre-conditions:            The BST Tree is ready for processing.
                                       If the first node in the tree is being
                                        searched for, Key = FirstKeyInOrder &
                                                   CurrentNext = FirstKeyInOrder.
           Post-conditions:           Post-Tree = Pre-Tree.
                                         CurrentNext will contain the next highest key
                                        value in the tree greater than Key, if one
                                         exist, otherwise CurrentNext is unchanged.
*)

PROCEDURE  TableTreesShutdown;
(*
          PRE:     None.
           POST:    The following actions are taken:
                     1. Any memory allocated by the module is returned to the
                     system.
                     2. Any files opened by the module are closed, flushing any
                       buffers which still contain data.
                     3. Any other influence which the module might have on the
                        system environment is reduced to a minimum.
*)

END  TableTrees.
```

A.2. Implementation Modules.

This section of the appendix contains the implementation modules for the sample implementation.

ADM.MOD 29
Exceptions.MOD 53
Expression.MOD 55
Interface.MOD 66
Lex.MOD 74
Listing.MOD 81
Location.MOD 89
M2Assem.MOD 90
ObjectGenerator.MOD 93
PseudoOps.MOD 96
Strings.MOD 101
Table.MOD 105
TableExt.MOD 108
TableTrees.MOD 112

```
IMPLEMENTATION  MODULE  ADM;

(*
          Module: ADM                Version 1.0      (c)1990 Mark Wickens

           Created: 22/01/90

             This is the module in the meta-assembler that performs the assembler
             specific operations.  This module defines assembler operation for the
             Motorola MC680x0 series of processors (specifically including
             instructions for the MC68000).  The instructions are as defined in
             the proposed IEEE Assembly Language Standard Draft of 1979.
*)

FROM  Exceptions              IMPORT   Raise,  ExceptionLevel,  ExceptionType;
FROM  TableExt                IMPORT   InsertCommand,  InsertSymbol,  SymbolStatus;
FROM  Table                   IMPORT   TableType,  IsIn,  Retrieve;
FROM  Location                IMPORT   CurrentLocation;
FROM  Strings                 IMPORT   EmptyString,  InitialiseAString,  CharInString,
                                       EqualStrings;
FROM  Expression              IMPORT   Evaluate,  CheckRegister;
FROM  Lex                     IMPORT   ScanOperand;
FROM  PseudoOps               IMPORT   AssemblePseudoOp;

TYPE
  RegType                  = (Data,  Address);
   AddressingModeSet       = SET  OF  ValidAddressingModes;

CONST
    AMS1    =  AddressingModeSet{Dir,Abs,Rel,Ind,Imm,Index,Reg,AutoPreInc,
                        AutoPostInc,AutoPreDec,AutoPostDec,IndPreIndex,IndPostIndex};
    AMS2    =  AddressingModeSet{Ind,AutoPreDec,IndPreIndex,Abs};
    AMS3    =  AddressingModeSet{Ind,AutoPostInc,IndPreIndex,Abs};
    AMS4    =  AddressingModeSet{Dir,Ind,AutoPostInc,AutoPreDec,IndPreIndex,Abs};
    AMS5    =  AddressingModeSet{Ind,AutoPreDec,AutoPostInc,IndPreIndex,Abs};

   LabelError                = "Label already defined - first value holds";
   CommandError              = "Command not defined";
   ModifierError             = "Wrong modifier";
    WrongRegisterError       = "Wrong register type";
   OperandNoError            = "Wrong number of operands or indices";
   BadRegisterError          = "Badly formed register";
   AbsoluteError             = "Argument must be absolute";
   AddrModeError             = "Wrong addressing mode";
   Type16ErrorA              = "Vector must be between 0 and 15";
   RegisterError             = "Argument must be a register";
   TooLargeError             = "Argument too large for modifier specified";
   AddressRegError           = "Register must be an address register";
   OutOfRangeError           = "Operand out of range";
   RelativeError             = "Argument must be relative";

PROCEDURE  RegisterType(Register:  String):  RegType;
VAR
    Indicator:  CHAR;

BEGIN
    Indicator  :=  CharInString(Register,  1);
   IF  Indicator  =  "D"  THEN
      RETURN  Data
   ELSIF  Indicator  =  "A"  THEN
      RETURN  Address
    ELSIF  EqualStrings("SP",  Register)  THEN
      RETURN  Address
   ELSE
```

```
        Raise(Code, Error, BadRegisterError);
      RETURN Data
   END
END RegisterType;


PROCEDURE AddBinary(VAR AI: Word; BinaryToAdd: Word; Position, Len: CARDINAL);
VAR
    BitCounter: CARDINAL;

BEGIN
   FOR BitCounter := Position TO Position + Len - 1 DO
       IF BitCounter - Position IN BinaryToAdd THEN
         INCL(AI, BitCounter)
     END
   END
END AddBinary;


PROCEDURE AddRegister(VAR AI: Word; RegisterToAdd: String; Position: CARDINAL);

VAR
    RegisterNumber: CHAR;
    RegisterValue: CARDINAL;

BEGIN
    RegisterNumber := CharInString(RegisterToAdd, 2);
    RegisterValue := CARDINAL(ORD(RegisterNumber) - ORD('0'));
    IF EqualStrings("SP", RegisterToAdd) THEN
      RegisterValue := 7
   END;
    CASE RegisterValue OF
      1:  AI := AI + Word{Position}|
      2:  AI := AI + Word{Position + 1}|
      3:  AI := AI + Word{Position + 1, Position}|
      4:  AI := AI + Word{Position + 2}|
      5:  AI := AI + Word{Position + 2, Position}|
      6:  AI := AI + Word{Position + 2, Position + 1}|
      7:  AI := AI + Word{Position + 2, Position + 1, Position}
   END
END AddRegister;


PROCEDURE Assemble(DecInstr: DecodedLine; PassNo: CARDINAL;
                       VAR AI: Instruction; VAR Length: CARDINAL);

(*
            The comments for all TypeXX commands provide information of the type
           of instruction being assembled.  The key to the word shown is;

          O - Opcode.
          R - Register.
          S - Size.
          8 - 8 Bit Constant.
          1 - Count.
          E - Effective Address.
          D - Data.
          C - Condition.
          P - Displacement.
          V - Vector
*)

VAR
   CommandEntry   : TableType;
```

```
PROCEDURE CalcEA        (
                          Operand           : OperandElement;
                           ValidModes        : AddressingModeSet;
                         Modifier          : CHAR;
                         Position          : CARDINAL;
                         VAR AI            : Instruction
                         ): CARDINAL;
(*
         This procedure determines the address type of the operand and
         using that creates the appropriate bit pattern in the instruction AI
         (on PassNo 2 only).  The resulting effective address is placed in
        word 1 of the AI and then returns the length of the instruction
        relative to the modifier used.
*)

VAR
   AddrMode      : ValidAddressingModes;
  Result        : LONGINT;
   ResultType    : SymbolStatus;
  Register      : String;


 PROCEDURE ARIWISecondWord(VAR SecondWord: Word;
                                   IndexRegString, DisplacementString: String);
VAR
    IndexRegister: String;
    Displacement: LONGINT;
     DisplacementType: SymbolStatus;

BEGIN
   SecondWord := Word{};
     InitialiseAString(IndexRegister);
     IF CheckRegister(IndexRegString, IndexRegister) THEN
       IF RegisterType(IndexRegister) = Address THEN
        INCL(SecondWord, 15)
    END;
        AddRegister(SecondWord, IndexRegister, 12);
      INCL(SecondWord, 11);
         Evaluate(DisplacementString, Displacement, DisplacementType);
      IF DisplacementType = Absolute THEN
         IF (Displacement >= -128) AND (Displacement <= 127) THEN
             AddBinary(SecondWord, Word(SHORTINT(Displacement)), 0 , 8)
      ELSE
           Raise(Code, Error, OutOfRangeError)
      END
    ELSE
         Raise(Code, Error, AbsoluteError)
    END
  ELSE
       Raise(Code, Error, RegisterError)
  END
 END ARIWISecondWord;


BEGIN
     ScanOperand(Operand, AddrMode);
     InitialiseAString(Register);
   IF AddrMode IN ValidModes THEN
     CASE AddrMode OF
      Dir:
         IF PassNo = 1 THEN
           RETURN 1
```

```
  ELSE
        IF CheckRegister(Operand.Argument, Register) THEN
         IF RegisterType(Register) = Address THEN
         INCL(AI[1], Position + 3)                  (* Set Mode *)
      END;
           AddRegister(AI[1], Register, Position)(* Set Register *)
    ELSE
         Raise(Code, Error, RegisterError)
    END;
    RETURN 1
  END
|
Ind:
   IF PassNo = 1 THEN
    RETURN 1
  ELSE
        IF CheckRegister(Operand.Argument, Register) THEN
         IF RegisterType(Register) = Address THEN
          INCL(AI[1], Position + 4);
            AddRegister(AI[1], Register, Position)
      ELSE
           Raise(Code, Error, AddressRegError)
      END
    ELSE
         Raise(Code, Error, RegisterError)
    END;
    RETURN 1
  END
|
 AutoPostInc:
   IF PassNo = 1 THEN
    RETURN 1
  ELSE
        IF CheckRegister(Operand.Argument, Register) THEN
         IF RegisterType(Register) = Address THEN
          INCL(AI[1], Position + 3);
          INCL(AI[1], Position + 4);
            AddRegister(AI[1], Register, Position)
      ELSE
           Raise(Code, Error, AddressRegError)
      END
    ELSE
         Raise(Code, Error, RegisterError)
    END;
    RETURN 1
  END
|
 AutoPreDec:
   IF PassNo = 1 THEN
    RETURN 1
  ELSE
        IF CheckRegister(Operand.Argument, Register) THEN
         IF RegisterType(Register) = Address THEN
          INCL(AI[1], Position + 5);
            AddRegister(AI[1], Register, Position)
      ELSE
           Raise(Code, Error, AddressRegError)
      END
    ELSE
         Raise(Code, Error, RegisterError)
    END;
    RETURN 1
  END
|
```

```
          IndPostIndex:
            IF PassNo = 1 THEN
              RETURN 2
           ELSE
                 IF CheckRegister(Operand.Argument, Register) OR
                                  (CharInString(Operand.Argument, 1) = '*') THEN
                  IF RegisterType(Register) = Address THEN
                    IF (Operand.NoOfIndices = 1) AND
                         (CharInString(Operand.Argument, 1) <> '*') THEN
(*
           Motorola Addressing mode:
            Address Register Indirect with Displacement
*)
                      INCL(AI[1], Position + 3);
                      INCL(AI[1], Position + 5);
                        AddRegister(AI[1], Register, Position);
                    ELSIF (Operand.NoOfIndices = 1) AND
                            (CharInString(Operand.Argument, 1) = '*') THEN
(*
           Motorola Addressing mode:
            Program Counter with Displacement
*)
                      INCL(AI[1], Position + 3);
                      INCL(AI[1], Position + 4);
                      INCL(AI[1], Position + 5);
                      INCL(AI[1], Position + 1)
                 END;
                    IF (Operand.NoOfIndices = 1) THEN
                         Evaluate(Operand.Indices[1], Result, ResultType);
                      IF ResultType = Absolute THEN
                         IF (Result >= -32768) AND (Result <= 32767) THEN
                           AI[2] := Word(INTEGER(Result))
                      ELSE
                            Raise(Code, Error, OutOfRangeError)
                      END
                    ELSE
                         Raise(Code, Error, AbsoluteError)
                    END
                    ELSIF (Operand.NoOfIndices = 2) AND
                            (CharInString(Operand.Argument, 1) <> "*") THEN
(*
           Motorola Addressing mode:
            Address Register Indirect with Index
*)
                      INCL(AI[1], Position + 5);
                      INCL(AI[1], Position + 4);
                        AddRegister(AI[1], Register, Position);
                          ARIWISecondWord(AI[2], Operand.Indices[1], Operand.Indices[2])
                    ELSIF (Operand.NoOfIndices = 2) AND
                            (CharInString(Operand.Argument, 1) = "*") THEN
(*
           Motorola Addressing mode:
           Program Counter with Index
*)
                      INCL(AI[1], Position + 3);
                      INCL(AI[1], Position + 4);
                      INCL(AI[1], Position + 5);
                      INCL(AI[1], Position);
                      INCL(AI[1], Position + 1);
                    ELSIF (Operand.NoOfIndices = 2) THEN
                         Evaluate(Operand.Indices[1], Result, ResultType);
                      IF ResultType = Absolute THEN
                         IF (Result >= -128) AND (Result <= 127) THEN
                           AI[2] := Word(SHORTINT(Result));
```

```
                        IF CheckRegister(Operand.Indices[1], Register) THEN
                        AddRegister(AI[2], Register, 12);
                         IF RegisterType(Register) = Address THEN
                        INCL(AI[2], 15)
                      END;
                       INCL(AI[2], 11)
                    ELSE
                        Raise(Code, Error, RegisterError)
                    END
                  ELSE
                       Raise(Code, Error, OutOfRangeError)
                  END
                ELSE
                     Raise(Code, Error, AbsoluteError)
                END
              END
            ELSE
                 Raise(Code, Error, AbsoluteError)
            END
          ELSE
               Raise(Code, Error, RegisterError)
          END;
          RETURN 2
        END
      |
      Abs:
(*
        Motorola Addressing mode:
       Absolute Long Address
*)
         IF PassNo = 1 THEN
           RETURN 3
        ELSE
               Evaluate(Operand.Argument, Result, ResultType);
            IF ResultType = Absolute THEN
              INCL(AI[1], Position + 3);
              INCL(AI[1], Position + 4);
              INCL(AI[1], Position + 5);
              INCL(AI[1], Position);
               AddBinary(AI[3], Word(Result), 0, 16);
               AddBinary(AI[2], Word(Result >> 16), 0, 16)
          ELSE
               Raise(Code, Error, AbsoluteError)
          END;
          RETURN 3
        END
      |
      Imm:
         IF PassNo = 1 THEN
             IF (Modifier = 'B') OR (Modifier = 'W') THEN
             RETURN 2
          ELSE
             RETURN 3
          END
        ELSE
               Evaluate(Operand.Argument, Result, ResultType);
            IF ResultType = Absolute THEN
              INCL(AI[1], Position + 3);
              INCL(AI[1], Position + 4);
              INCL(AI[1], Position + 5);
              INCL(AI[1], Position + 2);
              IF (Result >= -128) AND (Result <= 127) AND
                (Modifier = 'B') THEN
                AI[2] := Word(INTEGER(Result));
```

```
                RETURN 2
                 ELSIF (Result >= -32768) AND (Result <= 32767) AND
                      (Modifier = 'W') THEN
                   AI[2] := Word(INTEGER(Result));
                RETURN 2
               ELSIF Modifier = 'L' THEN
                  AI[2] := Word(Result DIV 32768);
                 AI[3] := Word(Result);
                RETURN 3
             ELSE
                  Raise(Code, Error, OutOfRangeError)
             END
           ELSE
                Raise(Code, Error, AbsoluteError)
           END;
           RETURN 3
         END
       |
       Rel:
          IF PassNo = 1 THEN
              IF (Modifier = 'B') OR (Modifier = 'W') THEN
              RETURN 2
           ELSE
              RETURN 3
           END
         ELSE
                Evaluate(Operand.Argument, Result, ResultType);
             IF ResultType = Absolute THEN
               INCL(AI[1], Position + 3);
               INCL(AI[1], Position + 4);
               INCL(AI[1], Position + 5);
               INCL(AI[1], Position + 2);
                IF (Result >= -128) AND (Result <= 127) AND
                   (Modifier = 'B') THEN
                   AI[2] := Word(INTEGER(Result));
                RETURN 2
                 ELSIF (Result >= -32768) AND (Result <= 32767) AND
                      (Modifier = 'W') THEN
                   AI[2] := Word(INTEGER(Result));
                RETURN 2
               ELSIF Modifier = 'L' THEN
                  AI[2] := Word(Result DIV 32768);
                 AI[3] := Word(Result);
                RETURN 3
             ELSE
                  Raise(Code, Error, OutOfRangeError)
             END
           ELSE
                Raise(Code, Error, AbsoluteError)
           END;
           RETURN 3
         END
     END
   ELSE
        Raise(Code, Error, AddrModeError);
     RETURN 0
   END
 END CalcEA;


  PROCEDURE Type1(): CARDINAL;
 (*
          Format: OOSSEEEEEEEEEEEE
           Assembles: LD, ST & MV (equivalent to the Motorola MOVE instruction).
```

```
                        These are selected with the type23 procedure to ensure
                       that each mode has the right type of operands.
*)

VAR
   Length, Counter       : CARDINAL;
   FirstWordCopy         : Word;

BEGIN
     Length  :=  CalcEA(DecInstr.Operand[2],  AMS1,  DecInstr.Modifier,  6,  AI);
    (* Swap Register and Mode positions in output word. *)
    FirstWordCopy  :=  AI[1];
   FOR Counter  :=  6  TO  8  DO
       IF  Counter  IN  FirstWordCopy  THEN
         INCL(AI[1],  Counter  +  3)
     ELSE
         EXCL(AI[1],  Counter  +  3)
     END
   END;
   FOR Counter  :=  9  TO  11  DO
       IF  Counter  IN  FirstWordCopy  THEN
         INCL(AI[1],  Counter  -  3)
     ELSE
         EXCL(AI[1],  Counter  -  3)
     END
   END;
   AI[4]  :=  AI[2];
   AI[5]  :=  AI[3];
     Length  :=  Length  +  CalcEA(DecInstr.Operand[1],  AMS1,  DecInstr.Modifier,
              0,  AI)  -  1;
   IF  PassNo  =  2  THEN
       CASE  DecInstr.Modifier  OF
     'B':
         AI[1]  :=  AI[1]  +  Word{12}
     |
     'W':
         AI[1]  :=  AI[1]  +  Word{12,  13}
     |
     'L':
         AI[1]  :=  AI[1]  +  Word{13}
     ELSE
          Raise(Code,  Error,  ModifierError)
     END;
   END;
   RETURN  Length
END  Type1;


 PROCEDURE  Type2():  CARDINAL;
(*
         Format:  OOOORRRMMMEEEEEE
         Assembles:  ADD,  SUB,  variant  of  CMP,  variant  of  AND
                     (Motorola:  ADD,  SUB,  CMP,  AND).
*)

VAR
 Length             :  CARDINAL;
 Register           :  String;
 LeftToRight        :  BOOLEAN;

BEGIN
     InitialiseAString(Register);
     IF  CheckRegister(DecInstr.Operand[1].Argument,  Register)  THEN
      IF  (DecInstr.NoOfOperands  =  2)  AND
```

```
            (DecInstr.Operand[1].NoOfIndices = 0) THEN
         LeftToRight := TRUE;
            Length := CalcEA(DecInstr.Operand[2], AMS1, DecInstr.Modifier, 0, AI);
     ELSE
          Raise(Code, Error, OperandNoError)
     END
      ELSIF CheckRegister(DecInstr.Operand[2].Argument, Register) THEN
       IF (DecInstr.NoOfOperands = 2) AND
            (DecInstr.Operand[2].NoOfIndices = 0) THEN
         LeftToRight := FALSE;
           Length := CalcEA(DecInstr.Operand[1], AMS1,
                            DecInstr.Modifier, 0, AI);
     ELSE
          Raise(Code, Error, OperandNoError)
     END
   ELSE
        Raise(Code, Error, RegisterError)
   END;
   IF PassNo = 2 THEN
        AddRegister(AI[1], Register, 9);
        IF RegisterType(Register) = Data THEN
          CASE DecInstr.Modifier OF
        'B':
            IF LeftToRight THEN
              AI[1] := AI[1] + Word{8}
          END
       |
        'W':
            IF LeftToRight THEN
              AI[1] := AI[1] + Word{8}
          END;
            AI[1] := AI[1] + Word{6}
       |
        'L':
            IF LeftToRight THEN
              AI[1] := AI[1] + Word{8}
          END;
            AI[1] := AI[1] + Word{7}
        ELSE
             Raise(Code, Error, ModifierError)
        END
     ELSE
         AI[1] := AI[1] + Word{6,7};
          CASE DecInstr.Modifier OF
        'L':
            AI[1] := AI[1] + Word{8}
       |
        'W':;
        ELSE
             Raise(Code, Error, ModifierError)
        END
     END
   END;
    RETURN Length
 END Type2;


  PROCEDURE Type3(): CARDINAL;
 (*
          Format: OOOORRROOOEEEEEE
           Assembles: MULU, MUL, DIVU, DIV, CHK (Motorola - MULU, MULS, DIVU,
                      DIVS, CHK).
 *)
```

```
VAR
  Length        : CARDINAL;
  Register      : String;

BEGIN
     Length  :=  CalcEA(DecInstr.Operand[1],  AMS1,  DecInstr.Modifier,  0,  AI);
  IF  PassNo  =  2  THEN
      IF  DecInstr.Modifier  <>  'W'  THEN
         Raise(Code,  Error,  ModifierError)
    ELSE
         IF  DecInstr.Operand[2].NoOfIndices  =  0  THEN
           InitialiseAString(Register);
             IF  CheckRegister(DecInstr.Operand[2].Argument,  Register)  THEN
            IF  RegisterType(Register)  =  Data  THEN
              AddRegister(AI[1],  Register,  9)
          ELSE
                Raise(Code,  Error,  WrongRegisterError)
          END
        ELSE
             Raise(Code,  Error,  RegisterError)
        END
      ELSE
           Raise(Code,  Error,  OperandNoError)
      END
    END
  END;
   RETURN  Length
END  Type3;


 PROCEDURE  Type4():  CARDINAL;
(*
        Format:  OOOORRRMMMOOORRR
*)

BEGIN
  RETURN  0
END  Type4;


 PROCEDURE  Type5():  CARDINAL;
(*
        Format:  OOOORRROSSOOORRR
         Assembles:  ADDC,  SUBC  instructions  (Motorola  -  ADDX,  SUBX)
*)

 VAR  Register1,  Register2:  String;

BEGIN
  IF  PassNo  =  2  THEN
      IF  DecInstr.Modifier  =  'L'  THEN
        AI[1]  :=  AI[1]  +  Word{7}
      ELSIF  DecInstr.Modifier  =  'W'  THEN
        AI[1]  :=  AI[1]  +  Word{6}
      ELSIF  DecInstr.Modifier  <>  'B'  THEN
         Raise(Code,  Error,  ModifierError)
    END;
       IF  (DecInstr.NoOfOperands  =  2)  AND  (DecInstr.Operand[1].NoOfIndices  =  0)
          AND  (DecInstr.Operand[2].NoOfIndices  =  0)  THEN
         InitialiseAString(Register1);
           IF  CheckRegister(DecInstr.Operand[1].Argument,  Register1)  THEN
           IF  RegisterType(Register1)  =  Data  THEN
              InitialiseAString(Register2);
                IF  CheckRegister(DecInstr.Operand[2].Argument,  Register2)  THEN
```

```
                IF RegisterType(Register2) = Data THEN
                  AddRegister(AI[1], Register1, 0);
                  AddRegister(AI[1], Register2, 9)
            ELSE
                  Raise(Code, Error, WrongRegisterError)
            END
          ELSE
                Raise(Code, Error, RegisterError)
          END
        ELSE
              Raise(Code, Error, WrongRegisterError)
        END
      ELSE
           Raise(Code, Error, RegisterError)
      END
    ELSE
         Raise(Code, Error, OperandNoError)
    END
  END;
  RETURN 1
END Type5;


 PROCEDURE Type6(): CARDINAL;
(*
        Format: OOOORRROOOOOORRR
         Assembles: SUBD, ADDD instructions (Motorola - SBCD, ABCD)
                     Current implementation allows only Data Register to
                   Data register operation.
*)

 VAR Register1, Register2: String;

BEGIN
  IF PassNo = 2 THEN
      IF DecInstr.Modifier = 'B' THEN
          IF (DecInstr.NoOfOperands = 2) AND (DecInstr.Operand[1].NoOfIndices = 0)
             AND (DecInstr.Operand[2].NoOfIndices = 0) THEN
           InitialiseAString(Register1);
             IF CheckRegister(DecInstr.Operand[1].Argument, Register1) THEN
            IF RegisterType(Register1) = Data THEN
               InitialiseAString(Register2);
                  IF CheckRegister(DecInstr.Operand[2].Argument, Register2) THEN
                 IF RegisterType(Register2) = Data THEN
                   AddRegister(AI[1], Register1, 0);
                   AddRegister(AI[1], Register2, 9)
              ELSE
                    Raise(Code, Error, WrongRegisterError)
              END
            ELSE
                  Raise(Code, Error, RegisterError)
            END
          ELSE
                Raise(Code, Error, WrongRegisterError)
          END
        ELSE
             Raise(Code, Error, RegisterError)
        END
      ELSE
           Raise(Code, Error, OperandNoError)
      END
    ELSE
         Raise(Code, Error, ModifierError)
    END
```

```
  END;
  RETURN 1
END Type6;


 PROCEDURE Type7(): CARDINAL;
(*
        Format: OOOORRRO88888888
*)

BEGIN
  RETURN 0
END Type7;


 PROCEDURE Type8(): CARDINAL;
(*
        Format: OOOO111OSSOOORRR
*)

BEGIN
  RETURN 0
END Type8;


 PROCEDURE Type9(): CARDINAL;
(*
        Format: OOOODDDOSSEEEEEE
*)

BEGIN
  RETURN 0
END Type9;


 PROCEDURE Type10(): CARDINAL;
(*
        Format: OOOOCCCCOOEEEEEE
         Assembles: Scc Instructions (Condition code field is implied in the
       opcode mask.
*)

BEGIN
     RETURN CalcEA(DecInstr.Operand[1], AMS4, DecInstr.Modifier, 0, AI)
END Type10;


 PROCEDURE Type11(): CARDINAL;
(*
        Format: OOOOCCCCPPPPPPPP
         Assembles: Bcc Instructions (Condition code field is implied in the
       opcode mask).
*)

VAR
    AddrMode      : ValidAddressingModes;
    Displacement: LONGINT;
   Dest          : LONGINT;
    DestType      : SymbolStatus;

BEGIN
   IF PassNo = 1 THEN
      IF DecInstr.Modifier = 'B' THEN
       RETURN 1
```

```
      ELSIF DecInstr.Modifier = 'W' THEN
       RETURN 2
    ELSE
      RETURN 1
    END
  ELSE
        ScanOperand(DecInstr.Operand[1], AddrMode);
      IF (AddrMode = Rel) OR (AddrMode = Dir) THEN
           Evaluate(DecInstr.Operand[1].Argument, Dest, DestType);
        IF (DestType = Relative) OR (AddrMode = Rel) THEN
            Displacement := Dest - LONGINT(CurrentLocation()) - 2;
          IF DecInstr.Modifier = 'B' THEN
              IF (Displacement >= -128) AND (Displacement <= 127) THEN
                AddBinary(AI[1], Word(Displacement), 0, 8);
            RETURN 1
          ELSE
                Raise(Code, Error, OutOfRangeError)
          END
          ELSIF DecInstr.Modifier = 'W' THEN
              IF (Displacement >= -32768) AND (Displacement <= 32767) THEN
               AI[2] := Word(Displacement);
            RETURN 2
          ELSE
                Raise(Code, Error, OutOfRangeError)
          END
        ELSE
              Raise(Code, Error, ModifierError)
        END
      ELSE
           Raise(Code, Error, RelativeError)
      END
    ELSE
         Raise(Code, Error, AddrModeError)
    END;
    RETURN 1
  END
END Type11;


 PROCEDURE Type12(): CARDINAL;
(*
        Format: OOOOCCCCOOOOORRR
         Assembles: DBcc Instructions (Condition code field is implied in the
       opcode mask).
*)

VAR
   Register     : String;
   Length       : CARDINAL;
    AddrMode      : ValidAddressingModes;
   Dest         : LONGINT;
    Displacement: LONGINT;
   DestType      : SymbolStatus;

BEGIN
   IF PassNo = 2 THEN
      IF DecInstr.Modifier = 'W' THEN
          InitialiseAString(Register);
            IF CheckRegister(DecInstr.Operand[1].Argument, Register) THEN
              ScanOperand(DecInstr.Operand[1], AddrMode);
          IF AddrMode = Dir THEN
              IF RegisterType(Register) = Data THEN
                AddRegister(AI[1], Register, 0);
                  ScanOperand(DecInstr.Operand[2], AddrMode);
```

```
                IF (AddrMode = Rel) OR (AddrMode = Dir) THEN
                      Evaluate(DecInstr.Operand[2].Argument, Dest, DestType);
                   IF (DestType = Relative) OR (AddrMode = Rel) THEN
                      Displacement := Dest - LONGINT(CurrentLocation()) - 2;
                      IF (Displacement >= -32768) AND (Displacement <= 32767) THEN
                       AI[2] := Word(Displacement)
                  ELSE
                        Raise(Code, Error, OutOfRangeError)
                  END
                ELSE
                     Raise(Code, Error, RelativeError)
                END
              ELSE
                   Raise(Code, Error, AddrModeError)
              END
            ELSE
                 Raise(Code, Error, WrongRegisterError)
            END
          ELSE
               Raise(Code, Error, AddrModeError)
          END
        ELSE
             Raise(Code, Error, AddrModeError)
        END
      ELSE
           Raise(Code, Error, ModifierError)
      END
    END;
    RETURN 2
  END Type12;


  PROCEDURE Type13(): CARDINAL;
 (*
          Format: OOOOOOOOSSEEEEEE
           Assembles: NEG, NEGC, NOT, TEST, CLR, immediate ADD, immediate SUB
          (Motorola - NEG, NEGX, NOT, TST, CLR, ADDI, SUBI).
 *)
 VAR
     Length: CARDINAL;
     SaveWord: Word;

 BEGIN
     IF DecInstr.Modifier = 'W' THEN
       INCL(AI[1], 6)
     ELSIF DecInstr.Modifier = 'L' THEN
       INCL(AI[1], 7)
     ELSIF DecInstr.Modifier <> 'B' THEN
        Raise(Code, Error, ModifierError)
   END;
      IF EqualStrings("ADD", DecInstr.Command) OR
          EqualStrings("SUB", DecInstr.Command) THEN
         Length := CalcEA(DecInstr.Operand[2], AMS4, DecInstr.Modifier, 0, AI);
      AI[4] := AI[2];
      AI[5] := AI[3];
      SaveWord := AI[1];
         Length := Length + CalcEA(DecInstr.Operand[1], AddressingModeSet{Imm},
                  DecInstr.Modifier, 0, AI) - 1;
      AI[1] := SaveWord;
      RETURN Length
   ELSE
         RETURN CalcEA(DecInstr.Operand[1], AMS1, DecInstr.Modifier, 0, AI)
   END
 END Type13;
```

```
 PROCEDURE  Type14():  CARDINAL;
(*
         Format:  OOOOOOOOOSEEEEEE
         Assembles:  LDM,  STM  (Motorola  -  MOVEM)
*)

VAR
   Length        :  CARDINAL;
    AddrMode       :  ValidAddressingModes;
   RegMask       :  LONGINT;
    RegMaskType  :  SymbolStatus;
   VAMS           :  AddressingModeSet;

BEGIN
   IF  PassNo  =  2  THEN
       IF  DecInstr.Modifier  =  'L'  THEN
         INCL(AI[1],  6)
       ELSIF  DecInstr.Modifier  =  'B'  THEN
          Raise(Code,  Error,  ModifierError)
     END
   END;
      IF  EqualStrings("STM",  DecInstr.Command)  THEN
      VAMS  :=  AMS2
   ELSE
      VAMS  :=  AMS3
   END;
      ScanOperand(DecInstr.Operand[2],  AddrMode);
      Length  :=  CalcEA(DecInstr.Operand[2],  VAMS,  DecInstr.Modifier,  0,  AI);
   IF  PassNo  =  1  THEN
      RETURN  Length  +  1
   END;
      ScanOperand(DecInstr.Operand[1],  AddrMode);
   IF  AddrMode  =  Imm  THEN
         Evaluate(DecInstr.Operand[1].Argument,  RegMask,  RegMaskType)
   ELSE
        Raise(Code,  Error,  AddrModeError);
      RegMask  :=  0;
       RegMaskType  :=  Absolute
   END;
   IF  Length  >=  3  THEN
      AI[4]  :=  AI[3];
      IF  Length  >=  2  THEN
        AI[3]  :=  AI[2];
     END
   END;
    IF  RegMaskType  =  Absolute  THEN
       IF  (RegMask  >=  0)  AND  (RegMask  <=  65536)  THEN
         AI[2]  :=  Word(RegMask)
     ELSE
          Raise(Code,  Error,  OutOfRangeError)
     END
   ELSE
       Raise(Code,  Error,  AbsoluteError)
   END;
    RETURN  Length  +  1
END  Type14;


 PROCEDURE  Type15():  CARDINAL;
(*
         Format:  OOOOOOOOOOEEEEEE
         Assembles:  NBCD,  (Motorola  -  JMP,  JSR,  PEA)
```

```
*)

BEGIN
     RETURN  CalcEA(DecInstr.Operand[1],  AMS1,  DecInstr.Modifier,  0,  AI)
END  Type15;


 PROCEDURE  Type16():  CARDINAL;
(*
         Format:  OOOOOOOOOOOOVVVV
        Assembles:  TRAP
*)

VAR
     AddrMode:  ValidAddressingModes;
    Vector:  LONGINT;
    RType:  SymbolStatus;

BEGIN
   IF  PassNo  =  1  THEN
     RETURN  1
  ELSE
         ScanOperand(DecInstr.Operand[1],  AddrMode);
      IF  AddrMode  =  Imm  THEN
         IF  (DecInstr.NoOfOperands  =  1)  AND
              (DecInstr.Operand[1].NoOfIndices  =  0)  THEN
              Evaluate(DecInstr.Operand[1].Argument,  Vector,  RType);
           IF  RType  =  Absolute  THEN
              IF  (Vector  >=  0)  AND  (Vector  <=  15)  THEN
                 AddBinary(AI[1],  Word(Vector),  0,  4);
              RETURN  1
           ELSE
                 Raise(Code,  Error,  Type16ErrorA)
           END
         ELSE
              Raise(Code,  Error,  AbsoluteError)
         END
       ELSE
            Raise(Code,  Error,  OperandNoError)
       END
     ELSE
          Raise(Code,  Error,  AddrModeError)
     END;
     RETURN  1
  END
END  Type16;


 PROCEDURE  Type17():  CARDINAL;
(*
         Format:  OOOOOOOOOOOOORRR
        Assembles:  EXT
*)
 VAR  Register:  String;

BEGIN
   IF  PassNo  =  1  THEN
     RETURN  1
  ELSE
       AI[1]  :=  CommandEntry.Mask;
        InitialiseAString(Register);
         IF  CheckRegister(DecInstr.Operand[1].Argument,  Register)  THEN
         IF  (DecInstr.NoOfOperands  =  1)  AND
              (DecInstr.Operand[1].NoOfIndices  =  0)  THEN
```

```
            IF RegisterType(Register) = Data THEN
               AddRegister(AI[1], Register, 0);
              IF DecInstr.Modifier = 'B' THEN
               AI[1] := AI[1] + Word{7}
              ELSIF DecInstr.Modifier = 'W' THEN
               AI[1] := AI[1] + Word{6,7}
            ELSE
                  Raise(Code, Error, ModifierError);
            END
          ELSE
                Raise(Code, Error, WrongRegisterError)
          END
        ELSE
             Raise(Code, Error, OperandNoError)
        END
      ELSE
           Raise(Code, Error, RegisterError);
      END;
      RETURN 1
   END
 END Type17;


  PROCEDURE Type18(): CARDINAL;
 (*
          Format: OOOOOOOOOOOOOOOO
 *)

 BEGIN
    IF PassNo = 1 THEN
      RETURN 1
   ELSE
        AI[1] := CommandEntry.Mask;
      RETURN 1
   END
 END Type18;


  PROCEDURE Type19(): CARDINAL;

 (*
         Format: VARIABLE
          Assembles: Takes the standard mnemonic of ADD or SUB
                       and converts into Motorola format ADD/SUB or ADDA/SUBA or
                       or ADDI/SUBI (ADDQ/SUBQ is not supported). The appropriate
                     type is the called from this.
 *)

 VAR
     Op1AddrMode, Op2AddrMode       : ValidAddressingModes;
    Register1, Register2           : String;

 BEGIN
      InitialiseAString(Register1);
      InitialiseAString(Register2);
      ScanOperand(DecInstr.Operand[1], Op1AddrMode);
      ScanOperand(DecInstr.Operand[2], Op2AddrMode);

      IF CheckRegister(DecInstr.Operand[2].Argument, Register2) THEN
        IF (DecInstr.NoOfOperands = 2) AND
             (DecInstr.Operand[2].NoOfIndices = 0) THEN
          RETURN Type2()
      ELSE
           Raise(Code, Error, OperandNoError)
```

```
    END
     ELSIF  CheckRegister(DecInstr.Operand[1].Argument,  Register1)  THEN
       IF  RegisterType(Register1)  =  Data  THEN
         IF  (DecInstr.NoOfOperands  =  2)  AND
              (DecInstr.Operand[2].NoOfIndices  =  0)  THEN
            RETURN  Type2()
       ELSE
            Raise(Code,  Error,  OperandNoError)
       END
    ELSE
         Raise(Code,  Error,  WrongRegisterError)
    END
   ELSIF  Op1AddrMode  =  Imm  THEN
      IF  (DecInstr.NoOfOperands  =  2)  AND
           (DecInstr.Operand[2].NoOfIndices  =  0)  THEN
        RETURN  Type13()
    ELSE
         Raise(Code,  Error,  OperandNoError)
    END
  ELSE
       Raise(Code,  Error,  AddrModeError)
  END
END  Type19;


 PROCEDURE  Type20():  CARDINAL;
BEGIN
  RETURN  0
END  Type20;


 PROCEDURE  Type21():  CARDINAL;
(*
        Format:  VARIABLE
         Assembles:  All  Shift  Instructions.
*)

VAR
    Register1,  Register2:  String;
   Shift:  LONGINT;
    ShiftType:  SymbolStatus;
     AddrMode:  ValidAddressingModes;

BEGIN
     InitialiseAString(Register1);
     InitialiseAString(Register2);
   IF  DecInstr.NoOfOperands  =  1  THEN
     IF  PassNo  =  2  THEN
        AI[1]  :=  AI[1]  +  Word{9,  7,  6}
    END;
        RETURN  CalcEA(DecInstr.Operand[1],  AMS5,  DecInstr.Modifier,  0,  AI)
  ELSE
     IF  PassNo  =  1  THEN
       RETURN  1
    END;
         IF  CheckRegister(DecInstr.Operand[2].Argument,  Register2)  THEN
         IF  RegisterType(Register2)  =  Data  THEN
           CASE  DecInstr.Modifier  OF
         'B':;
         |
         'W':
             AI[1]  :=  AI[1]  +  Word{6}
         |
         'L':
```

```
            AI[1] := AI[1] + Word{7}
        ELSE
             Raise(Code, Error, ModifierError)
        END;
           IF DecInstr.Operand[2].NoOfIndices = 0 THEN
             AddRegister(AI[1], Register2, 0);
                IF CheckRegister(DecInstr.Operand[1].Argument, Register1) THEN
               IF RegisterType(Register1) = Data THEN
                  IF DecInstr.Operand[1].NoOfIndices = 0 THEN
                    AddRegister(AI[1], Register1, 9);
                  AI[1] := AI[1] + Word{5};
              ELSE
                    Raise(Code, Error, OperandNoError)
              END
            ELSE
                  Raise(Code, Error, WrongRegisterError)
            END
          ELSE
                 ScanOperand(DecInstr.Operand[1], AddrMode);
             IF AddrMode = Imm THEN
                    Evaluate(DecInstr.Operand[1].Argument, Shift, ShiftType);
                 IF ShiftType = Absolute THEN
                   IF (Shift >= 1) AND (Shift <= 8) THEN
                    IF Shift <> 8 THEN
                          AddBinary(AI[1], Word(INTEGER(Shift)), 9, 3);
                   END
                 ELSE
                       Raise(Code, Error, OutOfRangeError)
                 END
              ELSE
                    Raise(Code, Error, AbsoluteError)
              END
            ELSE
                 Raise(Code, Error, AddrModeError)
            END
          END
        ELSE
             Raise(Code, Error, AddrModeError)
        END
      ELSE
           Raise(Code, Error, WrongRegisterError)
      END
    ELSE
         Raise(Code, Error, RegisterError)
    END
  END;
  RETURN 1
END Type21;


 PROCEDURE Type22(): CARDINAL;
(*
        Format: VARIABLE
          Selects: between two types of format possible with instructions
                   TEST1, TESTSET, TESTCLR, TESTNOT.
*)

VAR
    Register: String;
     AddrMode: ValidAddressingModes;
    Result: LONGINT;
     ResultType: SymbolStatus;
    Length: CARDINAL;
```

```
BEGIN
     InitialiseAString(Register);
     IF  CheckRegister(DecInstr.Operand[1].Argument,  Register)  THEN
       IF  RegisterType(Register)  =  Data  THEN
        RETURN  Type3()
    ELSE
          Raise(Code,  Error,  WrongRegisterError);
       RETURN  0
    END
  ELSE
         ScanOperand(DecInstr.Operand[1],  AddrMode);
     IF  AddrMode  =  Imm  THEN
        Length  :=  Type15();
            Evaluate(DecInstr.Operand[2].Argument,  Result,  ResultType);
         IF  (Result  >  0)  AND  (Result  <=  256)  AND  (ResultType  =  Absolute)  THEN
           AI[2]  :=  Word(CARDINAL(Result));
         RETURN  2
       ELSE
            Raise(Code,  Error,  OutOfRangeError);
         RETURN  0
       END
    ELSE
         Raise(Code,  Error,  AddrModeError);
       RETURN  0
    END
  END
END  Type22;


 PROCEDURE  Type23():  CARDINAL;
(*
         Format:  OOSSEEEEEEEEEEEE
         Selects:  between  LD,  ST  and  MOV  instructions.
                   (Motorola  -  MOVE,  MOVEA)

         Load  instructions  operands  are  swapped  because  they  are  the
         opposite  way  around  to  the  Motorola  standard.
*)
VAR
   Register:  String;
    TempOp1,  TempOp2:  OperandElement;

BEGIN
     IF  CheckRegister(DecInstr.Operand[1].Argument,  Register)  THEN
         IF  NOT  CheckRegister(DecInstr.Operand[2].Argument,  Register)  THEN
         (*  Must  be  either  LD  or  ST  instruction.  *)
           IF  EqualStrings(DecInstr.Command,  "LD")  THEN
            (*  LD  instruction  so  swap  operands.  *)
            TempOp1  :=  DecInstr.Operand[1];
            TempOp2  :=  DecInstr.Operand[2];
            DecInstr.Operand[1]  :=  TempOp2;
            DecInstr.Operand[2]  :=  TempOp1;
          RETURN  Type1()
           ELSIF  EqualStrings(DecInstr.Command,  "ST")  THEN
            (*  ST  instruction  -  no  swap  necessary.  *)
          RETURN  Type1()
       ELSE
          IF PassNo  =  2  THEN
              Raise(Code,  Error,  AddrModeError);
         END;
         RETURN  0
       END
    ELSE
         (*  Must  be  a  register  to  register  MV  instruction.  *)
```

```
        RETURN Type1()
      END
    ELSE
      (*
            First operand is not a register. Must be a memory to memory
           MV instruction.
      *)
       RETURN Type1()
    END
  END Type23;


   PROCEDURE Type24(): CARDINAL;
  (*
         Format: VARIABLE
           Assembles: XCH by selecting between Motorola equivalents of EXG
                    and SWAP.
  *)
  VAR
      Register1, Register2: String;

  BEGIN
      IF DecInstr.NoOfOperands = 1 THEN
        RETURN Type17()
    ELSE
         InitialiseAString(Register1);
         InitialiseAString(Register2);
          IF CheckRegister(DecInstr.Operand[1].Argument, Register1) AND
               CheckRegister(DecInstr.Operand[2].Argument, Register2) THEN
           IF RegisterType(Register1) = Data THEN
              IF RegisterType(Register2) = Data THEN
               AI[1] := AI[1] + Word{6}
          ELSE
               AI[1] := AI[1] + Word{3, 7}
          END
        ELSE
              IF RegisterType(Register2) = Data THEN
                 Raise(Code, Error, WrongRegisterError)
          ELSE
               AI[1] := AI[1] + Word{3, 6}
          END
        END;
         RETURN Type6()
      ELSE
           Raise(Code, Error, RegisterError);
         RETURN 0
      END
    END
  END Type24;


   PROCEDURE Type25(): CARDINAL;
  (*
         Format: VARIABLE
           Assembles: CALL by selecting between motorola equivalents of
                    BSR and JSR.
  *)

  VAR
       AddrMode: ValidAddressingModes;

  BEGIN
      IF DecInstr.NoOfOperands = 1 THEN
           ScanOperand(DecInstr.Operand[1], AddrMode);
```

```
        IF AddrMode = Imm THEN
           (* Must be a BSR instruction. *)
           RETURN Type11()
        ELSE
           RETURN Type15()
        END
      ELSE
          Raise(Code, Error, OperandNoError)
      END
    END Type25;


    PROCEDURE Type26(): CARDINAL;
    BEGIN
      RETURN 0
    END Type26;


BEGIN
   IF PassNo = 1 THEN
       IF NOT EmptyString(DecInstr.Label) THEN
          IF NOT(IsIn(DecInstr.Label)) THEN
               InsertSymbol(DecInstr.Label, CurrentLocation(), Relative)
       ELSE
            Raise(Code, Warning, LabelError)
       END
     END
   END;
    IF NOT(EmptyString(DecInstr.Command)) THEN
       IF IsIn(DecInstr.Command) THEN
           CommandEntry := Retrieve(DecInstr.Command);
         AI[1] := CommandEntry.Mask;
         CASE CommandEntry.Type OF
          1:       Length := Type1()       |
          2:       Length := Type2()       |
          3:       Length := Type3()       |
          4:       Length := Type4()       |
          5:       Length := Type5()       |
          6:       Length := Type6()       |
          7:       Length := Type7()       |
          8:       Length := Type8()       |
          9:       Length := Type9()       |
          10:      Length := Type10()      |
          11:      Length := Type11()      |
          12:      Length := Type12()      |
          13:      Length := Type13()      |
          14:      Length := Type14()      |
          15:      Length := Type15()      |
          16:      Length := Type16()      |
          17:      Length := Type17()      |
          18:      Length := Type18()      |
          19:      Length := Type19()      |
          20:      Length := Type20()      |
          21:      Length := Type21()      |
          22:      Length := Type22()      |
          23:      Length := Type23()      |
          24:      Length := Type24()      |
          25:      Length := Type25()      |
          26:      Length := Type26()
       ELSE
              AssemblePseudoOp(DecInstr, PassNo, CommandEntry.Type, Length)
       END
     ELSE
         Raise(Code, Error, CommandError)
```

```
    END
  END
END  Assemble;


PROCEDURE  InsertOpcodesInTable;
BEGIN
  InsertCommand("ADD",            Word{12,14,15},                    19);
  InsertCommand("ADDD",           Word{8,14,15},                     6);
  InsertCommand("ADDC",           Word{8,12,14,15},                  5);
  InsertCommand("SUB",            Word{12,15},                       19);
  InsertCommand("SUBD",           Word{8,15},                        6);
  InsertCommand("SUBC",           Word{8,12,15},                     5);
  InsertCommand("MULU",           Word{6,7,14,15},                   3);
  InsertCommand("MUL",            Word{6,7,8,14,15},                 3);
  InsertCommand("DIVU",           Word{6,7,15},                      3);
  InsertCommand("DIV",            Word{6,7,8,15},                    3);
  InsertCommand("CMP",            Word{12,13,15},                    19);
  InsertCommand("NEG",            Word{10,14},                       13);
  InsertCommand("NEGC",           Word{14},                          13);
  InsertCommand("NEGD",           Word{11,14},                       15);
  InsertCommand("EXT",            Word{11,14},                       17);
  InsertCommand("DBR",            Word{3,6,7,12,14},                 12);
  InsertCommand("DBE",            Word{3,6,7,8,9,10,12,14},          12);
  InsertCommand("DBNE",           Word{3,6,7,9,10,12,14},            12);
  InsertCommand("DBC",            Word{3,6,7,8,10,12,14},            12);
  InsertCommand("DBNC",           Word{3,6,7,10,12,14},              12);
  InsertCommand("DBP",            Word{3,6,7,9,11,12,14},            12);
  InsertCommand("DBN",            Word{3,6,7,8,9,11,12,14},          12);
  InsertCommand("DBV",            Word{3,6,7,8,11,12,14},            12);
  InsertCommand("DBNV",           Word{3,6,7,11,12,14},              12);
  InsertCommand("DBGT",           Word{3,6,7,9,10,11,12,14},         12);
  InsertCommand("DBGE",           Word{3,6,7,10,11,12,14},           12);
  InsertCommand("DBLT",           Word{3,6,7,8,10,11,12,14},         12);
  InsertCommand("DBLE",           Word{3,6,7,8,9,10,11,12,14},       12);
  InsertCommand("DBH",            Word{3,6,7,9,12,14},               12);
  InsertCommand("DBNH",           Word{3,6,7,8,12,14},               12);
  InsertCommand("AND",            Word{14,15},                       19);
  InsertCommand("OR",             Word{15},                          19);
  InsertCommand("XOR",            Word{12,13,15},                    19);
  InsertCommand("NOT",            Word{9,10,14},                     13);
  InsertCommand("SHR",            Word{3,13,14,15},                  21);
  InsertCommand("SHL",            Word{3,8,13,14,15},                21);
  InsertCommand("SHRA",           Word{13,14,15},                    21);
  InsertCommand("SHLA",           Word{8,13,14,15},                  21);
  InsertCommand("ROR",            Word{3,4,13,14,15},                21);
  InsertCommand("ROL",            Word{3,4,8,13,14,15},              21);
  InsertCommand("RORC",           Word{4,13,14,15},                  21);
  InsertCommand("ROLC",           Word{4,8,13,14,15},                21);
  InsertCommand("TEST",           Word{9,11,14},                     13);
  InsertCommand("TEST1",          Word{8},                           22);
  InsertCommand("TESTSET",        Word{6,7,8},                       22);
  InsertCommand("TESTCLR",        Word{7,8},                         22);
  InsertCommand("TESTNOT",        Word{6,8},                         22);
  InsertCommand("CHK",            Word{7,8,14},                      3);
  InsertCommand("LD",             Word{},                            23);
  InsertCommand("LDM",            Word{7,10,11,14},                  14);
  InsertCommand("ST",             Word{},                            23);
  InsertCommand("STM",            Word{7,11,14},                     14);
  InsertCommand("MOV",            Word{},                            23);
  InsertCommand("XCH",            Word{8,14,15},                     24);
  InsertCommand("CLR",            Word{9,14},                        13);
  InsertCommand("SET",            Word{6,7,12,14},                   10);
  InsertCommand("SETE",           Word{6,7,8,9,10,12,14},            10);
```

```
   InsertCommand("SETNE",          Word{6,7,9,10,12,14},              10);
   InsertCommand("SETC",           Word{6,7,8,10,12,14},              10);
   InsertCommand("SETNC",          Word{6,7,10,12,14},                10);
   InsertCommand("SETP",           Word{6,7,9,11,12,14},              10);
   InsertCommand("SETN",           Word{6,7,8,9,11,12,14},            10);
   InsertCommand("SETV",           Word{6,7,8,11,12,14},              10);
   InsertCommand("SETNV",          Word{6,7,11,12,14},                10);
   InsertCommand("SETGT",          Word{6,7,9,10,11,12,14},           10);
   InsertCommand("SETGE",          Word{6,7,10,11,12,14},             10);
   InsertCommand("SETLT",          Word{6,7,8,10,11,12,14},           10);
   InsertCommand("SETLE",          Word{6,7,8,9,10,11,12,14},         10);
   InsertCommand("SETH",           Word{6,7,9,12,14},                 10);
   InsertCommand("SETNH",          Word{6,7,8,9,12,14},               10);
   InsertCommand("BR",             Word{13,14},                       25);
   InsertCommand("BE",             Word{8,9,10,13,14},                11);
   InsertCommand("BNE",            Word{9,10,13,14},                  11);
   InsertCommand("BC",             Word{8,10,13,14},                  11);
   InsertCommand("BNC",            Word{10,13,14},                    11);
   InsertCommand("BP",             Word{9,11,13,14},                  11);
   InsertCommand("BN",             Word{8,9,11,13,14},                11);
   InsertCommand("BV",             Word{8,11,13,14},                  11);
   InsertCommand("BNV",            Word{11,13,14},                    11);
   InsertCommand("BGT",            Word{9,10,11,13,14},               11);
   InsertCommand("BGE",            Word{10,11,13,14},                 11);
   InsertCommand("BLT",            Word{8,10,11,13,14},               11);
   InsertCommand("BLE",            Word{8,9,10,11,13,14},             11);
   InsertCommand("BH",             Word{9,13,14},                     11);
   InsertCommand("BNH",            Word{8,9,13,14},                   11);
   InsertCommand("CALL",           Word{8,13,14},                     25);
   InsertCommand("RET",            Word{0,2,4,5,6,9,10,11,14},        18);
   InsertCommand("RETR",           Word{0,1,2,4,5,6,9,10,11,14},      18);
   InsertCommand("RETE",           Word{0,1,4,5,6,9,10,11,14},        18);
   InsertCommand("NOP",            Word{0,4,5,6,9,10,11,14},          18);
   InsertCommand("PUSH",           Word{6,11,14},                     26);
   InsertCommand("POP",            Word{6,7,8,14}, (* ** *)           26);
   InsertCommand("WAIT",           Word{1,4,5,6,9,10,11,14},          18);
   InsertCommand("BRK",            Word{6,9,10,11,14},                16);
   InsertCommand("BRKV",           Word{1,2,4,5,6,9,10,11,14},        18);
   InsertCommand("RESET",          Word{4,5,6,9,10,11,14},            18)
END  InsertOpcodesInTable;


PROCEDURE  ADMShutdown;
END  ADMShutdown;


BEGIN
    ValidRegisters[1]  :=  "D0";
    ValidRegisters[2]  :=  "D1";
    ValidRegisters[3]  :=  "D2";
    ValidRegisters[4]  :=  "D3";
    ValidRegisters[5]  :=  "D4";
    ValidRegisters[6]  :=  "D5";
    ValidRegisters[7]  :=  "D6";
    ValidRegisters[8]  :=  "D7";
    ValidRegisters[9]  :=  "A0";
    ValidRegisters[10]  :=  "A1";
    ValidRegisters[11]  :=  "A2";
    ValidRegisters[12]  :=  "A3";
    ValidRegisters[13]  :=  "A4";
    ValidRegisters[14]  :=  "A5";
    ValidRegisters[15]  :=  "A6";
    ValidRegisters[16]  :=  "A7";
    ValidRegisters[17]  :=  "SP";
END  ADM.
```

```
IMPLEMENTATION  MODULE  Exceptions;

(*
          MODULE: Exceptions        Version 1.0       (c) 1989 Mark Wickens

          Created: 28/11/89

            This module handles all exception processing for the meta-assembler.
*)

FROM Strings                  IMPORT   StringsShutdown;
FROM Interface                  IMPORT   WriteAString, WriteALine, ErrOut, WriteACard,
                                       InterfaceShutdown;
FROM Table                    IMPORT   TableShutdown;
FROM TableExt                 IMPORT   TableExtShutdown;
FROM TableTrees               IMPORT   TableTreesShutdown;
FROM Listing                    IMPORT   ListingShutdown, AddError, CurrentlyListing,
                                       CurrentSourceLine;
FROM Lex                      IMPORT   LexShutdown;
FROM Location                 IMPORT   LocationShutdown;
FROM Expression               IMPORT   ExpressionShutdown;
FROM  ObjectGenerator         IMPORT    ObjectGeneratorShutdown,  StopObject;
FROM PseudoOps                IMPORT   PseudoOpsShutdown;

VAR
    ExceptionCount:  CARDINAL;
    CurrentStatus:  StatusType;


PROCEDURE  CallShutdowns;
BEGIN
    InterfaceShutdown;
    TableShutdown;
    ListingShutdown;
    LexShutdown;
    LocationShutdown;
    ExpressionShutdown;
    ObjectGeneratorShutdown;
    PseudoOpsShutdown
END  CallShutdowns;


PROCEDURE  Raise(Type:  ExceptionType;  Level:  ExceptionLevel;  Message:  String);

BEGIN
    INC(ExceptionCount);

   CASE Type OF
      Code:          IF CurrentlyListing() THEN
                       AddError(Message)
                  END;
                      WriteAString(ErrOut, "Source line ");
                       WriteACard(ErrOut, CurrentSourceLine(), 1);
                    WriteAString(ErrOut, " ") |
      User:         WriteAString(ErrOut, "User ") |
        Environment:WriteAString(ErrOut,  "Environment  ")  |
       Internal:     WriteAString(ErrOut, "Internal ")
   END;

   CASE Level OF
       Warning:      WriteAString(ErrOut, "warning: ") |
       Error:         WriteAString(ErrOut, "error: ");
                   StopObject;
                   CurrentStatus := NoCode              |
```

```
      Fatal:          WriteAString(ErrOut, "fatal error: ");
                      WriteAString(ErrOut, Message);
                     WriteALine(ErrOut);
                    CallShutdowns;
                   HALT
   END;

    WriteAString(ErrOut, Message);
    WriteALine(ErrOut)
END Raise;


PROCEDURE ChangeStatus(NewStatus: StatusType);
BEGIN
    CurrentStatus := NewStatus
END ChangeStatus;


PROCEDURE ProgramStatus(): StatusType;
BEGIN
    RETURN CurrentStatus
END ProgramStatus;


PROCEDURE Number(): CARDINAL;
BEGIN
    RETURN ExceptionCount
END Number;


BEGIN
    ExceptionCount := 0;
    CurrentStatus := Normal
END Exceptions.
```

```
IMPLEMENTATION  MODULE  Expression;

(*
          MODULE: Expression        Version 1.0       (c) 1990 Mark Wickens

          Created: 03-01-90
          Updated: 05-01-90         Added error messages.

           Evaluates an expression according to the symbols in the symbol table
          and constants defined.
*)

FROM Strings                  IMPORT   CharInString, LengthOfString, MakeSubstring,
                                       InitialiseAString, EndOfLine, EqualStrings,
                                     ArrayToString;
FROM TableExt                IMPORT   TableType, EntryType;
FROM Exceptions               IMPORT   Raise, ExceptionType, ExceptionLevel;
FROM Table                   IMPORT   IsIn, Retrieve;
FROM Location                IMPORT   CurrentLocation;
FROM ADM                       IMPORT   ValidRegisters, NOOFREGISTERS;

TYPE
   CharSet         = SET OF CHAR;

CONST
(* Character Sets. *)
   BaseSet          = CharSet{'B', 'Q', 'D', 'H'};
   BaseSepSet       = CharSet{"'"};
   BinSet           = CharSet{'0', '1'};
   OctSet           = CharSet{'0'..'7'};
   DecSet           = CharSet{'0'..'9'};
   HexSet             = CharSet{'0'..'9', 'a'..'f', 'A'..'F'};
   OperatorSet      = CharSet{'.', '+', '-', '*', '/'};
   RegisterSet      = CharSet{'.'};
   EOLSet           = CharSet{EndOfLine};

(* Errors. *)
   UnknownBaseError         = "Base of constant is illegal";
   StatusError                = "Expression cannot be evaluated";
   AddError                   = "Addition would cause overflow";
   SubError                   = "Subraction would cause underflow";
   MultError                  = "Multiplication would cause overflow";
   DivError                  = "Cannot divide by zero";
   WrongTypeError            = "Symbol in expression is not addressable";
   OperandError              = "Unknown operand in expression";
   OperatorError             = "Invalid operator in expression";
   ExpError                  = "Cannot raise to power less than 0";
   RegisterError             = "Unknown register";

VAR
    DefaultBase: BaseType;


PROCEDURE Evaluate(Input: String; VAR Result: LONGINT;
                                                    VAR ResultType: SymbolStatus);

TYPE
   OpType = (Add, Sub, Mult, DivSig, DivUn, And, Or, Xor, Not, Shl, Shr, Mod,
             Exp, Bit);

VAR
  Op1, Op2                                  : LONGINT;
  Op1Status, Op2Status                      : SymbolStatus;
  CurrentPos                                : CARDINAL;
```

```
CurrentChar                                  : CHAR;
Operator                                     : OpType;


  PROCEDURE  GetOperand(VAR  OperandValue:  LONGINT;
                                       VAR OperandStatus: SymbolStatus): BOOLEAN;

VAR
    Base:  BaseType;

      PROCEDURE  GetBase(VAR  Base:  BaseType):  BOOLEAN;

   VAR
      CurrentBase         : CHAR;

   BEGIN
         CurrentChar  :=  CAP(CharInString(Input,  CurrentPos));
       IF  CurrentChar  IN  BaseSet  THEN
          CurrentBase  :=  CurrentChar;
         INC(CurrentPos);
           CurrentChar  :=  CharInString(Input,  CurrentPos);
         IF  CurrentChar  IN  BaseSepSet  THEN
           CASE  CurrentBase  OF
             'B': Base  := Bin  |
             'Q': Base  := Oct  |
             'D': Base  := Dec  |
             'H': Base  := Hex
         ELSE
               Raise(Code,  Error,  UnknownBaseError);
            DEC(CurrentPos);
           RETURN  FALSE
         END;
           INC(CurrentPos);
          RETURN  TRUE
       ELSE
           DEC(CurrentPos);
          RETURN  FALSE
       END
     ELSE
        RETURN  FALSE
     END
    END  GetBase;


      PROCEDURE  GetConstant(VAR  OperandValue:  LONGINT;
                                       VAR OperandStatus: SymbolStatus): BOOLEAN;

   VAR
      ValidSet             : CharSet;
       ConstantStart       : CARDINAL;
      Length               : INTEGER;
       ConstantString      : String;
      Negative             : BOOLEAN;


         PROCEDURE  ConvertBase(In:  String;  Base:  BaseType):  LONGINT;

     VAR
         CurrentValue      : LONGINT;
        Index              : INTEGER;
         BaseValue         : CARDINAL;


            PROCEDURE  Power(Value:  LONGINT;  Position:  INTEGER;  Base:  CARDINAL)
```

```
                                                                    : LONGINT;

    VAR
       Sum              : LONGINT;
        Counter         : INTEGER;

    BEGIN
       Sum := 1;
         FOR Counter := 0 TO Position DO
            Sum := Sum * LONGINT(Base)
       END;
        RETURN Sum * Value
    END Power;


        PROCEDURE ValueOfChar(In: CHAR): LONGINT;
    BEGIN
       CASE In OF
           '0'..'9': RETURN LONGINT(ORD(In) - ORD('0'))        |
            'a'..'f': RETURN LONGINT(ORD(In) - ORD('a')) + 10 |
            'A'..'F': RETURN LONGINT(ORD(In) - ORD('A')) + 10
       ELSE
            Raise(Code, Error, OperandError);
          RETURN 0
       END
      END ValueOfChar;


  BEGIN
     CASE Base OF
        Bin: BaseValue := 2   |
        Oct: BaseValue := 8   |
        Dec: BaseValue := 10  |
        Hex: BaseValue := 16
    END;
     CurrentValue := 0;
        Length := INTEGER(LengthOfString(In));
     FOR Index := 1 TO Length DO
         CurrentValue := CurrentValue +
                                Power(ValueOfChar(CharInString(In, Index)),
                                    Length - Index - 1, BaseValue)
    END;
      RETURN CurrentValue
   END ConvertBase;


BEGIN
   CASE Base OF
      Bin: ValidSet := BinSet |
      Oct: ValidSet := OctSet |
      Dec: ValidSet := DecSet |
      Hex: ValidSet := HexSet
  END;
    ConstantStart := CurrentPos;
     CurrentChar := CharInString(Input, CurrentPos);
   IF CurrentChar = '-' THEN
     Negative := TRUE;
      INC(CurrentPos);
      INC(ConstantStart);
        CurrentChar := CharInString(Input, CurrentPos)
  ELSE
     Negative := FALSE
  END;
    WHILE CurrentChar IN ValidSet DO
```

```
      INC(CurrentPos);
        CurrentChar := CharInString(Input, CurrentPos)
  END;
    IF CurrentPos = ConstantStart THEN
      OperandValue := 0;
       OperandStatus := Absolute;
      RETURN FALSE
  ELSE
        InitialiseAString(ConstantString);
         MakeSubstring(Input, ConstantStart, CurrentPos - ConstantStart,
                                                                ConstantString);
      IF Negative THEN
            OperandValue := -ConvertBase(ConstantString, Base)
    ELSE
            OperandValue := ConvertBase(ConstantString, Base)
    END;
      OperandStatus := Absolute;
     RETURN TRUE
  END
 END GetConstant;


   PROCEDURE GetSymbol(VAR OperandValue: LONGINT;
                            VAR OperandStatus: SymbolStatus): BOOLEAN;

VAR
   SymbolStart          : CARDINAL;
  Key                   : String;
  Entry                 : TableType;
  Negative              : BOOLEAN;

BEGIN
    SymbolStart := CurrentPos;
      CurrentChar := CharInString(Input, CurrentPos);
   IF CurrentChar = '-' THEN
     Negative := TRUE;
      INC(CurrentPos);
      INC(SymbolStart);
        CurrentChar := CharInString(Input, CurrentPos)
  ELSE
     Negative := FALSE
  END;
   IF CurrentChar = '*' THEN
     IF Negative THEN
         OperandValue := -LONGINT(CurrentLocation());
    ELSE
         OperandValue := LONGINT(CurrentLocation())
    END;
      OperandStatus := Relative;
      INC(CurrentPos);
        CurrentChar := CharInString(Input, CurrentPos);
     RETURN TRUE
  ELSE
       WHILE NOT(CurrentChar IN OperatorSet + EOLSet) DO
        INC(CurrentPos);
          CurrentChar := CharInString(Input, CurrentPos)
    END;
       InitialiseAString(Key);
         MakeSubstring(Input, SymbolStart, CurrentPos - SymbolStart, Key);
     IF IsIn(Key) THEN
        Entry := Retrieve(Key);
        IF Entry.Kind = Symbol THEN
          IF Negative THEN
              OperandValue := -Entry.Value
```

```
            ELSE
                 OperandValue := Entry.Value
            END;
               OperandStatus := Entry.Status;
             RETURN TRUE
          ELSE
               Raise(Code, Error, WrongTypeError);
             OperandValue := 0;
              OperandStatus := Absolute;
             RETURN FALSE
          END
        ELSE
           IF Negative THEN
              CurrentPos := SymbolStart - 1
          ELSE
              CurrentPos := SymbolStart
          END;
           RETURN FALSE
        END
      END
    END GetSymbol;


BEGIN
   IF GetBase(Base) THEN
        IF NOT GetConstant(OperandValue, OperandStatus) THEN
          Raise(Code, Error, OperandError);
        RETURN FALSE
     END;
      ELSIF NOT(GetSymbol(OperandValue, OperandStatus)) THEN
       Base := DefaultBase;
        IF NOT GetConstant(OperandValue, OperandStatus) THEN
        RETURN FALSE
     END
   END;
   RETURN TRUE
 END GetOperand;


 PROCEDURE GetOperator(VAR Operator: OpType): BOOLEAN;

VAR
   OperatorStart          : CARDINAL;
   OperatorString         : String;

BEGIN
    OperatorStart := CurrentPos;
     CurrentChar := CharInString(Input, CurrentPos);
    INC(CurrentPos);
   CASE CurrentChar OF
     '+':        Operator := Add;
                 RETURN TRUE                |
     '-':        Operator := Sub;
                 RETURN TRUE                |
      '*':          IF CharInString(Input, CurrentPos) = '*' THEN
                     INC(CurrentPos);
                    Operator := Exp
                 ELSE
                    Operator := Mult
                 END;
                 RETURN TRUE                |
      '/':          IF CharInString(Input, CurrentPos) = '/' THEN
                     INC(CurrentPos);
                    Operator := DivUn
```

```
                ELSE
                    Operator := DivSig
                END;
                RETURN TRUE                |
      '.':          CurrentChar := CharInString(Input, CurrentPos);
                    WHILE NOT(CurrentChar IN OperatorSet + EOLSet) DO
                    INC(CurrentPos);
                       CurrentChar := CharInString(Input, CurrentPos)
                END;
                    InitialiseAString(OperatorString);
                    MakeSubstring(Input, OperatorStart,
                                  CurrentPos - OperatorStart + 1, OperatorString);
                   IF CurrentChar IN OperatorSet THEN
                    INC(CurrentPos);
                       IF EqualStrings(OperatorString, ".AND.") THEN
                      Operator := And;
                      RETURN TRUE
                       ELSIF EqualStrings(OperatorString, ".OR.") THEN
                      Operator := Or;
                      RETURN TRUE
                       ELSIF EqualStrings(OperatorString, ".XOR.") THEN
                      Operator := Xor;
                      RETURN TRUE
                       ELSIF EqualStrings(OperatorString, ".NOT.") THEN
                      Operator := Not;
                      RETURN TRUE
                       ELSIF EqualStrings(OperatorString, ".SHL.") THEN
                      Operator := Shl;
                      RETURN TRUE
                       ELSIF EqualStrings(OperatorString, ".SHR.") THEN
                      Operator := Shr;
                      RETURN TRUE
                       ELSIF EqualStrings(OperatorString, ".MOD.") THEN
                      Operator := Mod;
                      RETURN TRUE
                   ELSE
                        Raise(Code, Error, OperatorError);
                      RETURN FALSE
                   END
                ELSE
                    RETURN FALSE
                END                        |
  EndOfLine:   RETURN FALSE
 ELSE
      Raise(Code,Error, OperatorError);
    RETURN FALSE
 END
END GetOperator;


PROCEDURE Eval(Operand1: LONGINT; Operand1Status: SymbolStatus;
                Operator: OpType;
                   Operand2: LONGINT; Operand2Status: SymbolStatus;
                   VAR Result: LONGINT; VAR ResultStatus: SymbolStatus);



   PROCEDURE Valid(Op1Status, Op2Status: SymbolStatus; Operator: OpType;
                         VAR ResultType: SymbolStatus): BOOLEAN;

 BEGIN
     IF (Op1Status = Absolute) AND (Op2Status = Absolute) THEN
       ResultType := Absolute;
      RETURN TRUE
```

```
    ELSIF (Operator = Add) AND
                 (((Op1Status = Absolute) AND (Op2Status = Relative)) OR
                  ((Op1Status = Relative) AND (Op2Status = Absolute))) THEN
       ResultType := Relative;
      RETURN TRUE
    ELSIF (Operator = Sub) AND
                 ((Op1Status = Relative) AND (Op2Status = Absolute)) THEN
       ResultType := Relative;
      RETURN TRUE
    ELSIF (Operator = Sub) AND
                 ((Op1Status = Relative) AND (Op2Status = Relative)) THEN
       ResultType := Relative;
      RETURN TRUE
  ELSE
       Raise(Code, Error, StatusError);
      RETURN FALSE
  END
END Valid;


  PROCEDURE DoAdd(Op1, Op2: LONGINT): LONGINT;
BEGIN
    IF (MAX(LONGINT) - ABS(Op1)) < ABS(Op2) THEN
       Raise(Code, Error, AddError);
     RETURN 0
  ELSE
     RETURN Op1 + Op2
  END
END DoAdd;


  PROCEDURE DoSub(Op1, Op2: LONGINT): LONGINT;
BEGIN
    IF (MAX(LONGINT) - ABS(Op1)) < ABS(Op2) THEN
       Raise(Code, Error, SubError);
     RETURN 0
  ELSE
     RETURN Op1 - Op2
  END
END DoSub;


  PROCEDURE DoMult(Op1, Op2: LONGINT): LONGINT;
BEGIN
    IF ABS(MAX(LONGINT) DIV Op1) < ABS(Op2) THEN
       Raise(Code, Error, MultError);
     RETURN 0
  ELSE
     RETURN Op1 * Op2
  END
END DoMult;


  PROCEDURE DoDivSig(Op1, Op2: LONGINT): LONGINT;
BEGIN
   IF Op2 = 0 THEN
       Raise(Code, Error, DivError);
     RETURN 0
  ELSE
     RETURN Op1 DIV Op2
  END
END DoDivSig;
```

```
  PROCEDURE DoDivUn(Op1, Op2: LONGINT): LONGINT;
BEGIN
  IF Op2 = 0 THEN
      Raise(Code, Error, DivError);
    RETURN 0
  ELSE
      RETURN ABS(Op1) DIV ABS(Op2)
  END
END DoDivUn;


  PROCEDURE DoShr(Op1, Op2: LONGINT): LONGINT; FORWARD;


  PROCEDURE DoShl(Op1, Op2: LONGINT): LONGINT;
VAR
    Counter, Op1ABS: LONGINT;

BEGIN
   Op1ABS := ABS(Op1);
  IF Op2 >= 0 THEN
     FOR Counter := 0 TO Op2 - 1 DO
        Op1ABS := DoMult(Op1ABS, 2)
    END
  ELSE
      Op1ABS := DoShr(Op1ABS, ABS(Op2))
  END;
   RETURN Op1ABS
END DoShl;


  PROCEDURE DoShr(Op1, Op2: LONGINT): LONGINT;
VAR
    Counter, Op1ABS: LONGINT;

BEGIN
   Op1ABS := ABS(Op1);
  IF Op2 >= 0 THEN
     FOR Counter := 0 TO Op2 - 1 DO
       Op1ABS := Op1ABS DIV 2
    END
  ELSE
      Op1ABS := DoShl(Op1ABS, ABS(Op2))
  END;
   RETURN Op1ABS
END DoShr;


  PROCEDURE DoAnd(Op1, Op2: LONGINT): LONGINT;
TYPE
  Bits       = SET OF [0..31];

VAR
    Op1Bits, Op2Bits, Result   : Bits;

BEGIN
    Op1Bits := Bits(Op1);
    Op2Bits := Bits(Op2);
    Result := Op1Bits * Op2Bits;
    RETURN LONGINT(Result)
END DoAnd;


  PROCEDURE DoOr(Op1, Op2: LONGINT): LONGINT;
```

```
TYPE
   Bits        = SET OF [0..31];

VAR
    Op1Bits, Op2Bits, Result   : Bits;

BEGIN
    Op1Bits := Bits(Op1);
    Op2Bits := Bits(Op2);
    Result := Op1Bits + Op2Bits;
     RETURN LONGINT(Result)
END DoOr;


  PROCEDURE DoXor(Op1, Op2: LONGINT): LONGINT;
TYPE
   Bits        = SET OF [0..31];

VAR
    Op1Bits, Op2Bits, Result   : Bits;

BEGIN
    Op1Bits := Bits(Op1);
    Op2Bits := Bits(Op2);
    Result := Op1Bits / Op2Bits;
     RETURN LONGINT(Result)
END DoXor;


  PROCEDURE DoNot(Op1: LONGINT): LONGINT;
TYPE
   Bits        = SET OF [0..31];

VAR
   Op1Bits, Result    : Bits;

BEGIN
    Op1Bits := Bits(Op1);
     Result := Bits{0..31} - Op1Bits;
     RETURN LONGINT(Result)
END DoNot;


  PROCEDURE DoMod(Op1, Op2: LONGINT): LONGINT;
BEGIN
   IF Op2 = 0 THEN
       Raise(Code, Error, DivError);
     RETURN 0
  ELSE
     RETURN Op1 MOD Op2
  END
END DoMod;


  PROCEDURE DoExp(Op1, Op2: LONGINT): LONGINT;
VAR
    Sum, Counter: LONGINT;

BEGIN
   IF Op2 < 0 THEN
       Raise(Code, Error, ExpError);
     RETURN 0
  ELSE
     Sum := 1;
```

```
          FOR Counter := 0 TO Op2 - 1 DO
             Sum := DoMult(Sum, Op1)
         END
       END;
       RETURN Sum
     END DoExp;


   BEGIN
        IF Valid(Operand1Status, Operand2Status, Operator, ResultType) THEN
        CASE Operator OF
           Add:      Result := DoAdd(Operand1, Operand2)       |
           Sub:      Result := DoSub(Operand1, Operand2)       |
           Mult:     Result := DoMult(Operand1, Operand2)      |
            DivSig:  Result := DoDivSig(Operand1, Operand2)    |
           DivUn:    Result := DoDivUn(Operand1, Operand2)     |
           Shl:      Result := DoShl(Operand1, Operand2)       |
           Shr:      Result := DoShr(Operand1, Operand2)       |
           And:      Result := DoAnd(Operand1, Operand2)       |
           Or:       Result := DoOr(Operand1, Operand2)        |
           Xor:      Result := DoXor(Operand1, Operand2)       |
          Not:     Result := DoNot(Operand1)                   |
           Mod:      Result := DoMod(Operand1, Operand2)       |
            Exp:      Result := DoExp(Operand1, Operand2)
       END
     END
   END Eval;

BEGIN
   CurrentPos := 1;
    IF GetOperand(Op1, Op1Status) THEN
       IF GetOperator(Operator) THEN
         IF NOT(Operator = Not) THEN
             IF GetOperand(Op2, Op2Status) THEN
                 CurrentChar := CharInString(Input, CurrentPos);
               IF CurrentChar <> EndOfLine THEN
                  Raise(Code, Error, OperandError)
             END
         ELSE
              Raise(Code, Error, OperandError)
         END
       END;
          Eval(Op1, Op1Status, Operator, Op2, Op2Status, Result, ResultType)
     ELSE
        Result := Op1;
         ResultType := Op1Status
     END
   ELSE
       Raise(Code, Error, OperandError);
      Result := 0;
       ResultType := Absolute
   END
END Evaluate;


PROCEDURE CheckRegister(Operand: String; VAR Register: String): BOOLEAN;

VAR
    CurrentPos, RegisterCount: CARDINAL;
    CurrentChar: CHAR;
    TempReg: ARRAY [1..133] OF CHAR;

BEGIN
   CurrentPos := 1;
```

```
    CurrentChar  :=  CharInString(Operand,  CurrentPos);
    IF  CurrentChar  IN  RegisterSet  THEN
       INC(CurrentPos);
        CurrentChar  :=  CharInString(Operand,  CurrentPos);
       WHILE  NOT(CurrentChar  IN  EOLSet)  DO
           TempReg[CurrentPos-1]  :=  CAP(CurrentChar);
         INC(CurrentPos);
           CurrentChar  :=  CharInString(Operand,  CurrentPos)
     END;
        TempReg[CurrentPos-1]  :=  EndOfLine;
        ArrayToString(TempReg,  Register);
       FOR  RegisterCount  :=  1  TO  NOOFREGISTERS  DO
            IF  EqualStrings(Register,  ValidRegisters[RegisterCount])  THEN
          RETURN  TRUE
       END
     END;
        Raise(Code,  Error,  RegisterError);
      RETURN  FALSE
   ELSE
        InitialiseAString(Register);
      RETURN  FALSE
   END
END  CheckRegister;


PROCEDURE  CurrentBase():  BaseType;

BEGIN
    RETURN  DefaultBase
END  CurrentBase;


PROCEDURE  ChangeBase(NewBase:  BaseType);

BEGIN
    DefaultBase  :=  NewBase
END  ChangeBase;


PROCEDURE  ExpressionShutdown;
END  ExpressionShutdown;


BEGIN
    DefaultBase  :=  Dec
END  Expression.
```

```
IMPLEMENTATION  MODULE  Interface;

(*
          MODULE: Interface         Version 1.0       (c) 1990 Mark Wickens

          Created: 01-01-90

            This module provides the interface for the meta-assembler to the
          outside world.
*)

FROM  Storage       IMPORT   ALLOCATE,  DEALLOCATE,  Available;
FROM  Lib           IMPORT   ParamCount,  ParamStr;
FROM  Strings       IMPORT   EqualStrings,  StringToArray,  ArrayToString,
                             LongestString, InitialiseAString, EndOfLine,
                             MakeSubstring, ConcatStrings;
FROM  Exceptions  IMPORT   Raise,  ExceptionLevel,  ExceptionType;
FROM  Lib         IMPORT   Dos;
FROM  SYSTEM      IMPORT   Registers;
IMPORT  FIO;
IMPORT  IO;

CONST
   (*  Dubugging  Switch  *)
  IODebug                  = TRUE;

  BufferSize               = 2;
   FileMissingError         = "File  to  be  opened  cannot  be  found";
   FileCreateError          = "File  could  not  be  created";
   FileOpenError            = "File  could  not  be  opened";
   FileCloseError           = "File  could  not  be  closed";
   OutOfMemoryError         = "File  buffer  could  not  be  allocated";
   NotOpenError             = "File  to  be  accessed  is  not  open";
   ReadError                = "Attempt  to  read  failed";
   WriteError               = "Attempt  to  write  failed";

TYPE
   BufferPtr       =  POINTER  TO  Buffer;
   Buffer            =  ARRAY  [0..(BufferSize  *  512  +  FIO.BufferOverhead)]  OF  BYTE;
    BufferList       =  ARRAY  [5..FIO.MaxOpenFiles]  OF  BufferPtr;

VAR
   UsedBuffers     :  BufferList;
    CurrentBuffer  :  CARDINAL;


PROCEDURE  FileExists(FileName:  String):  BOOLEAN;
BEGIN
    RETURN  FIO.Exists(FileName)
END  FileExists;


PROCEDURE  CreateAFile(F:  String):  FileHandle;

CONST
  FEE1            = "File ";
  FEE2             = " has  been  overwritten.";

VAR
  FHandle         :  FileHandle;
   FileBuffer      :  BufferPtr;
    FileExistsError:  String;

BEGIN
```

```
    IF  FIO.Exists(F)  THEN
        InitialiseAString(FileExistsError);
        MakeSubstring(FEE1,  1,  10,  FileExistsError);
        ConcatStrings(FileExistsError,  F);
        ConcatStrings(FileExistsError,  FEE2);
        Raise(Environment,  Warning,  FileExistsError);
   END;
    FHandle  :=  FIO.Create(F);
    IF  (FHandle  =  MAX(CARDINAL))  OR  ((NOT  IODebug  AND  (FIO.IOresult()  <>  0)))  THEN
        Raise(Environment,  Error,  FileCreateError);
      RETURN  NULLFile
    ELSIF  Available(SIZE(Buffer))  AND  NOT  ((FHandle  =  StdIn)  OR
                                                 (FHandle = StdOut) OR
                                                 (FHandle = ErrOut) OR
                                                 (FHandle = AuxDev) OR
                                                  (FHandle = PrnDev)) THEN
        ALLOCATE(FileBuffer,  SIZE(Buffer));
        FIO.AssignBuffer(FHandle,  FileBuffer^);
       INC(CurrentBuffer);
        UsedBuffers[CurrentBuffer]  :=  FileBuffer;
      RETURN  FHandle
   ELSE
        Raise(Environment,  Fatal,  OutOfMemoryError);
      RETURN  NULLFile
   END
END  CreateAFile;


PROCEDURE  OpenAFile(F:  String):  FileHandle;

VAR
   FHandle          :  FileHandle;
   FileBuffer       :  BufferPtr;

BEGIN
    IF  NOT(FIO.Exists(F))  THEN
        Raise(Environment,  Error,  FileMissingError);
      RETURN  NULLFile
   ELSE
       FHandle  :=  FIO.Open(F);
       IF  (FHandle  =  MAX(CARDINAL))  OR
                                          ((NOT  IODebug AND  (FIO.IOresult() <> 0))) THEN
           Raise(Environment,  Error,  FileOpenError);
         RETURN  NULLFile
        ELSIF  Available(SIZE(Buffer))  THEN
           ALLOCATE(FileBuffer,  SIZE(Buffer));
            FIO.AssignBuffer(FHandle,  FileBuffer^);
          INC(CurrentBuffer);
           UsedBuffers[CurrentBuffer]  :=  FileBuffer;
         RETURN  FHandle
     ELSE
            Raise(Environment,  Fatal,  OutOfMemoryError);
         RETURN  NULLFile
     END
   END
END  OpenAFile;


PROCEDURE  CloseAFile(F:  FileHandle);

BEGIN
   IF  (F  >=  5)  AND  (F  <=  FIO.MaxOpenFiles)  THEN
      FIO.Close(F);
      IF  (NOT  IODebug)  AND  (FIO.IOresult()  <>  0)  THEN
```

```
          Raise(Environment, Error, FileCloseError)
     END
   END
END CloseAFile;


PROCEDURE DeleteAFile(Filename: String);

BEGIN
    IF FIO.Exists(Filename) THEN
       FIO.Erase(Filename)
   END
END DeleteAFile;


PROCEDURE EndOfFile(): BOOLEAN;

BEGIN
   RETURN FIO.EOF
END EndOfFile;


PROCEDURE ReadAString(F: FileHandle; VAR S: String);

BEGIN
   IF F <> NULLFile THEN
      IF F = FIO.StandardInput THEN
        IO.RdStr(S)
     ELSE
        FIO.RdStr(F, S)
     END;
      IF NOT FIO.OK THEN
          Raise(Environment, Fatal, ReadError)
     END
   ELSE
       Raise(Internal, Error, NotOpenError)
   END
END ReadAString;


PROCEDURE WriteAString(F: FileHandle; S: String);

BEGIN
   IF F <> NULLFile THEN
      IF F = FIO.StandardOutput THEN
        IO.WrStr(S)
     ELSE
        FIO.WrStr(F, S)
     END;
      IF NOT FIO.OK THEN
          Raise(Environment, Fatal, WriteError);
         InitialiseAString(S)
     END
   ELSE
       Raise(Internal, Error, NotOpenError)
   END
END WriteAString;


PROCEDURE ReadAChar(F: FileHandle): CHAR;

VAR
   C: CHAR;
```

```
BEGIN
   IF F <> NULLFile THEN
      IF F = FIO.StandardInput THEN
        C := IO.RdChar()
     ELSE
        C := FIO.RdChar(F)
     END;
      IF NOT FIO.OK THEN
           Raise(Environment, Fatal, ReadError);
        C := EndOfLine
     END
   ELSE
        Raise(Internal, Error, NotOpenError);
      C := EndOfLine
   END;
   RETURN C
END ReadAChar;


PROCEDURE WriteAChar(F: FileHandle; C: CHAR);

BEGIN
   IF F <> NULLFile THEN
      IF F = FIO.StandardOutput THEN
        IO.WrChar(C)
     ELSE
        FIO.WrChar(F, C)
     END;
      IF NOT FIO.OK THEN
           Raise(Environment, Fatal, WriteError);
        C := EndOfLine
     END
   ELSE
        Raise(Internal, Error, NotOpenError)
   END
END WriteAChar;


PROCEDURE NextPrinting(F: FileHandle): CHAR;

VAR
   C: CHAR;

BEGIN
    C := ReadAChar(F);
   WHILE (C = CHR(10)) OR (C = CHR(13)) DO
      C := ReadAChar(F)
   END;
   RETURN C
END NextPrinting;


PROCEDURE WriteALine(F: FileHandle);

BEGIN
   IF F <> NULLFile THEN
      IF F = FIO.StandardOutput THEN
        IO.WrLn
     ELSE
        FIO.WrLn(F)
     END;
      IF NOT FIO.OK THEN
           Raise(Environment, Fatal, WriteError);
     END
```

```
   ELSE
       Raise(Internal, Error, NotOpenError)
   END
END WriteALine;


PROCEDURE ReadACard(F: FileHandle): CARDINAL;

VAR
   C: CARDINAL;

BEGIN
   IF F <> NULLFile THEN
      IF F = FIO.StandardInput THEN
        C := IO.RdCard()
     ELSE
        C := FIO.RdCard(F)
     END;
      IF NOT FIO.OK THEN
           Raise(Environment, Fatal, ReadError);
       C := 0
     END
   ELSE
       Raise(Internal, Error, NotOpenError);
     C := 0
   END;
   RETURN C
END ReadACard;


PROCEDURE WriteACard(F: FileHandle; C: CARDINAL; Length: CARDINAL);

BEGIN
   IF F <> NULLFile THEN
      IF F = FIO.StandardOutput THEN
         IO.WrCard(C, Length)
     ELSE
         FIO.WrCard(F, C, Length)
     END;
      IF NOT FIO.OK THEN
           Raise(Environment, Fatal, WriteError);
     END
   ELSE
       Raise(Internal, Error, NotOpenError)
   END
END WriteACard;


PROCEDURE ReadALongInt(F: FileHandle): LONGINT;

VAR
   L: LONGINT;

BEGIN
   IF F <> NULLFile THEN
      IF F = FIO.StandardInput THEN
        L := IO.RdLngInt()
     ELSE
        L := FIO.RdLngInt(F)
     END;
      IF NOT FIO.OK THEN
           Raise(Environment, Fatal, ReadError);
       L := 0
     END
```

```
   ELSE
       Raise(Internal,  Error,  NotOpenError);
     L := 0
   END;
   RETURN L
END  ReadALongInt;


PROCEDURE  WriteALongInt(F:  FileHandle;  L:  LONGINT;  Length:  CARDINAL);

BEGIN
   IF F <> NULLFile THEN
      IF F = FIO.StandardOutput THEN
         IO.WrLngInt(L, Length)
     ELSE
         FIO.WrLngInt(F, L, Length)
     END;
      IF NOT FIO.OK THEN
           Raise(Environment,  Fatal,  WriteError);
     END
   ELSE
       Raise(Internal,  Error,  NotOpenError)
   END
END  WriteALongInt;


PROCEDURE  ReadALongHex(F:  FileHandle):  LONGCARD;

VAR
   L:  LONGINT;

BEGIN
   IF F <> NULLFile THEN
      IF F = FIO.StandardInput THEN
        L := IO.RdLngHex()
     ELSE
        L := FIO.RdLngHex(F)
     END;
      IF NOT FIO.OK THEN
           Raise(Environment,  Fatal,  ReadError);
       L := 0
     END
   ELSE
       Raise(Internal,  Error,  NotOpenError);
     L := 0
   END;
   RETURN L
END  ReadALongHex;


PROCEDURE  WriteALongHex(F:  FileHandle;  L:  LONGCARD;  Length:  CARDINAL);

BEGIN
   IF F <> NULLFile THEN
      IF F = FIO.StandardOutput THEN
         IO.WrLngHex(L, Length)
     ELSE
         FIO.WrLngHex(F, L, Length)
     END;
      IF NOT FIO.OK THEN
           Raise(Environment,  Fatal,  WriteError);
     END
   ELSE
       Raise(Internal,  Error,  NotOpenError)
```

```
   END
END  WriteALongHex;


PROCEDURE  ReadArgument(Number:  CARDINAL;  VAR  Parameter:  String);

BEGIN
    ParamStr(Parameter,  Number);
END  ReadArgument;


PROCEDURE  NumberOfArguments():  CARDINAL;

BEGIN
    RETURN  ParamCount()
END  NumberOfArguments;


PROCEDURE  GetTime(VAR  Hours,  Minutes,  Seconds:  CARDINAL);

VAR
   r:  Registers;

BEGIN
   r.AH  :=  2CH;
   Dos(r);
    Hours  :=  CARDINAL(r.CH);
    Minutes  :=  CARDINAL(r.CL);
    Seconds  :=  CARDINAL(r.DH)
END  GetTime;


PROCEDURE  GetDate(VAR  Year,  Month,  Day:  CARDINAL);

VAR
   r:  Registers;

BEGIN
   r.AH  :=  2AH;
   Dos(r);
   Year  :=  r.CX;
    Month  :=  CARDINAL(r.DH);
    Day  :=  CARDINAL(r.DL)
END  GetDate;


PROCEDURE  InterfaceShutdown;

VAR
   Index  :  CARDINAL;

BEGIN
   FOR  Index  :=  5  TO  CurrentBuffer  DO
        DEALLOCATE(UsedBuffers[Index],  SIZE(Buffer))
   END;
    CurrentBuffer  :=  4;
END  InterfaceShutdown;


BEGIN
   IF  IODebug  THEN
      FIO.IOcheck  :=  TRUE
   ELSE
      FIO.IOcheck  :=  FALSE
```

```
   END;
    CurrentBuffer := 4;
END Interface.
```

```
IMPLEMENTATION  MODULE  Lex;

(*
          MODULE: Lex                Version 1.0      (c) 1989 Mark Wickens

           Created: 03-12-89
           Updated: 17-12-89         Works satisfactorily.
           Updated: 30-12-89         Modified so that arguments of operands can
                                       be expressions containing postfix characters
                                   e.g., + and -.

            This module converts the free format source code into a fixed format
            assembler instruction, the layout of which conforms to the IEEE
            proposed assembler language format.
*)

FROM  Strings                   IMPORT   CharInString, InitialiseAString, EmptyString,
                                         LengthOfString, MakeSubstring, EndOfLine,
                                       CapitaliseString;
FROM  Exceptions               IMPORT   ExceptionLevel, ExceptionType, Raise;

TYPE
   CharSet         = SET OF CHAR;

CONST
   (* Characters *)
   SP     = ' ';
   HT     = CHR(9);

   (* Character Sets *)
    AlphabetSet      =  CharSet{'A'..'Z',  'a'..'z'};
    AlphaNumSet      =  CharSet{'A'..'Z',  'a'..'z',  '0'..'9',  "'"};
    WhiteSpaceSet  =  CharSet{SP,  HT};
   CommentSet       =  CharSet{';'};
   PreIndexSet      =  CharSet{'('};
    PostIndexSet    =  CharSet{')'};
   OperandSet       =  CharSet{','};
   LabelSet         =  CharSet{':'};
    SeperatorSet    =  CharSet{'.'};
   ModifierSet      =  CharSet{'B',  'H',  'L',  'D',  'F',  '1',  '4',  'M',  'W'};
   EOLSet            =  CharSet{EndOfLine};
   PrefixSet        =  CharSet{'/',  '!',  '@',  '$',  '#',  '+',  '-'};
   PostfixSet       =  CharSet{'+',  '-'};

   (* Exception Messages *)
   LabelError       =  "Label starts with non-alphabetic character";
    ModifierError  =  "Illegal Modifier";
    CommandError    =  "Command contains non-alphabetic characters";
    ArgumentError  =  "Badly formed argument";
   CommentError    =  "Badly formed comment";
   OperandError    =  "Badly formed operand";
   IndexError       =  "Badly formed index";
   EOLError         =  "Unexpected end of line";
    TooManyOperandsError    =  "Number of operands exceeds maximum permitted";
    TooManyIndicesError     =  "Number of indices exceeds maximum permitted";
    AddrModeError  =  "Invalid addressing mode";

PROCEDURE  InitialiseDecoded(VAR  AI:  DecodedLine);

(*
            This procedure creates a completely blank structure which will be
            filled with the decoded line.  The structure is defined at compile
            time by constants defined in the definition module ADM.
*)
```

```
VAR
    OperandIndex,  IndicesIndex:  CARDINAL;

BEGIN
   WITH  AI  DO
        InitialiseAString(Label);
        InitialiseAString(Command);
      Modifier  :=  'W';
      NoOfOperands  :=  0;
       FOR  OperandIndex  :=  1  TO  MAXOPERANDS  DO
          Operand[OperandIndex].Prefix  :=  '  ';
            InitialiseAString(Operand[OperandIndex].Argument);
           Operand[OperandIndex].NoOfIndices  :=  0;
         FOR  IndicesIndex  :=  1  TO  MAXINDICES  DO
               InitialiseAString(Operand[OperandIndex].Indices[IndicesIndex])
       END;
          Operand[OperandIndex].Postfix  :=  '  '
     END
   END
END  InitialiseDecoded;


PROCEDURE  ScanLine(Line:  String):  DecodedLine;

(*
            This  is  the  only  procedure  exported  and  it  converts  Line  into  a
            structure  containing  all  the  elements  of  the  input  line.
*)

VAR
   Decoded                    :  DecodedLine;
   CurrentPos                 :  CARDINAL;
   CurrentChar                :  CHAR;


    PROCEDURE  SkipBlanks(Input:  String;  VAR  Skipped:  CARDINAL);

   (*
            Skips  over  any  blanks  in  the  string  Line,  starting  at  skipped.
            On  exit  skipped  will  contain  the  position  of  the  first  non  space  or
             tab  character  (or  LengthOfString(Input)  if  an  EndOfLine  character  is
           encountered.
   *)

   BEGIN
        CurrentChar  :=  CharInString(Input,  Skipped);
        WHILE  (CurrentChar  IN  WhiteSpaceSet)  AND  NOT(CurrentChar  IN  EOLSet)  DO
         INC(Skipped);
           CurrentChar  :=  CharInString(Input,  Skipped)
     END
    END  SkipBlanks;


    PROCEDURE  ExtractLabel;

   (*
             ExtractLabel  operates  on  the  global  (to  procedure  ScanLine)  variables
            CurrentPos  and  CurrentChar.   It  enters  the  Label  field  of  Decoded
           with  any  label  found.
   *)

   BEGIN
        CurrentChar  :=  CharInString(Line,  CurrentPos);
```

```
    IF CurrentChar IN AlphabetSet THEN
     REPEAT
         INC(CurrentPos);
           CurrentChar := CharInString(Line, CurrentPos)
       UNTIL CurrentChar IN WhiteSpaceSet + LabelSet + EOLSet;
        MakeSubstring(Line, 1, CurrentPos - 1, Decoded.Label);
      INC(CurrentPos)
    ELSIF NOT(CurrentChar IN WhiteSpaceSet + CommentSet + EOLSet) THEN
       Raise(Code, Error, LabelError)
  END
 END ExtractLabel;


 PROCEDURE ExtractOperands;

(*
          Upto MAXOPERANDS operands are extracted from the current input line.
*)

VAR
    MoreOperands: BOOLEAN;
    OperandCount, StartArgPos     : CARDINAL;


    PROCEDURE ReadIndices;

  (*
          Upto MAXINDICES indices are extracted from the current input line.
  *)

  VAR
       IndexCount, StartIndexPos : CARDINAL;
     MoreIndices                 : BOOLEAN;

  BEGIN
       CurrentChar := CharInString(Line, CurrentPos);
      IF CurrentChar IN PreIndexSet THEN
       IndexCount := 1;
        MoreIndices := TRUE;
        INC(CurrentPos);
        WHILE MoreIndices DO
           StartIndexPos := CurrentPos;
            CurrentChar := CharInString(Line, CurrentPos);
            WHILE NOT(CurrentChar IN WhiteSpaceSet + OperandSet + PostIndexSet +
                                                                  EOLSet) DO
            INC(CurrentPos);
              CurrentChar := CharInString(Line, CurrentPos)
         END;
            IF CurrentChar IN OperandSet + PostIndexSet THEN
             IF IndexCount <= MAXINDICES THEN
                  MakeSubstring(Line, StartIndexPos, CurrentPos - StartIndexPos,
                                  Decoded.Operand[OperandCount].Indices[IndexCount]);
               IF CurrentChar IN OperandSet THEN
                INC(IndexCount)
            ELSE
                MoreIndices := FALSE
            END;
              INC(CurrentPos)
           ELSE
                Raise(Code, Error, TooManyIndicesError);
              MoreIndices := FALSE
           END
         ELSE
              Raise(Code, Error, IndexError);
```

```
            MoreIndices := FALSE
        END
      END;
          Decoded.Operand[OperandCount].NoOfIndices := IndexCount
    END
   END ReadIndices;


BEGIN
     SkipBlanks(Line, CurrentPos);
     CurrentChar := CharInString(Line, CurrentPos);
    IF NOT(CurrentChar IN CommentSet + EOLSet) THEN
      OperandCount := 1;
      MoreOperands := TRUE;
      WHILE MoreOperands DO
          CurrentChar := CharInString(Line, CurrentPos);
        IF NOT(CurrentChar IN EOLSet) THEN
          IF CurrentChar IN PrefixSet THEN
               Decoded.Operand[OperandCount].Prefix := CurrentChar;
            INC(CurrentPos);
              CurrentChar := CharInString(Line, CurrentPos)
        END;
          StartArgPos := CurrentPos;
           WHILE NOT(CurrentChar IN WhiteSpaceSet + PreIndexSet +
                                                        OperandSet + EOLSet) DO
            INC(CurrentPos);
              CurrentChar := CharInString(Line, CurrentPos)
        END;
          IF OperandCount <= MAXOPERANDS THEN
              IF CharInString(Line, CurrentPos - 1) IN PostfixSet THEN
              DEC(CurrentPos)
          END;
               MakeSubstring(Line, StartArgPos, CurrentPos - StartArgPos,
                                   Decoded.Operand[OperandCount].Argument)
        ELSE
              Raise(Code, Error, TooManyOperandsError);
            MoreOperands := FALSE
        END;

         ReadIndices;

            CurrentChar := CharInString(Line, CurrentPos);
          IF CurrentChar IN PostfixSet THEN
               Decoded.Operand[OperandCount].Postfix := CurrentChar;
            INC(CurrentPos);
              CurrentChar := CharInString(Line, CurrentPos)
        END;
           IF NOT(CurrentChar IN OperandSet) THEN
            MoreOperands := FALSE;
              IF NOT(CurrentChar IN WhiteSpaceSet + EOLSet) THEN
               Raise(Code, Error, OperandError)
          END
        ELSE
            INC(CurrentPos);
            INC(OperandCount)
        END
      ELSE
          MoreOperands := FALSE
      END
    END;
       Decoded.NoOfOperands := OperandCount
 END
END ExtractOperands;
```

```
 PROCEDURE  ExtractCommand;

(*
          From the current position blanks are skipped.  A string of alphabetic
          characters is then read until either whitespace, a modifier character
          (in which case a modifier is read) or the end of line is encountered.
          If this is not the case, an error is produced.  The alphabetic string
          read is stored in the command field of decoded.  If a command was
          successfully read, ExtractOperands is called.
*)

VAR
   StartPos      : CARDINAL;

BEGIN
     SkipBlanks(Line,  CurrentPos);
     CurrentChar  :=  CharInString(Line,  CurrentPos);
    IF  NOT(CurrentChar  IN  EOLSet  +  CommentSet)  THEN
      StartPos  :=  CurrentPos;
      WHILE  CurrentChar  IN  AlphabetSet  DO
        INC(CurrentPos);
          CurrentChar  :=  CharInString(Line,  CurrentPos);
    END;
      IF  CurrentChar  IN  SeperatorSet  THEN
        INC(CurrentPos);
          CurrentChar  :=  CAP(CharInString(Line,  CurrentPos));
         IF  CurrentChar  IN  ModifierSet  THEN
            Decoded.Modifier  :=  CurrentChar
      ELSE
           Raise(Code,  Warning,  ModifierError)
      END;
         MakeSubstring(Line,  StartPos,  CurrentPos  -  StartPos  -  1,
                                                                Decoded.Command);
        INC(CurrentPos)
      ELSIF  CurrentChar  IN  EOLSet  +  WhiteSpaceSet  THEN
         MakeSubstring(Line,  StartPos,  CurrentPos  -  StartPos,
                                                                Decoded.Command);
        INC(CurrentPos)
    ELSE
         Raise(Code,  Error,  CommandError)
    END;
         CapitaliseString(Decoded.Command);
      ExtractOperands
  END
 END  ExtractCommand;


 PROCEDURE  ExtractComment;

(*
          Extract comment skips any blanks from the current position.  If the
          next character is a comment field of Decoded.  Otherwise, an error
       is produced.
*)

BEGIN
     SkipBlanks(Line,  CurrentPos);
     CurrentChar  :=  CharInString(Line,  CurrentPos);
    IF  NOT(CurrentChar  IN  EOLSet)  THEN
      IF  CurrentChar  IN  CommentSet  THEN
        INC(CurrentPos);
          MakeSubstring(Line,  CurrentPos,  LengthOfString(Line)  -  CurrentPos  +  1,
                                                                Decoded.Comment)
```

```
       ELSE
            Raise(Code, Error, CommentError)
       END
     END
    END  ExtractComment;

BEGIN
    InitialiseDecoded(Decoded);
   CurrentPos := 1;

    ExtractLabel;
    ExtractCommand;
    ExtractComment;

   RETURN Decoded
END  ScanLine;


PROCEDURE  ScanOperand(    OperandToScan:  OperandElement;
                               VAR AddressingMode: ValidAddressingModes);
(*
           Possible combinations:

          PREFIX   POSTFIX            AddressingMode
          ------   -------            --------------

                                     Direct
         /                           Absolute
         !                           Base Page
         @                           Indirect
         $                           Relative
         #                           Immediate
         .                           Register
         +                             Auto-pre-increment
                   +                   Auto-post-increment
         -                             Auto-pre-decrement
                   -                   Auto-post-decrement
                   @                   Indirect-pre-indexed
         @                             Indirect-post-indexed

            With indices appearing in brackets where appropriate.
*)

BEGIN
   WITH OperandToScan DO
     CASE Prefix OF
       '/': AddressingMode := Abs                    |
       '!': AddressingMode := BPage                  |
        '@': IF NoOfIndices = 0 THEN
                AddressingMode := Ind
            ELSE
                 AddressingMode := IndPostIndex
            END                                      |
        '$': AddressingMode := Rel                   |
        '#': AddressingMode := Imm                   |
         '+': AddressingMode := AutoPreInc           |
          '-': AddressingMode := AutoPreDec
     ELSE
        CASE Postfix OF
          '@': IF NoOfIndices > 0 THEN
                   AddressingMode := IndPreIndex
              ELSE
                    Raise(Code, Error, AddrModeError)
              END                                    |
```

```
          '+': AddressingMode := AutoPostInc        |
          '-': AddressingMode := AutoPostDec        |
          ' ': AddressingMode := Dir
       ELSE
            Raise(Code, Error, AddrModeError)
       END
     END;
      IF (Prefix <> ' ') AND (Postfix <> ' ') THEN
          Raise(Code, Error, AddrModeError)
     END
   END
END ScanOperand;


PROCEDURE LexShutdown;
END LexShutdown;


END Lex.
```

```
IMPLEMENTATION  MODULE  Listing;

(*
          MODULE: Listing           Version 1.0      (c) 1989 Mark Wickens

          Created: 28-11-89

          Creates the listing file.
*)

FROM  Interface    IMPORT    FileHandle,  CreateAFile,  WriteAString,  WriteACard,
                                WriteALine, LongestFileName, WriteAChar, CloseAFile,
                               WriteALongHex, WriteALongInt, GetDate, GetTime;
FROM  Strings        IMPORT    LengthOfString,  MakeSubstring,  InitialiseAString,
                            ConcatStrings;
FROM  Exceptions  IMPORT    Raise,  ExceptionLevel,  ExceptionType;
FROM  Table          IMPORT   NextInOrder,  Greater;
FROM  TableExt      IMPORT    TableType,  SymbolStatus,  EntryType;
FROM  ADM             IMPORT   WORDLENGTH,  MAXWORDSPERINSTRUCTION;

CONST
  FF                        = CHR(12);
  EXTRAPAGE                = 3;
  EXTRALINE                = 43;
  SYMBOLCONST              = 15;
  BLANK                    = "  ";
  ADDRESSBLANK            = "          ";
  MCBLANK                 = "                        ";
  MAXMCLISTLENGTH          = 20;
  SystemMessage             = "MetaASSEM   (c) 1990 M S Wickens   Version 1.0";

  PageLengthError           = "Illegal page length";
  LineLengthError           = "Illegal line length";
  HexConversionError       = "Illegal Number to Hex Converter";
  MCLengthError              = "Length of machine code greater than max permitted";

VAR
    PageNo,  LineNo,  SourceNo,  PageLength,  LineLength,  TitleLength  :  CARDINAL;
    AddressSwitch,  MCSwitch,  ListingSwitch,  SymbolTableSwitch         :  BOOLEAN;
  ListingFile                                                            : FileHandle;
    StartHours,  StartMinutes,  StartSeconds,  StartYear,  StartMonth,
    StartDay,  EndHours,  EndMinutes,  EndSeconds,  EndYear,  EndMonth,
  EndDay                                                                 : CARDINAL;
  StartTime, EndTime, AssemblyTime                                       : LONGCARD;
  TimeString, TitleString, ErrorString                                   : String;
  ErrorPending                                                           : BOOLEAN;
  AssemblyRate                                                           : LONGINT;

(*  Internal.  *)
PROCEDURE  WriteHeader;

VAR
    Count:  CARDINAL;

BEGIN
    WriteAString(ListingFile,  "Page:  ");
    WriteACard(ListingFile,  PageNo,  4);
    WriteAString(ListingFile,  "       ");
    WriteAString(ListingFile,  SystemMessage);
    WriteAString(ListingFile,  BLANK);

    WriteACard(ListingFile,  StartDay,  2);
    WriteAString(ListingFile,  "/");
    WriteACard(ListingFile,  StartMonth,  2);
```

```
    WriteAString(ListingFile,  "/");
    WriteACard(ListingFile,  StartYear,  4);
    WriteAString(ListingFile,  BLANK);

    WriteACard(ListingFile,  StartHours,  2);
    WriteAString(ListingFile,  ":");
    WriteACard(ListingFile,  StartMinutes,  2);
    WriteAString(ListingFile,  ":");
    WriteACard(ListingFile,  StartSeconds,  2);
    WriteALine(ListingFile);

    WriteAString(ListingFile,  TitleString);
    WriteALine(ListingFile);
    WriteALine(ListingFile)
END  WriteHeader;


PROCEDURE  DumpSymbolTable;

VAR
   Entry            : TableType;
   CurrentKey       : String;
   EntryLength      : CARDINAL;

BEGIN
    WriteALine(ListingFile);
    WriteALine(ListingFile);
    WriteAString(ListingFile,  "Symbol  Table  Dump:  ");
    WriteALine(ListingFile);
    WriteALine(ListingFile);
    InitialiseAString(CurrentKey);

    WHILE  Greater(CurrentKey)  DO
        NextInOrder(CurrentKey,  Entry);
        EntryLength  :=  LengthOfString(Entry.Key);
        MakeSubstring(Entry.Key,  1,  EntryLength,  CurrentKey);

      IF  Entry.Kind  =  Symbol  THEN
           WriteALongInt(ListingFile,  Entry.Value,  8);
           WriteAString(ListingFile,  BLANK);

         CASE  Entry.Status  OF
             Absolute:  WriteAString(ListingFile,  "Abs")  |
             Relative:  WriteAString(ListingFile,  "Rel")  |
       END;
           WriteAString(ListingFile,  BLANK);

          IF  EntryLength  >  (LineLength  -  SYMBOLCONST)  THEN
              MakeSubstring(Entry.Key,  1,  LineLength  -  SYMBOLCONST,  CurrentKey);
             WriteAString(ListingFile,  CurrentKey);
       ELSE
             WriteAString(ListingFile,  CurrentKey);
            WriteAString(ListingFile,  BLANK);
       END;
          WriteALine(ListingFile);
     END
   END
END  DumpSymbolTable;


PROCEDURE  DumpStatistics;

VAR
   EndHours,  EndMinutes,  EndSeconds          :  CARDINAL;
```

```
BEGIN
    GetTime(EndHours,  EndMinutes,  EndSeconds);
   EndTime  :=       ((LONGCARD(EndHours)  *  3600)  +
                      (LONGCARD(EndMinutes)  *  60)  +
                      (LONGCARD(EndSeconds)));
   IF  EndHours  <  StartHours  THEN
      EndTime  :=  EndTime  +  3600
   END;
    AssemblyTime  :=  EndTime  -  StartTime;
    AssemblyRate  :=  AssemblyTime;
    WriteALine(ListingFile);
    WriteAString(ListingFile,  "Assembly  Statistics:  ");
    WriteALine(ListingFile);
    WriteAString(ListingFile,  "Assembly  Time:  ");

    EndHours  :=  CARDINAL(AssemblyTime  DIV  3600);
    AssemblyTime  :=  AssemblyTime  MOD  3600;
    WriteACard(ListingFile,  EndHours,  2);
    WriteAChar(ListingFile,  ":");

    EndMinutes  :=  CARDINAL(AssemblyTime  DIV  60);
    AssemblyTime  :=  AssemblyTime  MOD  60;
    WriteACard(ListingFile,  EndMinutes,  2);
    WriteAChar(ListingFile,  ":");

    EndSeconds  :=  CARDINAL(AssemblyTime);
    WriteACard(ListingFile,  EndSeconds,  2);
    WriteALine(ListingFile);
    WriteAString(ListingFile,  "Assembly  Rate:  ");
    AssemblyRate  :=  (LONGINT(SourceNo)  *  60)  DIV  AssemblyRate;
    WriteALongInt(ListingFile,  AssemblyRate,  8);
    WriteAString(ListingFile,  "  lines  per  minute");
    WriteALine(ListingFile)
END  DumpStatistics;


PROCEDURE  WriteMC(MCode:  Instruction;  MCLength:  CARDINAL);


    PROCEDURE  Pow2(Number:  CARDINAL):  CARDINAL;

   VAR  Sum,  Counter:  CARDINAL;

   BEGIN
     Sum  :=  1;
     IF  Number  >  0  THEN
        FOR  Counter  :=  1  TO  Number  DO
          Sum  :=  Sum  *  2
       END;
     END;
     RETURN  Sum
   END  Pow2;


    PROCEDURE  ConvertToHex(input:  CARDINAL):  CHAR;
   BEGIN
     CASE  input  OF
        0..9:       RETURN  (CHR(ORD('0')  +  input))  |
        10..15:     RETURN  (CHR(ORD('A')  +  input  -  10))
     ELSE
          Raise(Internal,  Error,  HexConversionError)
     END
    END  ConvertToHex;
```

```
VAR
    NoNeeded,  CurrentWord,  CurrentNybble,  CurrentBit,
   NybbleSum,  Sum,  CharCount,  CharCounter                    : INTEGER;
   HexDigit                                                     : CHAR;

BEGIN
   IF  MCLength  >  0  THEN
       IF  MCLength  <=  MAXWORDSPERINSTRUCTION  THEN
        CurrentWord  :=  1;
        NybbleSum  :=  0;
        CharCount  :=  0;
          WHILE  (CurrentWord  <=  INTEGER(MCLength))  AND
                                        (NybbleSum <= MAXMCLISTLENGTH)  DO
           CurrentNybble  :=  (WORDLENGTH  DIV  4)  -  1;
           WHILE  CurrentNybble  >=  0  DO
            Sum  :=  0;
             CurrentBit  :=  0;
             WHILE  CurrentBit  <=  3  DO
                 IF  CARDINAL((CurrentNybble  *  4)  +  CurrentBit)
                                                        IN MCode[CurrentWord]  THEN
                       Sum  :=  Sum  +  INTEGER(Pow2(CurrentBit))
              END;
                INC(CurrentBit)
           END;
              HexDigit  :=  ConvertToHex(Sum);
               WriteAChar(ListingFile,  HexDigit);
             INC(CharCount);
             DEC(CurrentNybble);
             INC(NybbleSum)
         END;
           INC(CurrentWord);
            WriteAChar(ListingFile,  "  ");
           INC(CharCount)
       END;
         FOR  CharCounter  :=  CharCount  TO  25  DO
            WriteAChar(ListingFile,  "  ")
       END
     ELSE
          Raise(Internal,  Error,  MCLengthError)
     END
   ELSE
       WriteAChar(ListingFile,  "  ");
       FOR  CurrentWord  :=  1  TO  MAXWORDSPERINSTRUCTION  DO
         WriteAString(ListingFile,  "       ")
     END
   END
END  WriteMC;


(*  Constructors.  *)
PROCEDURE  AddLine(Address:  LONGCARD;  MCode:  Instruction;  MCLength:  CARDINAL;
                                                        SourceLine: String);

VAR
   SourceLength              :  CARDINAL;
   ShortLine                 :  String;
    SaveAddress,  SaveMC     :  BOOLEAN;

BEGIN
    IF  ListingSwitch  THEN
      IF  LineNo  =  1  THEN
         WriteHeader;
```

```
     END;
        SourceLength  :=  LengthOfString(SourceLine);
       IF  SourceLength  >  (LineLength  -  EXTRALINE)  THEN
            MakeSubstring(SourceLine,  1,  (LineLength  -  EXTRALINE),  SourceLine);
     END;
      (*  Source  Line  Number  *)
        WriteACard(ListingFile,  SourceNo,  4);
        WriteAString(ListingFile,  BLANK);

      (*  Address  *)
      IF  AddressSwitch  THEN
          WriteALongHex(ListingFile,  Address,  8);
         INC(SourceNo)
     ELSE
           WriteAString(ListingFile,  ADDRESSBLANK)
     END;
        WriteAString(ListingFile,  BLANK);

      (*  Machine  Code  *)
      IF  MCSwitch  THEN
          WriteMC(MCode,  MCLength)
     ELSE
           WriteAString(ListingFile,  MCBLANK)
     END;

      (*  Source  Line  *)
        WriteAString(ListingFile,  SourceLine);
        WriteALine(ListingFile);
      INCLine;

      (*  Error  if  Pending.  *)
      IF  ErrorPending  THEN
          WriteAString(ListingFile,  "Error:  ");
           WriteAString(ListingFile,  ErrorString);
          WriteALine(ListingFile);
        INCLine;
         ErrorPending  :=  FALSE
     END;
   END
END  AddLine;


PROCEDURE  AddError(ErrorMessage:  String);
BEGIN
   ErrorPending  :=  TRUE;
    MakeSubstring(ErrorMessage,  1,  LengthOfString(ErrorMessage),  ErrorString)
END  AddError;


PROCEDURE  StartListing(Filename:  String);

VAR
    ListingFName:  String;

BEGIN
    InitialiseAString(ListingFName);
    GetTime(StartHours,  StartMinutes,  StartSeconds);
    GetDate(StartYear,  StartMonth,  StartDay);
    StartTime  :=    (LONGCARD(StartHours)  *  3600)  +
                      (LONGCARD(StartMinutes)  *  60)  +
                      (LONGCARD(StartSeconds));
    MakeSubstring(Filename,  1,  LengthOfString(Filename),  ListingFName);
    ConcatStrings(ListingFName,  ".LST");
    ListingFile  :=  CreateAFile(ListingFName);
```

```
    ListingSwitch := TRUE;
   SourceNo := 1;
   LineNo := 1;
   PageNo := 1;
    AddressSwitch := TRUE;
   MCSwitch := TRUE;
    SymbolTableSwitch := FALSE;
    PageLength := DEFAULTPAGELENGTH;
    LineLength := DEFAULTLINELENGTH;
   TitleLength := 0;
    ErrorPending := FALSE
END StartListing;


PROCEDURE StopListing;

BEGIN
    IF SymbolTableSwitch THEN
       DumpSymbolTable
   END;
    DumpStatistics;
    WriteAChar(ListingFile, FF);
    CloseAFile(ListingFile);
   ListingSwitch := FALSE
END StopListing;


PROCEDURE SetPageLength(NewLength: CARDINAL);

BEGIN
    IF (NewLength < MINPAGELENGTH) OR (NewLength > MAXPAGELENGTH) THEN
        Raise(Code, Warning, PageLengthError);
       PageLength := DEFAULTPAGELENGTH
   ELSE
       PageLength := NewLength
   END
END SetPageLength;


PROCEDURE SetLineLength(NewLength: CARDINAL);

BEGIN
    IF (NewLength < MINLINELENGTH) OR (NewLength > MAXLINELENGTH) THEN
        Raise(Code, Warning, LineLengthError);
       LineLength := DEFAULTLINELENGTH
   ELSE
       LineLength := NewLength
   END
END SetLineLength;


PROCEDURE INCPage;

BEGIN
    INC(PageNo);
    WriteAChar(ListingFile, FF);
   LineNo := 1
END INCPage;


PROCEDURE INCLine;

BEGIN
   IF LineNo = PageLength - EXTRAPAGE THEN
```

```
     INCPage
   ELSE
      INC(LineNo)
   END
END  INCLine;


PROCEDURE  SetTitle(NewTitle:  String);

BEGIN
    TitleLength  :=  LengthOfString(NewTitle);
    IF  TitleLength  >  MAXTITLELENGTH  THEN
       TitleLength  :=  MAXTITLELENGTH
   END;
    MakeSubstring(NewTitle,  1,  TitleLength,  TitleString)
END  SetTitle;


PROCEDURE  PurgeError;
BEGIN
   ErrorPending  :=  FALSE
END  PurgeError;


PROCEDURE  Address(Include:  BOOLEAN);

BEGIN
   AddressSwitch  :=  Include
END  Address;


PROCEDURE  MachineCode(Include:  BOOLEAN);

BEGIN
   AddressSwitch  :=  Include
END  MachineCode;


PROCEDURE  SymbolTable(Include:  BOOLEAN);

BEGIN
    SymbolTableSwitch  :=  Include
END  SymbolTable;


(*  Predicates.  *)
PROCEDURE  CurrentlyListing():  BOOLEAN;

BEGIN
    RETURN  ListingSwitch
END  CurrentlyListing;


PROCEDURE  CurrentSourceLine():  CARDINAL;

BEGIN
   RETURN  SourceNo
END  CurrentSourceLine;


PROCEDURE  CurrentLine():  CARDINAL;

BEGIN
   RETURN  LineNo
```

```
END  CurrentLine;


PROCEDURE  CurrentPage():  CARDINAL;

BEGIN
   RETURN  PageNo
END  CurrentPage;


PROCEDURE  CurrentPageLength():  CARDINAL;

BEGIN
   RETURN  PageNo
END  CurrentPageLength;


PROCEDURE  CurrentLineLength():  CARDINAL;

BEGIN
   RETURN  LineNo
END  CurrentLineLength;


PROCEDURE  MCAssemblyRate():  LONGINT;

BEGIN
    RETURN  AssemblyRate
END  MCAssemblyRate;


PROCEDURE  ListingShutdown;

BEGIN
    CloseAFile(ListingFile)
END  ListingShutdown;


END  Listing.
```

```
IMPLEMENTATION  MODULE  Location;

(*
          MODULE: Location          Version 1.0      (c) 1989 Mark Wickens

          Created: 28-11-89

           Handles the location of the location counter relative to the start
          of the source file.
*)

FROM ADM           IMPORT   MEMLOW, MEMHIGH;
FROM  Exceptions  IMPORT    Raise,  ExceptionType,  ExceptionLevel;

CONST
   LocationError  =  "Location  of  current  instruction  is  out  of  bounds";

VAR
   LocationCounter:  LONGCARD;


PROCEDURE  ResetLocation;
BEGIN
   LocationCounter  :=  MEMLOW
END  ResetLocation;


PROCEDURE  SetLocation(NewLocation:  LONGCARD);
BEGIN
   IF  (NewLocation  >=  MEMLOW)  AND  (NewLocation  <=  MEMHIGH)  THEN
      LocationCounter  :=  NewLocation
  ELSE
      Raise(Code,  Fatal,  LocationError)
  END
END  SetLocation;


PROCEDURE  CurrentLocation():  LONGCARD;
BEGIN
   RETURN  LocationCounter
END  CurrentLocation;


PROCEDURE  INCLocation(Increment:  CARDINAL);
BEGIN
   IF  LocationCounter  +  LONGCARD(Increment)  <  MEMHIGH  THEN
       INC(LocationCounter,  LONGCARD(Increment))
  ELSE
      Raise(Code,  Fatal,  LocationError)
  END
END  INCLocation;


PROCEDURE  LocationShutdown;
BEGIN
   LocationCounter  :=  MEMLOW
END  LocationShutdown;


END  Location.
```

```
MODULE  M2Assem;

(*
          MODULE: M2Assem            Version 1.0       (c)1990 Mark Wickens

          Created: 22/01/90

           This is the main driver program for the Meta-Assembler.
*)

FROM ADM                        IMPORT InsertOpcodesInTable, Assemble, Instruction;
FROM Strings                    IMPORT String, InitialiseAString, EqualStrings,
                                       ConcatStrings, MakeSubstring, LengthOfString;
FROM Listing                    IMPORT StartListing, StopListing, AddLine, PurgeError,
                                       SymbolTable, CurrentSourceLine, MCAssemblyRate;
FROM ObjectGenerator        IMPORT StartObject, StopObject, SetFormat, ObjectFormat,
                                   AddCode;
FROM Exceptions                 IMPORT Raise, ExceptionLevel, ExceptionType, Number,
                                       ProgramStatus, StatusType;
FROM Interface                  IMPORT File, NumberOfArguments, ReadArgument, OpenAFile,
                                       WriteAString, WriteALine, StdOut, ReadAString,
                                       EndOfFile, CloseAFile, FileExists, WriteALongInt,
                                       WriteACard, NULLFile, WriteAChar;
FROM Lex                        IMPORT DecodedLine, ScanLine;
FROM Location                   IMPORT ResetLocation, CurrentLocation, INCLocation;

CONST
   ArgumentError = "Bad or missing filename";
   FileMissingError = "Input file could not be found";
   Message1 = "Modula-2 Meta-Assembler Version 1.0 (c)1990 Mark Wickens";
   Message2 = "Usage : M2Assem <filename>";
   Message3 = "where <filename> is name of source file without extension.";
   Message4 = "Extensions used: Source = .ASM, Listing = .LST, Object = .OBJ";
   Pass1Message = "Assembling: Pass 1";
   Pass2Message = "Assembling: Pass 2";
  CR            = 15C;

VAR
  PassNo                      : CARDINAL;
   Filename, InputFile        : String;
  InputHandle                 : File;
  InputLine                   : String;
  Decoded                     : DecodedLine;
  CurrentMC                   : Instruction;
  MCLength                    : CARDINAL;
  LineNo                      : CARDINAL;


PROCEDURE ShowUsage;
BEGIN
   WriteAString(StdOut, Message2); WriteALine(StdOut);
   WriteAString(StdOut, Message3); WriteALine(StdOut);
   WriteAString(StdOut, Message4); WriteALine(StdOut);
END ShowUsage;


PROCEDURE OpenInputFile;
   PROCEDURE GetFilename(VAR Filename: String);

  BEGIN
      InitialiseAString(Filename);
     IF NumberOfArguments() = 1 THEN
        ReadArgument(1, Filename);
        IF EqualStrings(Filename, "?") THEN
```

```
          ShowUsage;
(*          Raise(User, Fatal, ArgumentError)*)
         HALT
       END
     ELSE
        ShowUsage;
          Raise(User, Fatal, ArgumentError)
     END
    END GetFilename;

BEGIN
    GetFilename(Filename);
    InitialiseAString(InputFile);
    MakeSubstring(Filename, 1, LengthOfString(Filename), InputFile);
    ConcatStrings(InputFile, ".ASM")
END OpenInputFile;


PROCEDURE OpenOutputFiles;
BEGIN
    IF FileExists(InputFile) THEN
       SetFormat(Generic);
       StartObject(Filename);
       StartListing(Filename)
   ELSE
      ShowUsage;
        Raise(Environment, Fatal, FileMissingError)
   END
END OpenOutputFiles;


PROCEDURE CloseFiles;

BEGIN
    StopListing;
    StopObject
END CloseFiles;


PROCEDURE WriteStats;

BEGIN
    WriteAString(StdOut, "Assembled ");
    WriteACard(StdOut, CurrentSourceLine()-1, 1);
    WriteAString(StdOut, " lines at a rate of ");
    WriteALongInt(StdOut, MCAssemblyRate(), 1);
    WriteAString(StdOut, " lines per minute.");
    WriteALine(StdOut);
    WriteAString(StdOut, "There were ");
    WriteACard(StdOut, Number(), 1);
    WriteAString(StdOut, " exception(s) raised during assembly.");
    WriteALine(StdOut);
    IF ProgramStatus() = Normal THEN
       WriteAString(StdOut, "A valid object code file was writted.")
   ELSE
       WriteAString(StdOut, "Exceptions caused Object Code output to be halted.")
   END;
    WriteALine(StdOut);
    WriteALine(StdOut)
END WriteStats;


PROCEDURE Pass1;
BEGIN
```

```
   PassNo := 1;
   LineNo := 1;
    WriteAString(StdOut, Pass1Message); WriteALine(StdOut);
    InputHandle := OpenAFile(InputFile);
    IF InputHandle <> NULLFile THEN
        InsertOpcodesInTable;
       ResetLocation;
        ReadAString(InputHandle, InputLine);
       WHILE NOT EndOfFile() DO
           Decoded := ScanLine(InputLine);
            WriteAString(StdOut, "Assembling line: ");
           WriteACard(StdOut, LineNo, 6);
           WriteAChar(StdOut, CR);
            Assemble(Decoded, PassNo, CurrentMC, MCLength);
           INCLocation(MCLength);
          INC(LineNo);
            ReadAString(InputHandle, InputLine);
     END;
        CloseAFile(InputHandle);
       PurgeError
   END
END Pass1;


PROCEDURE Pass2;
BEGIN
   PassNo := 2;
   LineNo := 1;
    WriteAString(StdOut, Pass2Message); WriteALine(StdOut);
    InputHandle := OpenAFile(InputFile);
    IF InputHandle <> NULLFile THEN
       ResetLocation;
        ReadAString(InputHandle, InputLine);
       WHILE NOT EndOfFile() DO
           Decoded := ScanLine(InputLine);
            WriteAString(StdOut, "Assembling line: ");
           WriteACard(StdOut, LineNo, 6);
           WriteAChar(StdOut, CR);
            Assemble(Decoded, PassNo, CurrentMC, MCLength);
             AddLine(CurrentLocation(), CurrentMC, MCLength, InputLine);
             AddCode(CurrentLocation(), CurrentMC, MCLength);
           INCLocation(MCLength);
          INC(LineNo);
            ReadAString(InputHandle, InputLine);
     END;
        CloseAFile(InputHandle)
   END
END Pass2;

BEGIN
    WriteAString(StdOut, Message1);
    WriteALine(StdOut);
    OpenInputFile;
   Pass1;
    OpenOutputFiles;
   Pass2;
    CloseFiles;
    WriteStats
END M2Assem.
```

```
IMPLEMENTATION  MODULE  ObjectGenerator;

(*
           MODULE: ObjectGenerator Version 1.0       (c) 1989 Mark Wickens

          Created: 28-11-89
           Updated: 22-01-90          Added implementation module.

            Produces the object file in a format determined by calling module.
*)

FROM Strings                  IMPORT   String, InitialiseAString, MakeSubstring,
                                       ConcatStrings, LengthOfString;
FROM Interface                IMPORT   CreateAFile, CloseAFile, WriteALongHex,
                                        WriteAString, WriteAChar, FileHandle,
                                     WriteALine;
FROM Exceptions               IMPORT   Raise, ExceptionType, ExceptionLevel;
FROM ADM                      IMPORT   MAXWORDSPERINSTRUCTION, WORDLENGTH;

CONST
   FormatError     = "Cannot change object code format while assembling";
   HexConversionError       = "Illegal Number to Hex Converter";

VAR
    ObjectFile: FileHandle;
    GenerateObject: BOOLEAN;
    CurrentFormat: ObjectFormat;

(* Constructors. *)
PROCEDURE StartObject(Filename: String);

VAR
    ObjectFName: String;

BEGIN
   IF GenerateObject THEN
       InitialiseAString(ObjectFName);
       MakeSubstring(Filename, 1, LengthOfString(Filename), ObjectFName);
       ConcatStrings(ObjectFName, ".OBJ");
       ObjectFile := CreateAFile(ObjectFName);
  END
END StartObject;


PROCEDURE StopObject;
BEGIN
   IF GenerateObject THEN
       CloseAFile(ObjectFile);
      GenerateObject := FALSE
  END
END StopObject;


PROCEDURE SetFormat(NewType: ObjectFormat);
BEGIN
   CurrentFormat := NewType
END SetFormat;


PROCEDURE WriteObject(MCode: Instruction; MCLength: CARDINAL);


    PROCEDURE Pow2(Number: CARDINAL): CARDINAL;
```

```
   VAR Sum, Counter: CARDINAL;

  BEGIN
    Sum := 1;
    IF Number > 0 THEN
       FOR Counter := 1 TO Number DO
         Sum := Sum * 2
      END;
    END;
    RETURN Sum
  END Pow2;


   PROCEDURE ConvertToHex(input: CARDINAL): CHAR;
  BEGIN
    CASE input OF
       0..9:       RETURN (CHR(ORD('0') + input)) |
       10..15:     RETURN (CHR(ORD('A') + input - 10))
    ELSE
          Raise(Internal, Error, HexConversionError)
    END
   END ConvertToHex;


VAR
   NoNeeded, CurrentWord, CurrentNybble, CurrentBit,
  NybbleSum, Sum                                          : INTEGER;
  HexDigit                                                : CHAR;

BEGIN
   IF (MCLength > 0) AND (MCLength <= MAXWORDSPERINSTRUCTION) THEN
     CurrentWord := 1;
     NybbleSum := 0;
      WHILE CurrentWord <= INTEGER(MCLength) DO
        CurrentNybble := (WORDLENGTH DIV 4) - 1;
       WHILE CurrentNybble >= 0 DO
        Sum := 0;
         CurrentBit := 0;
         WHILE CurrentBit <= 3 DO
             IF CARDINAL((CurrentNybble * 4) + CurrentBit)
                                                   IN MCode[CurrentWord] THEN
               Sum := Sum + INTEGER(Pow2(CurrentBit))
           END;
             INC(CurrentBit)
         END;
           HexDigit := ConvertToHex(Sum);
            WriteAChar(ObjectFile, HexDigit);
           DEC(CurrentNybble);
          INC(NybbleSum)
      END;
        INC(CurrentWord);
    END
  END
END WriteObject;


PROCEDURE AddCode(Address: LONGCARD; MCCode: Instruction; MCLength: CARDINAL);
BEGIN
   IF GenerateObject AND (MCLength > 0) THEN
      IF CurrentFormat = Generic THEN
          WriteALongHex(ObjectFile, Address, 8);
         WriteAChar(ObjectFile, ' ');
          WriteObject(MCCode, MCLength);
         WriteALine(ObjectFile)
```

```
     END
   END
END  AddCode;


PROCEDURE  ObjectOutput(Switch:  BOOLEAN);
BEGIN
    GenerateObject  :=  Switch
END  ObjectOutput;


PROCEDURE  ObjectGeneratorShutdown;
BEGIN
    IF  GenerateObject  THEN
        CloseAFile(ObjectFile);
       GenerateObject  :=  FALSE
   END
END  ObjectGeneratorShutdown;


BEGIN
    GenerateObject  :=  TRUE
END  ObjectGenerator.
```

```
IMPLEMENTATION  MODULE  PseudoOps;

(*
          MODULE: PseudoOps         Version 1.0       (c)1990 Mark Wickens

          Created: 03/03/90

            This module handles all valid operations not assembler specific,
            i.e., all the assembly language-independent pseudo operations.
*)

FROM TableExt                IMPORT   InsertCommand;
FROM ADM                     IMPORT   MEMLOW, MEMHIGH, Word;
FROM Location                IMPORT   SetLocation, CurrentLocation;
FROM Exceptions              IMPORT   Raise, ExceptionLevel, ExceptionType;
FROM TableExt                IMPORT   SymbolStatus, InsertSymbol, TableType,
                                      EntryType;
FROM Listing                 IMPORT   SetPageLength, SetLineLength, INCPage,
                                      SymbolTable;
FROM Expression              IMPORT   Evaluate, ChangeBase, BaseType;
FROM Table                   IMPORT   Amend;
FROM Strings                 IMPORT   MakeSubstring, LengthOfString;
FROM ObjectGenerator         IMPORT   ObjectOutput;

PROCEDURE AssemblePseudoOp(DecInstr: DecodedLine; PassNo, Type: CARDINAL;
                           VAR Length: CARDINAL);
(*
           Performs the appropriate action on the peudo-op found.
*)

TYPE
    SwitchType = (SymbolSwitch, ObjectSwitch);

CONST
    OperandCountError = "Wrong number of operands";
    IndicesCountError = "Wrong number of indices";
    LocationRangeError = "Attempted to exceed location max/min limits";
    RelativeError = "Operand evaluated to a relative result";


    PROCEDURE DoORG;
   (*
           Relocates the origin of code produced from source line onwards.
   *)
   VAR
      Origin: LONGINT;
       OriginType: SymbolStatus;

   BEGIN
     Length := 0;
      IF DecInstr.NoOfOperands = 1 THEN
         IF DecInstr.Operand[1].NoOfIndices = 0 THEN
              Evaluate(DecInstr.Operand[1].Argument, Origin, OriginType);
           IF OriginType = Absolute THEN
              IF (Origin >= MEMLOW) AND (Origin <= MEMHIGH) THEN
                 SetLocation(LONGCARD(Origin))
           ELSE
                 Raise(Code, Error, LocationRangeError)
           END
         ELSE
                SetLocation(LONGCARD(Origin) - CurrentLocation());
              Raise(Code, Warning, RelativeError)
         END
       ELSE
```

```
          Raise(Code, Error, IndicesCountError)
     END
   ELSE
        Raise(Code, Error, OperandCountError)
   END
 END DoORG;


 PROCEDURE DoEQU;
(*
        Equates an expression to a symbol.
*)
VAR
    Result: LONGINT;
     ResultType: SymbolStatus;
     AmendEntry: TableType;

 BEGIN
   Length := 0;
   IF PassNo = 1 THEN
       IF DecInstr.NoOfOperands = 1 THEN
          IF DecInstr.Operand[1].NoOfIndices = 0 THEN
               Evaluate(DecInstr.Operand[1].Argument, Result, ResultType);
                MakeSubstring(DecInstr.Label, 1, LengthOfString(DecInstr.Label),
                             AmendEntry.Key);
            AmendEntry.Kind := Symbol;
            AmendEntry.Value := Result;
             AmendEntry.Status := ResultType;
           Amend(AmendEntry)
       ELSE
             Raise(Code, Error, IndicesCountError)
       END
     ELSE
          Raise(Code, Error, OperandCountError)
     END
   END
 END DoEQU;


 PROCEDURE DoEND;
 END DoEND;


 PROCEDURE DoPAGE;
(*
        If no arguments, advances by a page.
        If one argument, sets up page length.
         If two arguments, sets up page length and width.
*)
VAR
    Length, Width: LONGINT;
     LengthType, WidthType: SymbolStatus;

 BEGIN
   Length := 0;
   IF PassNo = 2 THEN
       IF (DecInstr.Operand[1].NoOfIndices = 0) AND
           (DecInstr.Operand[2].NoOfIndices = 0) THEN
         CASE DecInstr.NoOfOperands OF
         0: INCPage                 |
              1: Evaluate(DecInstr.Operand[1].Argument, Length, LengthType);
               SetPageLength(CARDINAL(Length)) |
              2: Evaluate(DecInstr.Operand[1].Argument, Length, LengthType);
               SetPageLength(CARDINAL(Length));
```

```
                    Evaluate(DecInstr.Operand[1].Argument, Width, WidthType);
                 SetLineLength(CARDINAL(Width))
        END
      ELSE
           Raise(Code, Error, IndicesCountError)
      END
    END
  END DoPAGE;


  PROCEDURE DoTITLE;
  END DoTITLE;


  PROCEDURE DoDATA;
  END DoDATA;


  PROCEDURE DoRES;
  END DoRES;


  PROCEDURE DoBASE;
  CONST
     BaseError = "Invalid base (must be 2, 8, 10 or 16)";

  VAR
     Base: LONGINT;
      BaseType: SymbolStatus;

  BEGIN
    Length := 0;
    IF PassNo = 1 THEN
        IF DecInstr.NoOfOperands = 1 THEN
            IF DecInstr.Operand[1].NoOfIndices = 0 THEN
                 Evaluate(DecInstr.Operand[1].Argument, Base, BaseType);
             IF BaseType = Absolute THEN
                CASE CARDINAL(Base) OF
                  2:  ChangeBase(Bin)          |
                  8:  ChangeBase(Oct)          |
                  10: ChangeBase(Dec)          |
                   16: ChangeBase(Hex)
             ELSE
                  Raise(Code, Error, BaseError)
             END
           ELSE
                Raise(Code, Error, RelativeError)
           END
        ELSE
             Raise(Code, Error, IndicesCountError)
        END
      ELSE
           Raise(Code, Error, OperandCountError)
      END
    END
  END DoBASE;


   PROCEDURE DoSWITCH(Switch: SwitchType);
  VAR
     Result: LONGINT;
      ResultType: SymbolStatus;
      SwitchStatus: BOOLEAN;
```

```
   BEGIN
     Length := 0;
     IF PassNo = 1 THEN
         IF DecInstr.NoOfOperands = 1 THEN
             IF DecInstr.Operand[1].NoOfIndices = 0 THEN
                  Evaluate(DecInstr.Operand[1].Argument, Result, ResultType);
              IF ResultType = Absolute THEN
               IF Result = 1 THEN
                  SwitchStatus := TRUE
              ELSE
                  SwitchStatus := FALSE
              END;
               CASE Switch OF
                   SymbolSwitch:    SymbolTable(SwitchStatus)         |
                    ObjectSwitch:    ObjectOutput(SwitchStatus)
              END
            ELSE
                 Raise(Code, Error, RelativeError)
            END
          ELSE
               Raise(Code, Error, IndicesCountError)
          END
        ELSE
             Raise(Code, Error, OperandCountError)
        END
      END
    END DoSWITCH;


    PROCEDURE DoOBJ;
   END DoOBJ;


BEGIN
   CASE Type OF
     1000: DoORG                    |
     1001: DoEQU                    |
     1002: DoEND                    |
     1003: DoPAGE                   |
     1004: DoTITLE                  |
     1005: DoDATA                   |
     1006: DoRES                    |
     1007: DoBASE                   |
        1008:  DoSWITCH(SymbolSwitch)|
        1009:  DoSWITCH(ObjectSwitch)
   END

END AssemblePseudoOp;


PROCEDURE PseudoOpsShutdown;
END PseudoOpsShutdown;


BEGIN
(*
            Main program part inserts all the available pseudo-ops in the hash-
            table. All pseudo-ops in this module have a type number of greater
            than 1000, so there is plenty of room for op-codes (they could use
            greater than the last pseudo-op if necessary - unlikely!).
*)
    InsertCommand("ORG",       Word{},  1000);
    InsertCommand("EQU",       Word{},  1001);
    InsertCommand("END",       Word{},  1002);
```

```
    InsertCommand("PAGE",      Word{},  1003);
    InsertCommand("TITLE",    Word{},  1004);
    InsertCommand("DATA",      Word{},  1005);
    InsertCommand("RES",       Word{},  1006);
    InsertCommand("BASE",      Word{},  1007);
    InsertCommand("SYMBOL",  Word{},  1008);
    InsertCommand("OBJECT",  Word{},  1009);
END  PseudoOps.
```

```
IMPLEMENTATION MODULE Strings;

(*
            This module implements the objects exported from the definition
         module.
*)

CONST
   NewLineCh       = CHR(10);
   HT              = CHR(9);


PROCEDURE InitialiseAString(VAR S: String);

BEGIN
   S[1] := EndOfLine
END InitialiseAString;


PROCEDURE ArrayToString(Array: ARRAY OF CHAR; VAR S: String);

VAR
   Counter         : CARDINAL;

BEGIN
   Counter := 0;
    WHILE (Array[Counter] <> EndOfLine) AND (Counter <= LongestString - 1) DO
       S[Counter + 1] := Array[Counter];
       INC(Counter)
   END;
    S[Counter + 1] := EndOfLine
END ArrayToString;


PROCEDURE StringToArray(S: String; VAR Array: ARRAY OF CHAR);

VAR
   Counter         : CARDINAL;

BEGIN
   Counter := 1;
    WHILE S[Counter] <> EndOfLine DO
       Array[Counter - 1] := S[Counter];
       INC(Counter)
   END
END StringToArray;


PROCEDURE CapitaliseString(VAR S: String);
VAR
    Position, Length: CARDINAL;

BEGIN
    Length := LengthOfString(S);
   FOR Position := 1 TO Length DO
       S[Position] := CAP(S[Position])
   END
END CapitaliseString;


PROCEDURE ConcatStrings(VAR S1: String; S2: String);
VAR
    Position, Length1, Length2: CARDINAL;
```

```
BEGIN
    Length1  :=  LengthOfString(S1);
    Length2  :=  LengthOfString(S2);
    IF  Length1  +  Length2  <  LongestString  THEN
       FOR  Position  :=  Length1  +  1  TO  Length1  +  1  +  Length2  DO
          S1[Position]  :=  S2[Position  -  Length1]
     END
   ELSE
       FOR  Position  :=  Length1  +  1  TO  LongestString  DO
          S1[Position]  :=  S2[Position  -  Length1]
     END
   END
END  ConcatStrings;


PROCEDURE  CharInString(S:  String;  Position:  CARDINAL):  CHAR;

BEGIN
    IF  Position  >  LengthOfString(S)  THEN
       RETURN  EndOfLine
   ELSE
       RETURN  S[Position]
   END
END  CharInString;


PROCEDURE  LengthOfString(S:  String):  CARDINAL;
VAR
   I:  CARDINAL;
   Sum:  CARDINAL;

BEGIN
   I  := 1;
   Sum  := 0;
   WHILE  S[I]  <>  EndOfLine  DO
      IF  S[I]  =  HT  THEN
         REPEAT
            INC(Sum)
         UNTIL  (Sum  MOD  8)  =  0;
         INC(I)
      ELSE
         INC(I);
         INC(Sum)
      END
   END;
   RETURN  Sum
END  LengthOfString;


PROCEDURE  MakeSubstring(S:  String;  StartPos,  Length:  CARDINAL;
                                                        VAR Substring: String);

VAR
    Dest,  Index:  CARDINAL;

BEGIN
   Dest  := 1;
   FOR  Index  :=  StartPos  TO  StartPos  +  Length  -  1  DO
       Substring[Dest]  :=  S[Index];
      Dest  := Dest  +  1
   END;
    Substring[Dest]  :=  EndOfLine
END  MakeSubstring;
```

```
PROCEDURE  EmptyString(S:  String):  BOOLEAN;

BEGIN
   RETURN  S[1]  =  EndOfLine
END  EmptyString;


PROCEDURE  ReverseString(VAR  S:  String);

VAR
  T:     CHAR;
   Front,  Back:   CARDINAL;

BEGIN
  Front  :=  1;
   Back  :=  LengthOfString(S);
  WHILE  Front  <  Back  DO
     T  :=  S[Front];
      S[Front]  :=  S[Back];
     S[Back]  :=  T;
      INC(Front);
     DEC(Back)
  END
END  ReverseString;


PROCEDURE  AlphabeticallyLess(S1,  S2:  String):  BOOLEAN;

VAR
   Index:  CARDINAL;
   Comparing,  ALess:  BOOLEAN;

BEGIN
  Index  :=  1;
   Comparing  :=  TRUE;
   WHILE  Comparing  DO
      IF  (Index  >  LongestString)  OR  (S2[Index]  =  EndOfLine)  THEN
        Comparing  :=  FALSE;
        ALess  :=  FALSE
      ELSIF  S1[Index]  =  EndOfLine  THEN
        Comparing  :=  FALSE;
        ALess  :=  TRUE
      ELSIF  S1[Index]  =  S2[Index]  THEN
        INC(Index)
     ELSE
        Comparing  :=  FALSE;
         ALess  :=  S1[Index]  <  S2[Index]
     END;
  END;
             RETURN ALess
END  AlphabeticallyLess;


PROCEDURE  EqualStrings(S1,  S2:  String):  BOOLEAN;

VAR
   Index:  CARDINAL;
   Comparing,  Same:   BOOLEAN;

BEGIN
  Index  :=  1;
   Comparing  :=  TRUE;
   WHILE  Comparing  DO
```

```
      IF (Index > LongestString) OR
                        ((S1[Index] = EndOfLine) AND (S2[Index] = EndOfLine)) THEN
         Comparing := FALSE;
        Same := TRUE
      ELSIF S1[Index] = S2[Index] THEN
        INC(Index)
     ELSE
         Comparing := FALSE;
        Same := FALSE
     END
   END;
   RETURN Same
END EqualStrings;


PROCEDURE StringsShutdown;
END StringsShutdown;


END Strings.
```

```
IMPLEMENTATION  MODULE  Table;

(*
          MODULE Table               Version 1.0      (c) 1989 Mark Wickens

           Created: 02-12-89
*)

FROM  Strings        IMPORT    String,  EqualStrings,  MakeSubstring,  LengthOfString,
                           CharInString;
FROM  TableTrees  IMPORT    BST,  InsertInTree,  IsInTree,  DeleteFromTree,
                                RetrieveFromTree, ElementsInTree, NextInOrderInTree,
                              InitialiseATree, IsRoomInTree;

TYPE
   TablePtr        = BST;              (* A TablePtr points to a Binary Search Tree. *)

CONST
   TableSize       = 100;              (* Size of Hash table, excluding Table[0]. *)

VAR
   HashTable                  : ARRAY  [0..TableSize]  OF  TablePtr;
                                        (* Hash Table is an array of TablePtr's. *)
   CurrentNext    :  String;          (* The key of the next String in order, used
                                            when searching through the complete table
                                          alphabetically. *)

PROCEDURE  Hash(KeyToHash:  String):  CARDINAL;
(*
            This operation produces locations in a hash table by performing
            a transformation on a key 'KeyToHash'.   The algorithm is based on
            the Division Hashing Algormith as defined in

                     Knuth, Donald E           'The Art Of Computer Programming'
                                               - Volume 3 'Sorting and Searching'
                     Section 6.4 Hashing, pp 509.

           As quoted from Knuth,
            'Extensive tests on typical files have shown that two major
            types of hash functions work quite well.   One of these is
            based on division, and the other is based on multiplication.'
            Both functions work well as progressions, ie., similar input data are
            mapped onto arithmetic progressions, which reduces the number of
            collisions produced by similar data.   The algorithm is,

           h(K)  =  K mod M where

           K is the Key to be Hashed,
           h(K)  is the hashed key,
           M is the Size of the Table,
*)

    PROCEDURE  HashString(KeyToHash:  String):  LONGCARD;
   (*
            Procedure to produce a HashValue from a key.   In this case,
            all members of the String are multiplied by their position in the
           string and added together.
            It should be noted that the maximum value produced by this procedure
            (for a string of length 30, all 'z's) is under 10,000.
   *)
   VAR
       CharPos,  Position,  Length     :  CARDINAL;
     Char                              : CHAR;
     HashValue                         : LONGCARD;
```

```
   BEGIN
      HashValue := 0;       (* Ready to be multiplied by first value. *)
        Length  := LengthOfString(KeyToHash);
      FOR Position := 1 TO Length DO
          CharPos     := ORD(CharInString(KeyToHash, Position)) * Position;
                     (* CharPos := Position of Char in alphabet * Position of Char
                      in the string. *)
          HashValue := HashValue + LONGCARD(CharPos)(* Keeps a running total. *)
     END;
      RETURN HashValue      (* Total calculated. *)
   END HashString;

BEGIN
    RETURN CARDINAL(HashString(KeyToHash) MOD LONGCARD(TableSize))
            (* Performs the algorithm detailed earlier, and returns result. *)
END Hash;

PROCEDURE InitialiseTable;
(*         Resets the complete table by initalising the structure at each node. *)
VAR
    Current: CARDINAL;

BEGIN
   FOR Current := 0 TO TableSize DO
       InitialiseATree(HashTable[Current])
   END
END InitialiseTable;

PROCEDURE IsRoom(): BOOLEAN;
(*         Checks whether there would be enough room for another node.*)
BEGIN
    RETURN IsRoomInTree()
END IsRoom;

PROCEDURE IsIn(Key: String): BOOLEAN;
(*         Checks whether there is an entry for surname in the table.*)
BEGIN
    RETURN IsInTree(HashTable[Hash(Key)], Key)
END IsIn;

PROCEDURE Insert(Entry: TableType);
(*       Inserts a record into the table.                            *)
BEGIN
    InsertInTree(HashTable[Hash(Entry.Key)], Entry)
END Insert;

PROCEDURE Retrieve(Key: String): TableType;
(*       Retrieves a record from the table.                          *)
VAR
    TableEntry: TableType;

BEGIN
    RetrieveFromTree(HashTable[Hash(Key)], Key, TableEntry);
    RETURN TableEntry
END Retrieve;

PROCEDURE Delete(Key: String);
(*       Deletes a record from a table.                              *)
BEGIN
    DeleteFromTree(HashTable[Hash(Key)], Key)
END Delete;

PROCEDURE Amend(VAR Entry: TableType);
```

```
(*        Amends the details of a record in the table.               *)
BEGIN
    DeleteFromTree(HashTable[Hash(Entry.Key)],  Entry.Key);
    InsertInTree(HashTable[Hash(Entry.Key)],  Entry)
END  Amend;

PROCEDURE  Greater(Key:  String):  BOOLEAN;
(*         Checks whether there is an entry in the table which is the next in
         order.                                                        *)
VAR
   Counter:  CARDINAL;       (* Counts through the entries in the table.*)

BEGIN
    MakeSubstring(Key,  1,  LengthOfString(Key),  CurrentNext);
                                                (* CurrentNext := Key *)
   FOR Counter := 0 TO TableSize DO          (* Count through each location in
                                                 the table.   *)
        NextInOrderInTree(HashTable[Counter],  CurrentNext,  Key)
   END;
    RETURN  NOT  EqualStrings(Key,  CurrentNext)
                                                 (* If Key = CurrentNext, there is
                                                    not an entry in the table with a
                                                   greater surname. *)
END  Greater;

PROCEDURE  NextInOrder(Key:  String;  VAR  Entry:  TableType);
(*        Gets the next entry in the table with a greater Key.      *)
BEGIN
   IF IsIn(CurrentNext) THEN                   (* If CurrentNext exists in table, *)
     Entry := Retrieve(CurrentNext)        (* retrieve it.                *)
   END
END  NextInOrder;

PROCEDURE  SizeOfTable():  CARDINAL;
(*        Returns the number of locations in the hash table.        *)
BEGIN
   RETURN TableSize + 1   (* Add one because the table starts at location 0. *)
END  SizeOfTable;

PROCEDURE  ElementsAtLocation(Location:  CARDINAL):  CARDINAL;
(*         Returns the number of entries at a location in the table.*)
BEGIN
    RETURN  ElementsInTree(HashTable[Location])       (* Get number of elements in
                                                            the tree at that position. *)
END  ElementsAtLocation;

PROCEDURE  TableShutdown;
END  TableShutdown;

END  Table.
```

```
IMPLEMENTATION  MODULE  TableExt;

(*
          Module: TableExt          Version 1.0      (c) 1989 Mark Wickens

          Created: 24-12-89

           This module serves two purposes.

           (1).  It contains all the data specifics so that any changes to the
            record type stored within the table are contained in one module only.

           (2).  It extends the basic table operations to allow specific fields
            of a table record to be specified as parameters in an insert procedure,
            allowing the initial set-up of the table to be performed without
           reference to variables.
*)
CONST
FROM ADM           IMPORT   WORDLENGTH;
FROM Table         IMPORT   Insert, IsIn, IsRoom;
FROM  Exceptions  IMPORT    ExceptionLevel,  ExceptionType,  Raise;
FROM  Strings        IMPORT    EqualStrings,  LengthOfString,  CharInString,  EndOfLine,
                                 InitialiseAString, LongestString, ArrayToString;
FROM  Interface    IMPORT    ReadAString,  WriteAString,  ReadACard,  WriteACard,
                                 WriteALine, ReadALongInt, WriteALongInt, StdIn, StdOut;

CONST
   TableFullError            = "Table size has exceeded memory availability";
   InsertError               = "Key has already been inserted in table";
   WordLengthError           = "Length of mask string is not WORDLENGTH";
            a transformation on a key 'KeyToHash'.  The algorithm is based on
            the Division Hashing Algormith as defined in
(*  Extended  Operations.  *)
                    Knuth, Donald E           'The Art Of Computer Programming'
                                              - Volume 3 'Sorting and Searching'
PROCEDURE  ConvertTo(Mask:  String):  Word;

VAR
   ConvInstruction          : Word;
   Length,  CurrentPos      : CARDINAL;
            based on division, and the other is based on multiplication.'
BEGIN
    Length  :=  LengthOfString(Mask);
   IF  Length  <>  WORDLENGTH  THEN
        Raise(Internal,  Error,  WordLengthError)
   ELSE
      FOR  CurrentPos  :=  0  TO  WORDLENGTH  -  1  DO
         IF  CharInString(Mask,  CurrentPos  +  1)  =  '1'  THEN
             INCL(ConvInstruction,  WORDLENGTH  -  CurrentPos);
       END
     END
   END;
    RETURN  ConvInstruction
END  ConvertTo;
            Procedure to produce a HashValue from a key.  In this case,
            all members of the String are multiplied by their position in the
PROCEDURE  ConvertFrom(Mask:  Word;  VAR  ConvString:  String);
            It should be noted that the maximum value produced by this procedure
VAR
   CurrentPos      :  CARDINAL;
    StringSoFar     :  ARRAY  [1..LongestString]  OF  CHAR;
      CharPos,  Position,  Length     :  CARDINAL;
BEGIN
    InitialiseAString(ConvString);
```

```
   CurrentPos := 0;
   WHILE CurrentPos <= WORDLENGTH - 1 DO
      IF CurrentPos IN Mask THEN
          StringSoFar[WORDLENGTH - CurrentPos] := '1'
     ELSE
          StringSoFar[WORDLENGTH - CurrentPos] := '0'
     END;
       INC(CurrentPos)
   END;
    StringSoFar[WORDLENGTH + 1] := EndOfLine;
    ArrayToString(StringSoFar, ConvString)
END ConvertFrom;

BEGIN
PROCEDURE InsertCommand(Key: String; Mask: Word; Type: CARDINAL);
            (* Performs the algorithm detailed earlier, and returns result. *)
VAR
    Temp: TableType;
PROCEDURE InitialiseTable;
BEGIN
   Temp.Key        := Key;
   Temp.Kind       := Command;
   Temp.Mask       := Mask;
   Temp.Type       := Type;
   IF IsRoom() THEN
      IF IsIn(Key) THEN
          Raise(User, Error, InsertError)
     ELSE
         Insert(Temp)
     END
   ELSE
       Raise(User, Error, TableFullError)
   END
END InsertCommand;

PROCEDURE IsIn(Key: String): BOOLEAN;
PROCEDURE InsertSymbol(Key: String; Value: LONGINT; Status: SymbolStatus);
BEGIN
VAR
    Temp: TableType;

BEGIN
   Temp.Key        := Key;
   Temp.Kind       := Symbol;
   Temp.Value      := Value;
   Temp.Status     := Status;
   IF IsRoom() THEN
      IF IsIn(Key) THEN
          Raise(User, Error, InsertError)
     ELSE
         Insert(Temp)
     END
   ELSE
       Raise(User, Error, TableFullError)
   END
END InsertSymbol;

PROCEDURE Delete(Key: String);
(* I/O Operations. *)
BEGIN
    DeleteFromTree(HashTable[Hash(Key)], Key)
PROCEDURE GetRec(Input: FileHandle; KeyToInput: String; VAR NewRec: TableType);

VAR
```

```
   What            : String;
   BitPattern      : String;
   InputValue      : LONGINT;
   Prompts         : BOOLEAN;
END  Amend;
BEGIN
    InitialiseAString(What);
    InitialiseAString(BitPattern);
   IF Input = StdIn THEN
      Prompts := TRUE
   ELSE
      Prompts := FALSE
   END;
   WITH NewRec DO
      Key := KeyToInput;
      IF Prompts THEN
          WriteAString(StdOut, "Entry Type: ")
     END;
       ReadAString(Input, What);
       IF EqualStrings(What, "Command") THEN
        Kind := Command;
        IF Prompts THEN
            WriteAString(StdOut, "Bit Pattern: ")
       END;
          ReadAString(Input, BitPattern);
         Mask := ConvertTo(BitPattern);
        IF Prompts THEN
            WriteAString(StdOut, "Type: ")
       END;
         Type := ReadACard(Input)
       ELSIF EqualStrings(What, "Symbol") THEN
        Kind := Symbol;
        IF Prompts THEN
            WriteAString(StdOut, "Value: ")
       END;
          InputValue := ReadALongInt(Input);
        Value := InputValue;
        IF Prompts THEN
            WriteAString(StdOut, "Status: ")
       END;
         ReadAString(Input, What);
         IF      EqualStrings(What, "Absolute") THEN Status := Absolute
          ELSIF EqualStrings(What, "Relative") THEN Status := Relative
          ELSIF EqualStrings(What, "Register") THEN Status := Register
       ELSE
           Status := Absolute
       END
     END
   END
END GetRec;


PROCEDURE PutRec(Output: FileHandle; Record: TableType);

VAR
   Legends         : BOOLEAN;
   MaskString      : String;

BEGIN
   IF Output = StdOut THEN
      Legends := TRUE
   ELSE
      Legends := FALSE
   END;
```

```
   WITH Record DO
      IF Legends THEN
          WriteAString(StdOut, "Key: ")
     END;
       WriteAString(Output, Key);
       WriteALine(Output);
     IF Kind = Command THEN
        IF Legends THEN
             WriteAString(StdOut, "Kind: Command");
            WriteALine(StdOut);
             WriteAString(StdOut, "Bit Pattern: ");
       END;
          ConvertFrom(Mask, MaskString);
           WriteAString(Output, MaskString);
         WriteALine(Output);
        IF Legends THEN
            WriteAString(StdOut, "Type: ");
       END;
         WriteACard(StdOut, Type, 0);
         WriteALine(StdOut)
     ELSIF Kind = Symbol THEN
        IF Legends THEN
             WriteAString(StdOut, "Kind: Symbol");
            WriteALine(StdOut);
             WriteAString(StdOut, "Value: ")
       END;
          WriteALongInt(Output, Value, 0);
         WriteALine(Output);
        IF Legends THEN
            WriteAString(StdOut, "Status: ")
       END;
        CASE Status OF
            Absolute           : WriteAString(Output, "Absolute");
                                WriteALine(Output)                        |
            Relative           : WriteAString(Output, "Relative");
                                WriteALine(Output)                        |
            Register           : WriteAString(Output, "Regsiter");
                                 WriteALine(Output)
       END
     END
   END
END PutRec;


PROCEDURE TableExtShutdown;
END TableExtShutdown;


END TableExt.
```

```
IMPLEMENTATION  MODULE  TableTrees;

(*
          MODULE TableTrees         Version 1.0       (c) 1989 Mark Wickens

         Created:          02-12-89

           This library module implements a binary search tree.
           Each node of the tree data of type TableType, and the tree does not
           contain duplications.  It can handle any number of trees, the type of
           which is exported opaquely to user programs.
*)

FROM Table         IMPORT   TableType;
FROM Storage       IMPORT   ALLOCATE, DEALLOCATE, Available;
FROM Strings       IMPORT   AlphabeticallyLess, EqualStrings;

TYPE
   BST    = POINTER TO Node;
   Node   = RECORD
               Data          : TableType;
              Left,
              Right        : BST
           END;


PROCEDURE  InitialiseATree(VAR  Tree:  BST);
BEGIN
   Tree := NIL;
END  InitialiseATree;


PROCEDURE  IsRoomInTree():  BOOLEAN;
BEGIN
    RETURN  Available(SIZE(Node))
END  IsRoomInTree;


PROCEDURE  InsertInTree(VAR  Tree:  BST;  DataItem:  TableType);
BEGIN
   IF Tree <> NIL THEN
        IF  AlphabeticallyLess(DataItem.Key,  Tree^.Data.Key)  THEN
           InsertInTree(Tree^.Left,  DataItem)
        ELSIF  AlphabeticallyLess(Tree^.Data.Key,  DataItem.Key)  THEN
           InsertInTree(Tree^.Right,  DataItem)
     END
   ELSE
      NEW(Tree);
      WITH Tree^ DO
        Data        := DataItem;
        Left        := NIL;
        Right       := NIL
     END
   END
END  InsertInTree;


PROCEDURE  IsInTree(Tree:  BST;  Key:  String):  BOOLEAN;
BEGIN
   IF Tree <> NIL THEN
        IF  AlphabeticallyLess(Key,  Tree^.Data.Key)  THEN
          RETURN  IsInTree(Tree^.Left,  Key)
        ELSIF  AlphabeticallyLess(Tree^.Data.Key,  Key)  THEN
          RETURN  IsInTree(Tree^.Right,  Key)
```

```
     ELSE
        RETURN TRUE
     END
   ELSE
      RETURN FALSE
   END
END IsInTree;


PROCEDURE DeleteFromTree(VAR Tree: BST; Key: String);
    PROCEDURE DeleteSmallest(VAR P: BST; VAR Key: String);
   VAR
      Temp: BST;

   BEGIN
      IF P^.Left = NIL THEN
        Temp := P;
         Key := P^.Data.Key;
         P := P^.Right;
         DISPOSE(Temp)
     ELSE
          DeleteSmallest(P^.Left, Key)
     END
    END DeleteSmallest;

VAR
    TempData: TableType;
   Temp: BST;

BEGIN
    IF AlphabeticallyLess(Key, Tree^.Data.Key) THEN
        DeleteFromTree(Tree^.Left, Key)
    ELSIF AlphabeticallyLess(Tree^.Data.Key, Key) THEN
        DeleteFromTree(Tree^.Right, Key)
   ELSE
       IF (Tree^.Left = NIL) AND (Tree^.Right = NIL) THEN
        Temp := Tree;
         DISPOSE(Temp);
        Tree := NIL
      ELSIF Tree^.Right = NIL THEN
        Temp := Tree;
         Tree := Tree^.Left;
         DISPOSE(Tree)
      ELSIF Tree^.Left = NIL THEN
        Temp := Tree;
         Tree := Tree^.Right;
         DISPOSE(Tree)
     ELSE
            DeleteSmallest(Tree^.Right, TempData.Key);
         Tree^.Data := TempData
     END
   END
END DeleteFromTree;


PROCEDURE ElementsInTree(Tree: BST): CARDINAL;
VAR
    Counter: CARDINAL;

    PROCEDURE TraverseTreeCount(Tree: BST);
   BEGIN
      IF Tree <> NIL THEN
           TraverseTreeCount(Tree^.Left);
           TraverseTreeCount(Tree^.Right);
```

```
        INC(Counter)
     END
    END  TraverseTreeCount;

BEGIN
   Counter  :=  0;
    TraverseTreeCount(Tree);
   RETURN  Counter
END  ElementsInTree;


PROCEDURE  RetrieveFromTree(Tree:  BST;  Key:  String;
                                                        VAR DataItem: TableType);
BEGIN
   IF  Tree  <>  NIL  THEN
        IF  AlphabeticallyLess(Key,  Tree^.Data.Key)  THEN
           RetrieveFromTree(Tree^.Left,  Key,  DataItem)
        ELSIF  AlphabeticallyLess(Tree^.Data.Key,  Key)  THEN
           RetrieveFromTree(Tree^.Right,  Key,  DataItem)
     ELSE
         DataItem   :=  Tree^.Data
     END
   END
END  RetrieveFromTree;


PROCEDURE  NextInOrderInTree(Tree:  BST;  VAR  CurrentNext:  String;
                                                       Key: String);
    PROCEDURE  SearchTree(Tree:  BST);
   BEGIN
      IF  Tree  <>  NIL  THEN
           IF  AlphabeticallyLess(Tree^.Data.Key,  Key)  THEN
            SearchTree(Tree^.Right)
          ELSIF  (EqualStrings(CurrentNext,  Key)  AND
                   AlphabeticallyLess(Key,  Tree^.Data.Key))  OR
                    (AlphabeticallyLess(Tree^.Data.Key,  CurrentNext)  AND
                   NOT  EqualStrings(Tree^.Data.Key,  Key))  THEN
            CurrentNext  :=  Tree^.Data.Key;
            SearchTree(Tree^.Left)
       ELSE
            SearchTree(Tree^.Left)
       END
     END
    END  SearchTree;

BEGIN
    SearchTree(Tree)
END  NextInOrderInTree;


PROCEDURE  TableTreesShutdown;
END  TableTreesShutdown;


END  TableTrees.
```

A.3. Testing Modules.

This section of the appendix contains the testing modules for the sample implementation.

TestLex.MOD 116
TestStrings.MOD 118
TestTable.MOD 120

```
MODULE  TestLex;

(*
          MODULE: TestLex           Version 1.0      (c) 1989 Mark Wickens

           Created: 17-12-89

            This is the test module for the lexical scanner.
*)

FROM Lex            IMPORT   ScanLine, DecodedLine;
FROM  Strings       IMPORT    String, InitialiseAString, EqualStrings;
FROM IO            IMPORT   RdStr, WrCard;
FROM ADM            IMPORT   MAXOPERANDS, MAXINDICES;
FROM  Interface    IMPORT    StdInputName, StdOutputName, OpenAFile, CloseAFile,
                                ReadAString, WriteAString, FileHandle, WriteALine,
                            WriteAChar;

VAR
   UnLexed:  String;
   Lexed:  DecodedLine;
   Input,  Output:  FileHandle;

PROCEDURE  WriteDecoded(VAR  AI:  DecodedLine);

VAR
   OperandIndex,  IndicesIndex:  CARDINAL;

BEGIN
  WITH AI DO
      WriteAString(Output,  "Label:  ");
      WriteAString(Output,  Label);
      WriteALine(Output);

      WriteAString(Output,  "Command:  ");
      WriteAString(Output,  Command);
      WriteALine(Output);

      WriteAString(Output,  "Modifier:  ");
      WriteAChar(Output,  Modifier);
      WriteALine(Output);
      WriteALine(Output);

     FOR OperandIndex := 1 TO MAXOPERANDS DO
          WriteAString(Output,  "Operand:  ");  WrCard(OperandIndex,  2);
        WriteALine(Output);
        WriteAString(Output,  "   Prefix:  ");
          WriteAChar(Output,  Operand[OperandIndex].Prefix);
        WriteALine(Output);
        WriteAString(Output,  "   Argument:  ");
          WriteAString(Output,  Operand[OperandIndex].Argument);
        WriteALine(Output);

        FOR IndicesIndex := 1 TO MAXINDICES DO
          WriteAString(Output,  "   Index:  ");
              WriteAString(Output,  Operand[OperandIndex].Indices[IndicesIndex]);
          WriteALine(Output)
      END;
        WriteAString(Output,  "   Postfix:  ");
          WriteAChar(Output,  Operand[OperandIndex].Postfix);
        WriteALine(Output);
        WriteALine(Output);
    END;
```

```
        WriteAString(Output,  "Comment:  ");
        WriteAString(Output,  Comment);
       WriteALine(Output)
   END
END  WriteDecoded;


BEGIN
    Input  :=  OpenAFile(StdInputName);
    Output  :=  OpenAFile(StdOutputName);

    InitialiseAString(UnLexed);
    ReadAString(Input,  UnLexed);
    WHILE  NOT  EqualStrings(UnLexed,  "quit")  DO
       Lexed  :=  ScanLine(UnLexed);
        WriteDecoded(Lexed);
         InitialiseAString(UnLexed);
        ReadAString(Input,  UnLexed);
   END;

    CloseAFile(Input);
    CloseAFile(Output)
END  TestLex.
```

```
MODULE  TestStrings;

(*
          Module: TestStrings       Version 1.0       (c) 1989 Mark Wickens

          Created: 24-12-89

           This is a test module for the Strings Abstract Data Type.
*)

FROM  Interface    IMPORT    WriteAString,  WriteALine,  ReadACard,  WriteAChar,
                                StdOut, NextPrinting, ReadAString, WriteACard, StdIn;
FROM  Strings       IMPORT    String,  InitialiseAString,
                               CharInString, LengthOfString, MakeSubstring,
                               EmptyString, ReverseString, AlphabeticallyLess,
                               EqualStrings, ConcatStrings, CapitaliseString;

VAR
  S1, S2                            : String;
   Start, Length, Position          :  CARDINAL;
  Command                           : CHAR;

BEGIN
    InitialiseAString(S1);
    InitialiseAString(S2);

    WriteAString(StdOut,  'This  program  exercises  string  processing  operations');
    WriteALine(StdOut);

   REPEAT
       WriteALine(StdOut);
       WriteAString(StdOut,  'Type  in  the  first  string:  ');
       WriteALine(StdOut);
       ReadAString(StdIn,  S1);
       WriteAString(StdOut,  'Type  in  the  second  string:  ');
       WriteALine(StdOut);
       ReadAString(StdIn,  S2);
       WriteALine(StdOut);

       WriteAString(StdOut,  S1);
       WriteAString(StdOut,  '  was  the  first  string  typed  in');
       WriteALine(StdOut);
       WriteAString(StdOut,  S2);
       WriteAString(StdOut,  '  was  the  second  string  typed  in');
       WriteALine(StdOut);
       WriteALine(StdOut);

       ConcatStrings(S1,  S2);
       WriteAString(StdOut,  S1);
       WriteAString(StdOut,  '  is  first  and  second  string  concatanated.');
       WriteALine(StdOut);

       CapitaliseString(S1);
       WriteAString(StdOut,  S1);
       WriteAString(StdOut,  '  is  the  first  string  capitalised.');
       WriteALine(StdOut);
       WriteALine(StdOut);

       WriteAString(StdOut,  'What  substring  of  the  first  string  do  you  wish');
       WriteALine(StdOut);
       WriteAString(StdOut,  'to  assign  to  the  second  string?');
       WriteALine(StdOut);
       WriteAString(StdOut,  'Enter  the  start  position  of  the  substring:  ');
       Start  :=  ReadACard(StdIn);
```

```
      WriteALine(StdOut);
      WriteAString(StdOut, 'Enter the length of the substring: ');
      Length := ReadACard(StdIn);
      WriteALine(StdOut);
      MakeSubstring(S1, Start, Length, S2);

      IF EmptyString(S2) THEN
         WriteAString(StdOut, 'The second string is now empty');
    ELSE
         WriteAString(StdOut, S2);
         WriteAString(StdOut, ' is now the second string');
    END;
      WriteALine(StdOut);
      WriteALine(StdOut);

      IF AlphabeticallyLess(S1, S2) THEN
         WriteAString(StdOut, 'The first string is alphabetically less ');
         WriteAString(StdOut, 'than the second string');
      ELSIF EqualStrings(S1, S2) THEN
         WriteAString(StdOut, 'The two strings are identical')
    ELSE
         WriteAString(StdOut, 'The second string is alphabetically less ');
         WriteAString(StdOut, 'than the first string')
    END;
      WriteALine(StdOut);
      WriteALine(StdOut);

      WriteAString(StdOut, 'The length of the first string is: ');
       WriteACard(StdOut, LengthOfString(S1), 4);
      WriteALine(StdOut);
      WriteAString(StdOut, 'The length of the second string is: ');
       WriteACard(StdOut, LengthOfString(S2), 4);
      WriteALine(StdOut);

      WriteAString(StdOut, 'Which character of the first string do you wish ');
       WriteAString(StdOut, 'to retrieve?');
      WriteALine(StdOut);
      WriteAString(StdOut, 'Enter the position of the character: ');
      Position := ReadACard(StdIn);
      WriteALine(StdOut);
      WriteAString(StdOut, 'The character in this position is: ');
       WriteAChar(StdOut, CharInString(S1, Position));
      WriteALine(StdOut);
      WriteALine(StdOut);

      ReverseString(S1);
      WriteAString(StdOut, S1);
      WriteAString(StdOut, ' is now the first string, which has been reversed');
      WriteALine(StdOut);
      WriteALine(StdOut);

      WriteAString(StdOut, 'Type "c" to continue, "e" to exit the program: ');
      Command := NextPrinting(StdIn)
   UNTIL Command = 'e';

END TestStrings.
```

```
MODULE  TestTable;

(*
          MODULE: TestTable         Version 1.0       (c) 1989 Mark Wickens

          Created: 24-12-89
           Updated: 04-01-90          Tests modules Lex and Expression also.

           This program is the driver program for a suite of programs which
           together manage storage of, and access to, data in a table.
*)

FROM  Strings        IMPORT    String,  InitialiseAString,  EqualStrings;
FROM  Table          IMPORT    InitialiseTable,  IsRoom,  IsIn,  Insert,  Retrieve,
                               Delete,  Amend,  Greater,  NextInOrder,  SizeOfTable,
                                ElementsAtLocation,  TableType,  EntryType,  SymbolStatus;
FROM  TableExt      IMPORT    TableType,  EntryType,  SymbolStatus,  GetRec,  PutRec,
                             ConvertFrom;
FROM  Interface    IMPORT    WriteAChar,  WriteACard,  WriteAString,  WriteALine,
                                ReadAChar,  ReadAString,  ReadACard,  FileHandle,  StdIn,
                              StdOut,  WriteALongInt,  NextPrinting;
FROM  ADM             IMPORT    MAXOPERANDS,  MAXINDICES,  Instruction,  Word,
                                 MAXWORDSPERINSTRUCTION,  InsertOpcodesInTable,  Assemble;
FROM  Lex            IMPORT   ScanLine,  DecodedLine;
FROM  Expression  IMPORT    Evaluate,  ChangeBase,  CurrentBase,  BaseType;
FROM  Listing        IMPORT    StartListing,  StopListing,  SetPageLength,  SymbolTable,
                               SetLineLength,  AddLine,  INCLine,  INCPage,  SetTitle;


VAR
   Selection       :  CHAR;
   Name            :  String;
   Entry            :  TableType;
   Line            :  String;
   AssemInstr       :  DecodedLine;
   Result          :  LONGINT;
   ResultType       :  SymbolStatus;
   NewBase         :  BaseType;
   ICoun,  OCoun   :  CARDINAL;
   Counter         :  CARDINAL;
   MCInstr          :  Instruction;
   MCString        :  String;
   MCWord          :  CARDINAL;
   Number          :  CARDINAL;
   InstrLength     :  CARDINAL;
   PassNo          :  CARDINAL;

PROCEDURE  WriteLine(Line:  String);
BEGIN
    WriteAString(StdOut,  Line);
    WriteALine(StdOut)
END  WriteLine;


PROCEDURE  Instruct;
BEGIN
    WriteLine('Meta-Assembler  Module  Tester.');
    WriteLine('Developed  by  M  S  Wickens   (c)  1990');
    WriteALine(StdOut);
    WriteLine('Type  one  of:');
   WriteLine('I      to  add  a  new  entry  to  the  table');
   WriteLine('R       retrieve  details  of  an  entry');
   WriteLine('D      to  delete  an  entry  from  the  table');
   WriteLine("A      to  assemble  a  line  input  with  option  'S'");
```

```
   WriteLine('P      to display the contents of the table in order');
   WriteLine('N      to display the number of entries in each hash location');
   WriteLine('S      to input a line and perform a lexical scan');
   WriteLine('E      to input an expression and evaluate it.');
   WriteLine('C      change to the default base of constants in expressions.');
   WriteLine('L      test listing module.');
   WriteLine('?      to display this message');
   WriteLine('X      to leave the program')
END Instruct;


PROCEDURE Performance;
VAR
    PositionInTable: CARDINAL;
BEGIN
   WriteLine('Position        NoOfElements');
   FOR PositionInTable := 0 TO SizeOfTable() - 1 DO
       WriteACard(StdOut, PositionInTable, 8);
     WriteAString(StdOut, '       ');
       WriteACard(StdOut, ElementsAtLocation(PositionInTable), 8);
      WriteALine(StdOut)
  END
END Performance;


PROCEDURE WriteDecoded(VAR AI: DecodedLine);

VAR
    OperandIndex, IndicesIndex: CARDINAL;

BEGIN
  WITH AI DO
      WriteAString(StdOut, "Label: ");
      WriteAString(StdOut, Label);
      WriteALine(StdOut);

      WriteAString(StdOut, "Command: ");
      WriteAString(StdOut, Command);
      WriteALine(StdOut);

      WriteAString(StdOut, "Modifier: ");
      WriteAChar(StdOut, Modifier);
      WriteALine(StdOut);
      WriteALine(StdOut);

     FOR OperandIndex := 1 TO AI.NoOfOperands DO
          WriteAString(StdOut, "Operand: "); WriteACard(StdOut, OperandIndex, 2);
        WriteALine(StdOut);
        WriteAString(StdOut, "   Prefix: ");
          WriteAChar(StdOut, Operand[OperandIndex].Prefix);
        WriteALine(StdOut);
        WriteAString(StdOut, "   Argument: ");
          WriteAString(StdOut, Operand[OperandIndex].Argument);
        WriteALine(StdOut);

         FOR IndicesIndex := 1 TO AI.Operand[OperandIndex].NoOfIndices DO
          WriteAString(StdOut, "   Index: ");
              WriteAString(StdOut, Operand[OperandIndex].Indices[IndicesIndex]);
         WriteALine(StdOut)
      END;
        WriteAString(StdOut, "   Postfix: ");
          WriteAChar(StdOut, Operand[OperandIndex].Postfix);
        WriteALine(StdOut);
        WriteALine(StdOut);
```

```
     END;

        WriteAString(StdOut,  "Comment:  ");
        WriteAString(StdOut,  Comment);
       WriteALine(StdOut)
   END
END  WriteDecoded;


BEGIN
    InitialiseTable;
    InsertOpcodesInTable;
   Instruct;
    WriteAString(StdOut,  '>>  ');
    Selection  :=  CAP(NextPrinting(StdIn));
   WHILE Selection <> 'X' DO
      CASE Selection OF
        'I': IF IsRoom() THEN
                  WriteAString(StdOut, "Enter Key: ");
                 ReadAString(StdIn, Name);
               IF IsIn(Name) THEN
                   WriteAString(StdOut, Name);
                   WriteLine(' is already in the table');
              ELSE
                  GetRec(StdIn, Name, Entry);
                 Insert(Entry);
                  WriteAString(StdOut, Name);
                   WriteLine(' has been inserted in the table')
              END
            ELSE
                 WriteLine('Table is already full')
            END
     |
          'R': WriteAString(StdOut, "Enter Key: ");
               ReadAString(StdIn, Name);
              IF IsIn(Name) THEN
                 Entry := Retrieve(Name);
                 WriteLine('Details are:');
                 PutRec(StdOut, Entry)
            ELSE
                 WriteAString(StdOut, Name);
                 WriteLine(' is not in the table')
            END
     |
          'D': WriteAString(StdOut, "Enter Key: ");
               ReadAString(StdIn, Name);
              IF IsIn(Name) THEN
                Delete(Name);
                 WriteAString(StdOut, Name);
                 WriteLine(' has been deleted from the table')
            ELSE
                 WriteAString(StdOut, Name);
                 WriteLine(' is not in the table')
            END
     |
          'A': WriteAString(StdOut, "Enter Pass Number: ");
               PassNo := ReadACard(StdIn);
                 Assemble(AssemInstr, PassNo, MCInstr, InstrLength);
                WriteAString(StdOut, "Instruction Length: ");
               WriteACard(StdOut, InstrLength, 1);
              WriteALine(StdOut);
             IF PassNo = 2 THEN
                 FOR Counter := 1 TO InstrLength DO
                    WriteAString(StdOut, "MC: ");
```

```
                InitialiseAString(MCString);
                 ConvertFrom(MCInstr[Counter], MCString);
                WriteAString(StdOut, MCString);
               WriteAString(StdOut, " == ");
                   WriteALongInt(StdOut, LONGINT(INTEGER(MCInstr[Counter])), 1);
              WriteALine(StdOut)
          END
        END
|
     'P': InitialiseAString(Name);
         WHILE Greater(Name) DO
            NextInOrder(Name, Entry);
            PutRec(StdOut, Entry);
            WriteALine(StdOut);
           Name := Entry.Key
        END;
           WriteLine('End of list of table entries')
|
    'N': Performance()
|
     'S': InitialiseAString(Line);
          WriteLine("Enter ASSEM line: ");
          ReadAString(StdIn, Line);
          AssemInstr := ScanLine(Line);
          WriteDecoded(AssemInstr);
         WriteALine(StdOut);
|
     'E': InitialiseAString(Line);
          WriteLine("Enter an expression: ");
          ReadAString(StdIn, Line);
           Evaluate(Line, Result, ResultType);
           WriteAString(StdOut, "Result: ");
           WriteALongInt(StdOut, Result, 0);
         WriteALine(StdOut);
           WriteAString(StdOut, "Result Type: ");
         CASE ResultType OF
             Absolute:   WriteLine("Absolute") |
             Relative:   WriteLine("Relative")
        END;

|
     'C': WriteAString(StdOut, "Current Default: ");
         CASE CurrentBase() OF
            Bin: WriteLine("Binary")    |
           Oct: WriteLine("Octal")     |
            Dec: WriteLine("Decimal")   |
             Hex: WriteLine("Hexadecimal")
        END;
           WriteAString(StdOut, "New Default: ");
          InitialiseAString(Line);
          ReadAString(StdIn, Line);
           IF EqualStrings(Line, "Binary") THEN
            ChangeBase(Bin)
           ELSIF EqualStrings(Line, "Octal") THEN
            ChangeBase(Oct)
           ELSIF EqualStrings(Line, "Decimal") THEN
            ChangeBase(Dec)
            ELSIF EqualStrings(Line, "Hexadecimal") THEN
            ChangeBase(Hex)
        ELSE
             WriteLine("Base not recognised.")
        END
|
     'L': InitialiseAString(Line);
```

```
              WriteLine("Enter line to list: ");
              ReadAString(StdIn, Line);
              FOR Counter := 1 TO MAXWORDSPERINSTRUCTION DO
                 WriteAString(StdOut, "Enter MC for Word: ");
                MCWord := ReadACard(StdIn);
                 MCInstr[Counter] := Word(MCWord);
            END;
               WriteAString(StdOut, "Enter number of lines: ");
              Number := ReadACard(StdIn);

              SetTitle("This is the title.");
               StartListing("LIST.LST");
              SetLineLength(132);
              SymbolTable(TRUE);
             FOR Counter := 1 TO (Number * 3) BY 3 DO
                  AddLine(LONGCARD(Counter), MCInstr, 3, Line)
            END;
             StopListing
    |
       '?': Instruct

      ELSE     WriteLine('Illegal instruction.   Type "?" for help')
     END;
       WriteAString(StdOut, '>> ');
        Selection := CAP(NextPrinting(StdIn));
   END;
END TestTable.
```

A.4. A Sample Run.

Command Line:

```
C:\M2>M2Assem Input
Modula-2 Meta-Assembler Version 1.0 (c)1990 Mark Wickens
Assembling: Pass 1
Assembling: Pass 2
Assembled 11 lines at a rate of 36 lines per minute.
There were 0 exception(s) raised during assembly.
A valid object code file was written.
```

Input File:

```
                OBJECT  1
                SYMBOL  1
                ORG     1000
                BASE    16
Start           EQU     25
                BASE    D'10
End             EQU     25-10
LABEL4:         NOP
LABEL3:         ADDC    .D1,.D4
LABEL2:         SHL.L   #5,.D4
LABEL1:         BRK     #10
```

Object Code File:

```
     3E8 4E71
     3E9 D941
     3EA EB8C
     3EB 4E4A
```

Listing File:

```
Page:   1   MetaASSEM  (c) 1990 M S Wickens  Version 1.0  20/ 4/1990  11:59:57


   1         0                                                OBJECT  1
   2         0                                                SYMBOL  1
   3       3E8                                                ORG     1000
   4       3E8                                                BASE    16
   5       3E8                             Start              EQU     25
   6       3E8                                                BASE    D'10
   7       3E8                             End                EQU     25-10
   8       3E8  4E71                       LABEL4:            NOP
   9       3E9  D941                       LABEL3:            ADDC    .D1,.D4
  10       3EA  EB8C                       LABEL2:            SHL.L   #5,.D4
  11       3EB  4E4A                       LABEL1:            BRK     #10


Symbol Table Dump:

      15  Abs  End
    1003  Rel  LABEL1
    1002  Rel  LABEL2
    1001  Rel  LABEL3
    1000  Rel  LABEL4
      37  Abs  Start

Assembly Statistics:
Assembly Time:  0: 0:20
Assembly Rate:       36 lines per minute
```